Additional document containing papers, previous research experience reports and coursework projects.

Contents

1 Research Intenrship: Uncertainty Quantification for 3D Human Pose Estimation 2

2 Ethics4EU Poster: Is GPT-3 The Type of AI We Are Looking For From Ethical Perspective ? 42

3 Joint Unsupervised IR-RGB Video Registration and Fusion 44

4 Covid Tweets Prediction: Course project X-INF554 50

5 Multi Echelon Supply Chain Optimization with Reinforcement Learning: Project Course X-INF581 57

6 Sustainable Cities and Human Settlements: Project Course Logics & Symbolic AI 66

1 Research Intenrship: Uncertainty Quantification for 3D Human Pose Estimation

Final report for my internship at U2IS Laboratory, ENSTA Paris.

INSTITUT POLYTECHNIQUE DE PARIS

Research Internship Report

Computer Science
Data&AI Masters Degree
Course INF-592

Uncertainty Quantification for 3D Human Pose Estimation

Intern :
Imad Eddine MAROUF

Internship Tutors :
Pr. A. Manzanera
Pr. G. Franchi

Unite d'Informatique et d'Ingenierie des Systemes (U2IS)

ENSTA
IP PARIS

11 novembre 2021

Summary

1 Acknowledgements **3**

2 Glossary **4**

3 ABSTRACT **5**

4 Research Laboratory Presentation **6**

5 Introduction **7**
5.1 Internship Theme . . . 7

6 Related Work **8**
6.1 3D/2D Human Pose Estimation . . . 8
6.2 Datasets . . . 9
6.3 Evaluation Metrics . . . 10
6.4 Uncertainty Estimation . . . 11
6.4.1 Estimating Uncertainty Methods . . . 11
6.4.2 Uncertainty Quantification Metrics . . . 12
6.5 Uncertainty Quantification for 3D Human Pose Estimation . . . 12

7 Backbone Model : **13**
7.1 Pose Estimation : PoseNet . . . 13

8 Contribution : **16**
8.1 Proposed Method for Uncertainty Quantification . . . 16
8.1.1 Heatmaps Variance . . . 16
8.1.2 Human Pose Clustering : K-Medoids . . . 16

9 Experiments And Analysis **18**
9.1 Ablation Study . . . 18
9.1.1 Deep Ensembles : . . . 18
9.1.2 MC Dropout : . . . 18
9.1.3 Log-likelihood maximization (Log) : . . . 20
9.1.4 Regularized heatmaps : . . . 21
9.2 3D HPE with K-Medoids . . . 24
9.3 Uncertainty with K-Medoids . . . 28
9.4 Out-of-Distribution Detection . . . 30

10 Limitations And Challenges **31**

11 Conclusion **32**

12 Appendices **33**
12.1 Appendix A : K-Medoids Clustering . . . 33
12.2 Appendix B : Regularised Heatmaps . . . 34

Bibliographie **37**

Acknowledgements

I would like to thank my internship Supervisors **Pr. Gianni Franchi, and Pr. Antoine Manzanera** for the fruitful and useful discussion all along the internship period. As well, their advices and guidance for future research endeavor.

Finally, I cannot forget to thank all the **U2IS** Team members : Pascal Caroline, Fnadi Mohammed, and interns : Hu Yufie, Yan Chen and Julien for a cherished time spent together in the lab

2 Glossary

Term	Description
CNN	Convolutional Neural Network
DNN	Deep Neural Network
GNN	Graph Neural Network
BNN	Bayesian Neural Network
DNN	Deep Neural Network
AUSE	Area Under Sparsification Curve
AUCE	Area Under Calibration Curve
HPE	Human Pose Estimation
QU	Quantify Uncetainty
IPR	Integral Pose Regression: Method transforms heatmaps into joint location coordinates as: $J_k = \int_{p \in \Omega} p \times \tilde{H}_k(p)$ where $\tilde{H}_k$ is the normalized heatmaps and Ω is its domain.
OOD	Out-Of-Distribution
IND	In-Distribution
$\mathcal{L}_{train}$	Training Loss.
$\mathcal{L}_{test}$	Testing Loss.
MPJPE	Mean Per Joint Position Error
P-MPJPE	Mean Per Joint Position Error after Procrustes Alignment
3DPCK	3D Percentage of Correct Key-points
PCA	Principal Component Analysis
GT	Ground-truth

Table 1. Terms and formula used in the report.

ABSTRACT

Today, one of the major challenges in artificial intelligence applications is to develop reliable and certain systems while producing remarkable results. Deep learning became indispensable in this kind of task. It freed us from features engineering. However, relying on only accuracy metrics as our end goal for high-stack applications is not sufficient. Deploying this kind of model requires estimating their strengths and pitfalls, whether they are certain about the outputs given. Human Pose estimation has been studied intensively. Recent frameworks using deep learning have achieved astonishing results, but neural networks cannot measure the uncertainty about their outputs. Few works have investigated this perspective in human pose estimation.

Based on this motivation, we investigate the uncertainty quantification in 3D Human Pose estimation by conducting different experiments using well-known methods related to Uncertainty estimation in deep learning. Furthermore, we propose a new approach to estimate the model's predicted pose's reliability by using a memory bank of poses used during training.

Keywords : Machine Learning, Deep Learning, Human Pose Estimation, Uncertainty Estimation, Human Poses Clustering, PoseNet

4 Research Laboratory Presentation

The ENSTA Paris, also known as the **École nationale supérieure de techniques avancées** (English : Superior National School of Advanced Techniques) is a prestigious French graduate school of engineering ("école d'ingénieurs"). Founded in 1741, it is the oldest "grande école" in France. It is located in Palaiseau in the south of Paris, on the Paris-Saclay campus, and is a constituent faculty of the Polytechnic Institute of Paris. Every year some 180 engineers graduate from the school.

ENSTA affords its students a general training course in engineering. To this end, the school provides high-level scientific and technological training. Research professors give the teaching with the participation of numerous auxiliary teachers from the economic and industrial world familiar with the latest technical developments in a wide variety of fields. The general nature of the instruction enables ENSTA's students to find a career in various sectors such as the automotive or naval industry, networks and telecommunications, space propulsion, robotics, oceanology and the environment. [1]

Computer Science and Systems Engineering Laboratory (U2IS) is developing research to integrate reliable autonomous systems decision-making processes with intelligent transport, robotics, defense, and energy applications. The laboratory brings together the research activities of the ENSTA Paris School in computer science, robotics, vision, embedded systems, signal and image processing, and hybrid system design and analysis. [2]

The laboratory is focusing on application-oriented research. Even if they can come from fundamental fields, most of its work's motivation is to be integrated into products (components, robotic systems, design, or validation tools) that are evaluated finally by the industry and can give place to technology transfers.

The laboratory maintains close and privileged relations with numerous laboratories located in the Saclay area, in France, and abroad. They can be informal, in the frame of collaborative projects, or result in the creation of joint teams, as is the case with INRIA Saclay (Grace Project Team) and INRIA Bordeaux (Flowers Project Team).

`Source` : ENSTA/U2IS website

1. https://www.ensta-paris.fr/en
2. http://u2is.ensta-paris.fr/

5 Introduction

The quantification of the uncertainty in robotics and computer vision is critical to developing more robust and reliable systems. Human Pose estimators are achieving SOTA results on benchmarked datasets. But they do not usually estimate uncertainties. Higher-level decision blocks using them have no means if we cannot know when we can rely on these estimators.

Uncertainty is often present in the post-analysis of a model, but it is scarce in Deep Learning approaches. Furthermore, Deep Learning approaches tend to overfit standard datasets. This might lead us to assume an excellent general performance while they are strongly biased. In such cases, Deep Neural Network's (DNN) outputs should not be trusted in real-world applications.

5.1 Internship Theme

Goal The aim of the work is to perform a study on the uncertainty of the different approaches, evaluate their reliability and propose techniques to improve them.

Methodology In this work, we evaluate the two different sources of uncertainty –epistemic and aleatoric– in 3D Human intending to determine the quality of the used model predictions and hence its potential for real-life applications. First, we provide a unified framework and a thorough evaluation of scalable uncertainty estimation approaches, namely Monte Carlo (MC) dropout, and Deep Ensembles. Secondly, we prove that applying MC Dropout in the encoder or decoder affects the learning behavior of the model contrary to recent works done [45], [17] -dropout in the encoder achieves a better performance than the dropout in the decoder. In addition, it has better uncertainty quantification scores than other approaches-. Deep Ensembles with the log likelihood-maximization loss provide the best uncertainty estimation.

Finally, we propose a method to give confidence scores for the predicted human pose. This technique relies on measuring the predicted pose's distance to a memory of 3D human poses that synthesize the training set distribution.

- What are the motivations behind this project ?
  - **Reliability** : 3D HPE has various applications in many important fields. Therefore, having an accurate model capable of expressing its uncertainties is a must before being deployed in the real world.
  - **Exploration/Investigation** : Unlike various sub-fields in deep learning applications, uncertainty estimation has been widely studied by different stunning research works such as Optical Flow, Depth Estimation [section 6.4]. On the other hand, little to no research work has been done on the quantifying uncertainty of DNNs for 3D human pose estimation [section 6.5].

Figure 1. Some samples of multi-person pose estimation

6 Related Work

6.1 3D/2D Human Pose Estimation

HPE is generally regarded as the task of predicting the articulated joint locations of a human body from an image or a sequence of images of that person. Due to its wide range of potential applications, human pose estimation is a fundamental and active research direction in computer vision. Driven by powerful deep learning techniques and recently collected large-scale datasets, human pose estimation has continued making significant progress, especially on 2D images.

Since 3D pose representation provides additional depth information compared with 2D pose representation, 3D human pose estimation enables more widespread applications. To better understand the use of 3D human pose estimation, we provide a brief description of some of its interesting real-world applications :

- **Human-Computer Interaction** : A robot can better serve and help users if it can understand 3D poses, actions, and emotions of people. For example, a robot can take timely actions when it detects the 3D pose of a person who is prone to fall. In addition, assistant robots can better socially interact with human users, provided they can perceive 3D human poses. Meanwhile, it is also very useful for computer control,i.e.as, input for productive software packages. Moreover, people can play games using their poses and gestures through Microsoft Kinect sensors.
- **Biomechanics** : Human pose and movement can indicate the health status of humans. Thus, 3D pose estimation techniques could be used to construct a sitting posture correction system to monitor the status of users. For exercise, the system can be used to avoid injury by providing timely feedback of correct movement poses to users. Moreover, pose estimation systems are also able to assist doctors for remote to diagnose, and telerehabilitation of patients [1].
- **Others** : 3D HPE can also be used to assist other computer vision tasks such as pose transfer [28], action recognition [33], person image generation [49].

Two common approaches are employed in estimating the poses of individuals in a given image :

- **Top-down approaches** : The processing is done from low to high resolutions, follow the detection of the individual instances in the image first using a bounding box object detector and then focus on determining their poses next [43], [53].
  These approaches always suffer from early commitment, which means if the detection of individuals fails, there is no possibility of recovering. Also, it is vulnerable when multiple individuals are nearby. Furthermore, the computational cost depends on the number of people in the image. Hence, the run-time of these approaches is directly proportional to the number of people : means for every detection, a single-person pose estimator is run.
- **Bottom-Up approaches** : The processing is done from high to low resolutions. It starts by localizing identity-free semantic parts and then grouping them into person instances. Bottom-up approaches overcame early commitment and exhibited a run-time complexity detached from the number of people in the image. In addition to that, some research using bottom-up approaches uses global contextual cues from other body parts and other people. However, bottom-up approaches face challenges in grouping body parts when there is a large overlap between people [54], [4], [7].

DeepPose[52] is one of the earliest CNN-based models for human pose estimation. The model architecture is mostly convolutional, with fully connected output layers used to

predict the joint coordinates directly as numerical values. DeepPose is based on a cascaded design, where predicted coordinates are repeatedly refined to produce more accurate results. The basic concept of cascading has been carried forward into more recent 2D pose estimation architectures, such as Stacked Hourglass [43]. The Stacked Hourglass architecture makes extensive use of residual connections [19] and is segmented into multiple stages, which permit intermediate supervision to guide training. The high accuracy of Stacked Hourglass—as demonstrated by practical results on the MPII Human Pose dataset —has inspired subsequent work to build upon the architecture directly [8], [55].

The overwhelming majority of 2D pose estimation models developed after DeepPose (including Stacked Hourglass) have ceased using fully connected output. Indeed, most of them are base on a heatmap matching approach [51]. Heatmap matching works by training a model to output spatial maps which indicate joint locations with high-valued pixels. During training, a loss is calculated using the mean squared error between the output heatmap and an ideal target, 2D spherical Gaussian mean-centred on the ground truth joint location. During inference, numerical coordinates are calculated from heatmaps through the use of a non-differentiable argmax operation. In some implementations, the pixels neighboring the highest-valued pixel are also considered to make minor sub-pixel adjustments for higher precision results [43].

Recently, soft-argmax [5], [50] has emerged as a differentiable alternative to argmax for heatmap-based human pose estimation models [34]. Since it is possible to backpropagate through the soft-argmax operation, a distance-based loss is applied directly to the predicted coordinates.

Some works focus on lifting the 2D human pose to the 3D space. Benefiting from the excellent performance of state-of-the-art 2D pose detectors, 2D to 3D lifting approaches generally outperform approaches directly estimating the 3D pose. In the first stage, off-the-shelf 2D HPE models are employed to estimate 2D pose, and then in the second stage, a 2D to 3D lifting is used to obtain a 3D pose. Martinez et al. [36] introduced a simple network that consists of consecutive fully connected layers, which lifts the 2D human pose to the 3D space. Zhao et al. [56] developed a semantic GraphCNN to use spatial relationships between joint coordinates.

6.2 Datasets

In this section, we present the different datasets used for HPE. Many datasets have been proposed to evaluate and compare results based on different metrics. In this section, we will focus on the main one :

- **Human3.6M Dataset** [22] [3] is the largest 3D single-person pose benchmark. It consists of 3.6 million video frames. 11 subjects performing 15 activities are captured from 4 camera viewpoints. The ground-truth 3D poses are obtained using a motion capture system.

3. http://vision.imar.ro/human3.6m/description.php

Figure 2. Human3.6M dataset consists of 3.6 million different human poses collected with 4 digital cameras. It contains 15 training motions such as walking , sitting and laying down poses.

- **MuCo-3DHP/MuPoTS Dataset** : These are the 3D multi-person pose estimation datasets proposed by Mehta et al [38]. The training set MuCo-3DHP is generated by per-camera composites with 1 to 4 subjects are first generated in the images randomly selected from the MPI-INF-3DHP dataset [37], in which each camera has 16 sequences. The composited dataset covers many kinds of inter-person overlaps and activities. MuPoTS-3D (Multiperson Pose Test Set in 3D)[4] comprises 20 general real-world scenes (5 indoor and 15 outdoor) for up to three subjects with challenging elements such as drastic illuminations and lens flares for outdoor settings.

Figure 3. Samples from datasets : MuCo (1st and 2nd), MuPoTS (3rd and 4th)

6.3 Evaluation Metrics

- **MPJPE** is the Euclidean distance between ground-truth and prediction for a joint by the model, after aligning the root joints (typically the *pelvis*) of the estimated and ground-truth 3D pose. Mean per joint position error is the mean of per joint position error for all k joints (Typically, k = 16) -Lower score is better-.

$$Error_{MPJPE}(f,S) = \frac{1}{N_S}\sum_{i=1}^{N_S} ||P^f_{f,S}(i) - P^f_{gt,S}(i)||_2 \quad (1)$$

4. http://gvv.mpi-inf.mpg.de/projects/SingleShotMultiPerson/

where f denotes a frame and S denotes the corresponding skeleton. $P^f_{f,S}(i)$ is the estimated position of joint i and $P^f_{gt,S}(i)$ is the corresponding ground-truth position. Finally, the MPJPEs are averaged over all the frames.

- **PA-MPJPE** similar to **MPJPE**, but the euclidean distance is calculated after aligning the two poses coordinates using Procrustes analysis [14]. Procrustes method is simply a similarity transformation to have scale, rotation, and translation invariant shapes.
- **3DPCK** Predicted joint is considered correct if the distance between the predicted and the true joint is within a certain threshold (e.g.,150 mm).

6.4 Uncertainty Estimation

Uncertainty is decomposed into three types [25] :

- **Approximation** : due to the fact that the model is not sufficiently expressive to model the data-to-label association.
- **Aleatoric** : due to the intrinsic stochastic nature of the association. Aleatoric uncertainty captures noise inherent in the observations. This could be sensor noise or motion noise, resulting in uncertainty that cannot be reduced even if more data were to be collected.

  Aleatoric uncertainty can further be categorized into :
  - ☐ `homoscedastic uncertainty` : uncertainty which stays constant for different inputs.
  - ☐ `heteroscedastic uncertainty` : Heteroscedastic uncertainty depends on the inputs.
- **Epistemic** : accounts for uncertainty in the model parameters – uncertainty, which captures our ignorance about which model generated our collected data. This uncertainty can be explained away given enough data, and is often referred to as model uncertainty.

Traditionally, Bayesian Neural Networks were used to model epistemic uncertainty through probability distributions over the model parameters. However, these distributions are often intractable and researchers have proposed interesting solutions to perform approximate Bayesian inference to measure uncertainty, including Variational Inference [15], [3], [46] and Deep Ensembles [27]. Alternatively, Gal et al. [12], [13] showed that applying dropout [12] at inference time yields a form of variational inference where parameters of the network are modeled as a mixture of multivariate Gaussian distributions with small variances. This technique, called Monte Carlo (MC) dropout, became popular also due to its adaptability to non-probabilistic deep learning frameworks. In computer vision, uncertainty estimation using MC dropout has been applied for depth regression tasks [23], scene segmentation [41], [23] and, more recently, LiDAR 3D object detection for cars [11].

6.4.1 Estimating Uncertainty Methods

This class of methods aims at encoding uncertainty empirically, for instance, by measuring the variance between a set of all the possible network configurations. It allows explaining the model epistemic uncertainty.

1- Monte Carlo Dropout : Gal et al [12] represent a popular method to sample N independent models without requiring multiple and independent trainings. At training time, connections between layers are randomly dropped with a probability p to avoid overfitting. At test time, all connections are kept. By keeping dropout enabled at test time, we can perform multiple forwards sampling a different network every time. Empirical mean μ and variance σ are computed, as follows, performing multiple M inferences :

$$\mu_u = \frac{1}{M}\sum_{i=1}^{M} u_{w_x}(x) \tag{2}$$

$$\sigma_u^2 = \frac{1}{M}\sum_{i=1}^{M}(u_{w_i}(x) - \mu_u)^2 \tag{3}$$

2- Deep Ensembles [27] : is a technique proposed by Lakshminarayanan *et al*consisting of training M instances of the same architecture and then averaging their output.

Mean and variance are calculated in the same manner as equations 2, 3.

3- Log-Likelihood Maximization (Log) [23] : is a technique proposed by Kendall *et al*that considers that the output of the DNN follows a Gaussian distribution. Hence their goal is to infer the mean and the variance of the distribution. The network is trained by log-likelihood maximization. In some papers[20], they consider that the distribution is Laplace distribution. In our case, the loss will be :

$$\zeta_{Laplace} = \frac{|\mu_d - y|}{\sigma^2} + \log \sigma^2 \tag{4}$$

$$\zeta_{Gaussian} = \frac{|\mu_d - y|}{2\sigma^2} + \frac{1}{2}\log \sigma^2 \tag{5}$$

where y and μ_d and σ are the predicted output, the ground-truth coordinates and variance, respectively.

6.4.2 Uncertainty Quantification Metrics

1- Sparsification Curve -AUSE- : is a technique proposed in [26] that checks if the confidence of the prediction is linked to its error. The error can be capture by any metrics such as **MAE** or **RMSE**). The goal is that if we erase the less confident predictions, the erased predictions should also have the more important error.

2- Spearman's Correlation Score [47] : The Spearman's rank correlation coefficient is a nonparametric measure of rank correlation, i.e., statistical dependence between the rankings of two variables. It assesses how well the relationship between two variables can be described using a monotonic function. Compared to the classic *Pearson correlation*, the Spearman's rank correlation supports any nonlinear dependence and it is also more robust.

Thus, Spearman's score will be used as a measurement to characterize the correlation between the pose estimation error $error(p, \tilde{p})$, and the variance of the predicted pose coordinates. The goal is to have a strong positive correlation between them.

6.5 Uncertainty Quantification for 3D Human Pose Estimation

To the best of our knowledge, QU for 3D HPE literature is limited. Unlike, other disciplines in computer vision such as depth estimation [23] and semantic segmentation [41]. Bertoni et al [2] incorporated a Laplacian loss [23] to estimate aleatoric uncertainty for each predicted distance location of the person, and MC dropout at inference time is used to capture epistemic uncertainty. Alternatively, Gundavarapu et al. [16] introduced a novel formulation to capture aleatoric uncertainty in human pose using a multivariate Gaussian distribution over all the joints of the human body, and showing that it improves generalization in 2D human pose estimation. While Guanya et al. [48] adapted deep ensembles [27] to compute their average pairwise disagreement against one another to obtain the uncertainty quantification for Object Pose Estimation.

7 Backbone Model :

7.1 Pose Estimation : PoseNet

We performed our experiments with the `PoseNet` neural network architecture proposed by Moon et al. [40] published on ICCV2019 Conference [5]. We decided to use an existing method rather than developing an entirely new approach because the focus of our work was uncertainty estimation for 3D human pose prediction and proposing improvements that can be applied to regression architectures of human pose estimation.

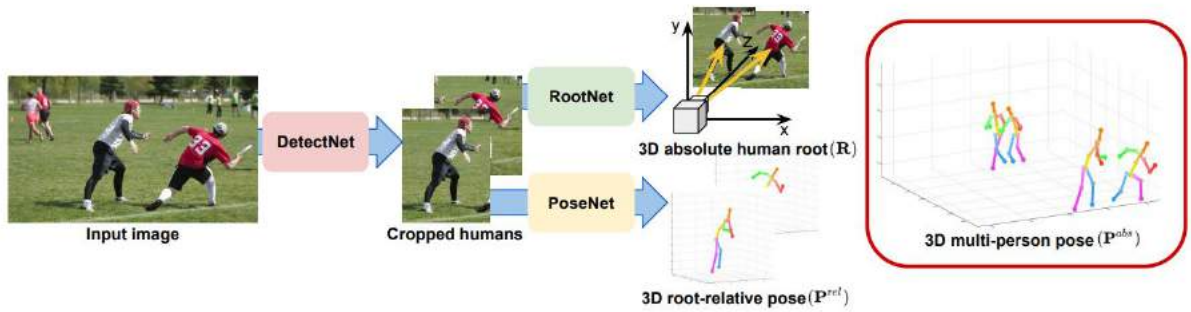


Figure 4. Illustration of Moon's framework, There are 3 main components of 3D human pose estimation pipeline, namely *DetectNet*, *RootNet* and *PoseNet*. In order to provide camera-aware 3D joint's coordinates

Moon et al [40] propose a general framework for 3D multi-person pose estimation. To the best of our knowledge, this study is the first to propose a fully learning-based camera distance-aware top-down approach of which components are compatible with most of the previous human detection and 3D human pose estimation methods.

The goal of Moon's framework is to recover the absolute camera-centered coordinates of multiple persons' keypoints $(P_j^{abs})_J^{j=1}$, where J denotes the number of joints. To achieve that, they constructed a framework consists of three modules.

- **DetectNet** : a human detection network (DetectNet) detects the bounding boxes of humans in an input image, detects a human bounding box of each person in the input image.

  Mask R-CNN [18] is used as the framework of DetectNet. Mask R-CNN [18] consists of three parts. The first one, backbone, extracts useful local and global features from the input image by using deep residual network (ResNet) [19] and feature pyramid network [30]. The second part, the region proposal network, proposes human bounding box candidates based on the extracted features.

  The RoIAlign layer extracts the features of each proposal and passes them to the third part, which is the classification head network. The head network determines whether the given proposal is a human or not and estimates the bounding box refinement offsets. It achieves the state-of the-art performance on publicly available object detection datasets [31].
- **RootNet** : the proposed 3D human root localization network (RootNet) estimates the camera-centered coordinates of the detected humans' roots, takes the cropped human image from the DetectNet, and localizes the root of the human $R = (x_R, y_R, Z_R)$, in which x_R and y_R are pixel coordinates, and Z_R is an absolute depth value.
- **PoseNet** : a root-relative 3D single-person pose estimation network (PoseNet) estimates the root-relative 3D pose for each detected human. A cropped human image is fed to

5. The International Conference on Computer Vision (ICCV) is the premier gathering of professionals dedicated to the advancements of the branch computer vision.

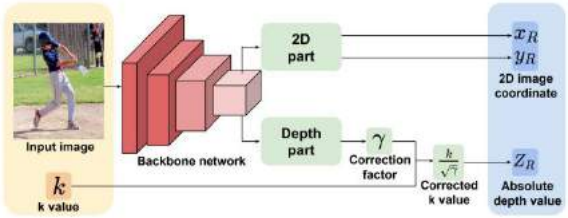


Figure 5. Network architecture of *RootNet*. RootNet estimates the human body root coordinates.

the *PoseNet*, which estimates the root-relative 3D pose $P_j^{rel} = (x_j, y_j, Z_j^{rel})$, in which x_j and y_j are pixel coordinates in the cropped image space and Z_j^{rel} is root-relative depth value. We convert Z_j^{rel} into Z_j^{abs} by adding Z_R and transform x_j and y_j to the original input image space. Then, the final absolute 3D pose $P_j^{abs}{}_{J}^{j=1}$ is obtained by simple back-projection.

This component consists of two parts. The first part is the backbone, which extracts a useful global feature from the cropped human image using ResNet [19]. Second, the pose estimation part takes a feature map from the backbone part and upsamples it using three consecutive deconvolutional layers with batch normalization layers [21] and ReLU activation function. A 1-by-1 convolution is applied to the upsampled feature map to produce the 3D heatmaps for each joint. The soft-argmax operation is used to extract the 2D image coordinates (x_j, y_j), and the root-relative depth values Z_j^{rel}.

Note : Our study focus mainly on *PoseNet*, while using the published bounding boxes, and depth-root values from *DetectNet* and *RootNet* of the used dataset, respectively.

Motivation `PoseNet` proposes a top-down approach with convolutional neural network CNN consisting of decoder and encoder for efficient 3D pose estimation based on regression on heatmaps for each joint of the human pose. To the best of our knowledge, scarce research work has been done on GNN uncertainty estimation.

Figure 16 compare PoseNET to SOTA frameworks for 3D Pose Estimation on Human3.6M dataset, while figure 16 shows a comparison with respect to MuPoTS dataset.

Methods	Dir.	Dis.	Eat	Gre.	Phon.	Pose	Pur.	Sit	SitD.	Smo.	Phot.	Wait	Walk	WalkD.	WalkP.	Avg
With groundtruth information in inference time																
Yasin [50]	88.4	72.5	108.5	110.2	97.1	81.6	107.2	119.0	170.8	108.2	142.5	86.9	92.1	165.7	102.0	108.3
Chen [5]	71.6	66.6	74.7	79.1	70.1	67.6	89.3	90.7	195.6	83.5	93.3	71.2	55.7	85.9	62.5	82.7
Moreno [32]	67.4	63.8	87.2	73.9	71.5	69.9	65.1	71.7	98.6	81.3	93.3	74.6	76.5	77.7	74.6	76.5
Zhou [53]	47.9	48.8	52.7	55.0	56.8	49.0	45.5	60.8	81.1	53.7	65.5	51.6	50.4	54.8	55.9	55.3
Martinez [26]	39.5	43.2	46.4	47.0	51.0	41.4	40.6	56.5	69.4	49.2	56.0	45.0	38.0	49.5	43.1	47.7
Sun [43]	42.1	44.3	45.0	45.4	51.5	43.2	41.3	59.3	73.3	51.0	53.0	44.0	38.3	48.0	44.8	48.3
Fang [7]	38.2	41.7	43.7	44.9	48.5	40.2	38.2	54.5	64.4	47.2	55.3	44.3	36.7	47.3	41.7	45.7
Sun [44]	36.9	36.2	40.6	40.4	41.9	34.9	35.7	50.1	59.4	40.4	44.9	39.0	30.8	39.8	36.7	40.6
Ours (PoseNet)	**31.0**	**30.6**	**39.9**	**35.5**	**34.8**	**30.2**	**32.1**	**35.0**	**43.8**	**35.7**	**37.6**	**30.1**	**24.6**	**35.7**	**29.3**	**34.0**
Without groundtruth information in inference time																
Rogez [41]*	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	42.7
Ours (Full)	**32.5**	**31.5**	**41.5**	**36.7**	**36.3**	**31.9**	**33.2**	**36.5**	**44.4**	**36.7**	**38.7**	**31.2**	**25.6**	**37.1**	**30.5**	**35.2**

Figure 6. Comparison with state-of-the-art human pose estimators on Human3.6M using P-MPJPE (lower score is better).

Methods	Dir.	Dis.	Eat	Gre.	Phon.	Pose	Pur.	Sit	SitD.	Smo.	Phot.	Wait	Walk	WalkD.	WalkP.	Avg
With groundtruth information in inference time																
Chen [5]	89.9	97.6	90.0	107.9	107.3	93.6	136.1	133.1	240.1	106.7	139.2	106.2	87.0	114.1	90.6	114.2
Tome [46]	65.0	73.5	76.8	86.4	86.3	68.9	74.8	110.2	173.9	85.0	110.7	85.8	71.4	86.3	73.1	88.4
Moreno [32]	69.5	80.2	78.2	87.0	100.8	76.0	69.7	104.7	113.9	89.7	102.7	98.5	79.2	82.4	77.2	87.3
Zhou [53]	68.7	74.8	67.8	76.4	76.3	84.0	70.2	88.0	113.8	78.0	98.4	90.1	62.6	75.1	73.6	79.9
Jahangiri [17]	74.4	66.7	67.9	75.2	77.3	70.6	64.5	95.6	127.3	79.6	79.1	73.4	67.4	71.8	72.8	77.6
Mehta [28]	57.5	68.6	59.6	67.3	78.1	56.9	69.1	98.0	117.5	69.5	82.4	68.0	55.3	76.5	61.4	72.9
Martinez [26]	51.8	56.2	58.1	59.0	69.5	55.2	58.1	74.0	94.6	62.3	78.4	59.1	49.5	65.1	52.4	62.9
Fang [7]	50.1	54.3	57.0	57.1	66.6	53.4	55.7	72.8	88.6	60.3	73.3	57.7	47.5	62.7	50.6	60.4
Sun [43]	52.8	54.8	54.2	54.3	61.8	53.1	53.6	71.7	86.7	61.5	67.2	53.4	47.1	61.6	63.4	59.1
Sun [44]	**47.5**	**47.7**	**49.5**	**50.2**	**51.4**	**43.8**	**46.4**	**58.9**	**65.7**	**49.4**	**55.8**	**47.8**	**38.9**	**49.0**	**43.8**	**49.6**
Ours (PoseNet)	50.5	55.7	50.1	51.7	53.9	46.8	50.0	61.9	68.0	52.5	55.9	49.9	41.8	56.1	46.9	53.3
Without groundtruth information in inference time																
Rogez [40]	76.2	80.2	75.8	83.3	92.2	79.9	71.7	105.9	127.1	88.0	105.7	83.7	64.9	86.6	84.0	87.7
Mehta [29]	58.2	67.3	61.2	65.7	75.8	62.2	64.6	82.0	93.0	68.8	84.5	65.1	57.6	72.0	63.6	69.9
Rogez [41]*	55.9	60.0	64.5	56.3	67.4	71.8	55.1	**55.3**	84.8	90.7	67.9	57.5	47.8	63.3	54.6	63.5
Ours (Full)	**51.5**	**56.8**	**51.2**	**52.2**	**55.2**	**47.7**	**50.9**	63.3	**69.9**	**54.2**	**57.4**	**50.4**	**42.5**	**57.5**	**47.7**	**54.4**

Figure 7. Comparison with state-of-the-art human pose estimators on Human3.6M MPJPE (lower scores is better).

Methods	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	Avg
Accuracy for all groundtruths																					
Rogez [40]	67.7	49.8	53.4	59.1	67.5	22.8	43.7	49.9	31.1	78.1	50.2	51.0	51.6	49.3	56.2	66.5	65.2	62.9	66.1	59.1	53.8
Mehta [29]	81.0	60.9	64.4	63.0	69.1	30.3	65.0	59.6	64.1	83.9	68.0	68.6	62.3	59.2	70.1	80.0	79.6	67.3	66.6	67.2	66.0
Rogez [41]*	87.3	61.9	67.9	74.6	78.8	48.9	58.3	59.7	78.1	89.5	69.2	73.8	66.2	56.0	74.1	82.1	78.1	72.6	73.1	61.0	70.6
Ours	**94.4**	**77.5**	**79.0**	**81.9**	**85.3**	**72.8**	**81.9**	**75.7**	**90.2**	**90.4**	**79.2**	**79.9**	**75.1**	**72.7**	**81.1**	**89.9**	**89.6**	**81.8**	**81.7**	**76.2**	**81.8**
Accuracy only for matched groundtruths																					
Rogez [40]	69.1	67.3	54.6	61.7	74.5	25.2	48.4	63.3	69.0	78.1	53.8	52.2	60.5	60.9	59.1	70.5	76.0	70.0	77.1	81.4	62.4
Mehta [29]	81.0	65.3	64.6	63.9	75.0	30.3	65.1	61.1	64.1	83.9	72.4	69.9	71.0	72.9	71.3	83.6	79.6	73.5	78.9	**90.9**	70.8
Rogez [41]*	88.0	73.3	67.9	74.6	81.8	50.1	60.6	60.8	78.2	89.5	70.8	74.4	72.8	64.5	74.2	84.9	85.2	78.4	75.8	74.4	74.0
Ours	**94.4**	**78.6**	**79.0**	**82.1**	**86.6**	**72.8**	**81.9**	**75.8**	**90.2**	**90.4**	**79.4**	**79.9**	**75.3**	**81.0**	**81.0**	**90.7**	**89.6**	**83.1**	**81.7**	77.3	**82.5**

Figure 8. Comparison with state-of-the-art human pose estimators on MuPoTS using 3DPCK (bigger scores is better).

8 Contribution :

8.1 Proposed Method for Uncertainty Quantification

8.1.1 Heatmaps Variance

Let X, Y , and Z be random variables corresponding to the x, y, and z coordinates of the predicted location for a particular human pose joint in 3D space. Under the soft-argmax prediction strategy for pose estimation [50], the estimated location of the joint, μ, is taken to be the expectation of the random variables. That is, $\mu_x = E[X]$, $\mu_y = E[Y]$, and $\mu_z = E[Z]$. Thus, we can have the variance across our volumetric heatmaps for each axis-coordinates by :

$$var_x = \int_{p \in \Omega} p \times \tilde{H}_k^2(p) \tag{6}$$

where $\tilde{H}_k$ is the normalized heatmaps and Ω is its domain.

Calculating volumetric heatmaps variance for PoseNet model can be be used as a metric to quantify uncertainty for each joint within the predicted pose -Lower values of variance are better-.

8.1.2 Human Pose Clustering : K-Medoids

Previously, investigated methods treat each joint independently and evaluate MPJPE error on all the test dataset. Hence, fail to asses the whole pose as a structure, whether it is physically feasible. To adress this, we propose to create a memory bank based on trained poses used as reference poses. Then, we apply similarity contrastive loss [6] as our pose matching between the predicted human pose and the references.

Memory bank is generated by applying *K-Medoids clustering* -section 12.1- for normalized 3D human poses from the train dataset.

> *K-Medoids clustering* is used to have coherent and physically feasible human pose, unlike *K-means clustering* outputs averaged coordinates for human pose which could be not within the dataset neither physically attainable.

Normalizaton Our normalization procedure is similar to that in [42], in order to have invariant 3D human poses to translation, scale, and rotation as follows :

- **Transform invariant** : in each frame, we transform the origin of the coordinate system from real-world coordinates to hip center of the person. This transformation makes the position of the skeleton joints invariable to the location of the subject. This transformation is demonstrated in below, where i is joint index.

$$(\tilde{x}, \tilde{y}, \tilde{z}) = (x_i - x_{center}, y_i - y_{center}, z_i - z_{center}) \tag{7}$$

- **Scale invariant** : Rescale all the poses with respect to the height of the person.

$$length = \sqrt{(x_{max} - x_{min})^2 + (y_{max} - y_{min})^2 + (z_{max} - z_{min})^2} \tag{8}$$

- **Rotation Invariant** : To make skeleton joints invariant to camera view, a specific rotation is performed with respect to specified view point of the camera. This transformation makes sure that the projection of the vector passing from left hip to right hip on ground plane to stay parallel with x axis in real world coordinates where the rotation angle is computed by :

$$\theta = \arctan \frac{z_{rhip} - z_{lhip}}{x_{rhip} - x_{lhip}} \tag{9}$$

for each skeleton joint in corresponding frame to rotate around the y axis in a counter-clockwise fashion.

$$\begin{bmatrix} \tilde{x}_i \\ \tilde{y}_i \\ \tilde{z}_i \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ z_i \\ 1 \end{bmatrix} \tag{10}$$

By further investigation, we have noticed that scale normalization has an important effect on the clustested centers. Thus, we adapted scale normalization used in [32]. By scaling the pelvis to neck distance to a unit scale, leads to having more difference and sparse poses as centers for clusters.

Afterwards, the pose matching with the reference center from the memory bank is done by calculating the similarity d between the pairwise of predicted pose and the centers according to :

$$score_{i,j} = \frac{\exp(d(\tilde{p}_i, C_j))/\tau}{\sum_{j=1}^{N} \exp(d(\tilde{p}_i, C_j))/\tau} \tag{11}$$

where d, $\tilde{p}_i$, C_j, N and τ are distance, normalized predicted pose, the center pose, number of pose centers and temperature parameter.

The score is calculated across all pairs between normalized predicted pose and the centers of the clustering. To have well calibrated confidence score, various distance metrics should be analyzed :

- **L2/L1 Distance** : is used to calculate the difference between two 3D pose coordinates shaped as 1D vector of length (3 $\times$ Joints).
- **Cosine Similarity** : is a metric used to measure how similar the documents are irrespective of their size. The cosine similarity is advantageous because even if the two similar documents are far apart by the Euclidean distance, chances are they may still be oriented closer together. The smaller the angle, higher the cosine similarity.

$$Cosine(x, y) = \frac{X.Y}{||X|| \times ||Y||} \tag{12}$$

- **Fréchet Distance** : is a measure of similarity between curves that takes into account the location and ordering of the points along the curves. It is named after Maurice Fréchet.
- **Hausdorff Distance** : measures how far two subsets of a metric space are from each other. It turns the set of non-empty compact subsets of a metric space into a metric space in its own right. It is named after Felix Hausdorff.
- **PA-MPJPE Error** : Pair poses are matching if they are visually similar meaning that their MPJPE error is very low after applying Procrustes analysis [14].

9 Experiments And Analysis

Following the most of uncertainty estimation approaches, we perform experiments in two phases : Adapt/Evaluation of the methods to 3D HPE and evaluate the proposed method. We use a *PoseNet* [40] as a reference model. In what follows, we explain in details each method from section 6.4.1 implementation, hyperparameters setting and perform analysis on the obtained results. [6], [7]

9.1 Ablation Study

9.1.1 Deep Ensembles :

3 models of PoseNet are trained with different initializations. We treat the ensemble as a uniformly-weighted mixture model and combine the predictions as

$$p(y|x) = \frac{1}{M} \sum_{m=1}^{M} p_{\theta_m}(y|x, \theta_m) \tag{13}$$

where the different initialization adapted are :

- Model 1 with
  istribution with std :10^{-3})*Default Initialization (Normal D*
  istribution with std :10^{-3}*)*
- *Model 2 with* Kaiming Initialization
- *Model 3 with* Xavier Initialization

Algorithme 1. Pseudocode of the training

- Initialise θ_1, θ_2,..., θ_M randomly.

foreach m = *1 :* M **do**

  1- Train for 24 epochs with *Adam Optimizer*[24], Learning rate $\alpha = 10^{-3}$ with decreasing factor of 10 after epoch 17.

end foreach

Table 5, 6 represent the results from the ensemble models. Which emphasis on the importance of initialitiation where stochasticity of Stochastic Gradient Descent (SGD) has an important effect on the performance of the model.

> **Reproducibility :** We were not able to reproduce the exact values presented in [40]. Even the shared checkpoint at GitHub offical implementation is lower by 0.5 on average for MuPoTS dataset than the results on paper.

9.1.2 MC Dropout :

For MC dropout, we explore the accuracy and uncertainty outcomes for dropouts at different levels of the chosen architecture. Applying dropout during inference time with different p ∈0.1, 0.2, 0.3, 0.4 values. This sampling at test time results again in a set of $\Theta_{m=0}^{M}$ different parameters.

Mi et al [39] show that infer-dropout has same effect as models trained with dropout of different valuesd. Thus, we built a model with $p = 0.2$ and used infer-dropout for remaining p values without training to reduce computational footprint. We perform 10 forward passes and then calculate mean/variance according to equations 2, and 3. Trained models are three types :

6. All of our experiments were performed using **2** ×**NVIDIA RTX 3090**
7. Baseline code used is ***PoseNet* official implementation** : https://bit.ly/37QiZqc

- `Adding dropout to` ***`encoder`*** : Dropout is applied in the encoder component -ResNet18 block- to see how it influences the encoding of the feature maps leading to a variance throughout the whole network. We place the dropout after each convolutional layer of the encoder.
- `Adding dropout to` ***`decoder`*** : We apply dropout only in the decoder of *PoseNet* -deconvolution blocks before arg-max operation- and examine whether this affects the recomposition of the feature maps obtaining epistemic uncertainty for deterministic encoding. We place dropout layer after every convolutional layer in the decoder.
- `Adding dropout to encoder/decoder "`***`dropout -All-`***`"` : Dropout in applied in both encoder and decoder of PoseNet. We place dropout layer with $p = 0.2$ after every convolutional layer. In order to benefit, from both deterministic encoding and robustness to noisy decoding of the features.

Tables 2, and 3 summarizes the results of models with dropout layers on MuPoTS dataset. Table 4 represent the PCK scores for MuPoTS and AUSE and Spearkman's scores for each model of dropout. Figure 9 shows the sparsification curves for each dropout model.

Sequence	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	Avg
Published Model	93.18	75.60	80.31	81.48	84.62	75.35	84.47	69.31	90.13	91.97	80.95	80.96	73.38	73.48	81.79	89.63	88.35	84.28	74.53	70.56	81.2
Encoder p:0.1	91.84	75.09	72.49	76.34	84.09	73.03	80.69	65.81	89.05	92.63	76.04	80.27	71.77	71.24	80.32	90.16	88.86	82.94	76.06	67.96	**79.33**
Encoder p:0.2*	91.42	75.28	72.68	76.27	84.28	73.10	80.39	65.88	88.39	92.15	75.61	80.32	72.25	71.27	80.13	90.32	88.90	83.35	75.99	67.90	79.30
Encoder p:0.3	91.24	75.24	72.85	75.83	84.30	73.17	80.15	65.77	86.96	91.75	75.53	79.90	72.42	71.00	80.19	90.30	89.19	83.33	76.29	67.95	79.17
Encoder p:0.4	90.23	75.27	73.90	75.58	84.10	72.68	79.34	65.54	84.31	90.44	74.88	79.36	72.36	70.16	79.82	90.27	89.13	83.71	76.39	67.76	78.76
Decoder p:0.1	90.01	71.93	74.63	75.14	82.83	70.10	78.82	67.82	87.21	91.18	77.95	83.43	71.86	67.45	77.92	88.73	84.40	82.70	71.89	67.65	78.18
Decoder p:0.2*	90.09	71.63	74.43	74.94	83.02	69.73	78.46	67.52	87.30	91.31	78.08	82.99	71.83	66.89	77.45	88.72	84.26	82.52	72.03	67.62	78.04
Decoder p:0.3	89.71	70.45	74.14	75.01	82.95	69.12	78.49	67.59	87.14	90.71	78.07	82.60	70.99	66.17	77.11	88.74	84.06	82.16	72.23	67.44	77.74
Decoder p:0.4	88.66	70.32	73.82	74.57	82.66	69.08	78.32	67.26	86.80	90.52	78.20	81.34	70.44	65.33	76.77	88.43	83.52	81.67	72.23	67.36	77.36

Table 2. Sequence-wise 3DPCK scores comparison for different models using **all ground-truth** sequences

Sequence	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	Avg
Published Model	93.88	83.04	80.31	81.48	85.44	75.35	84.47	77.15	90.13	91.97	80.95	80.96	74.25	75.95	81.79	89.63	88.35	84.28	75.46	76.25	82.55
Encoder p:0.1	92.53	80.71	72.48	76.34	84.74	73.03	80.68	77.48	89.05	92.63	76.04	80.27	72.84	72.79	80.32	90.16	88.86	82.94	76.83	73.97	**80.73**
Encoder p:0.2*	92.57	80.75	72.68	76.27	84.93	73.10	80.39	77.15	88.39	92.15	75.61	80.32	73.25	73.01	80.13	90.32	88.90	83.35	76.76	74.01	80.70
Encoder p:0.3	92.16	81.40	72.85	75.83	85.12	73.17	80.15	77.27	86.96	91.75	75.53	79.90	73.42	72.63	80.19	90.30	89.19	83.33	77.00	74.11	80.61
Encoder p:0.4	91.13	80.22	73.90	75.58	84.75	72.68	79.34	77.00	84.31	90.44	74.88	79.36	73.06	71.69	79.82	90.28	89.13	83.75	76.98	73.75	80.10
Decoder p:0.1	91.15	77.32	74.63	75.14	84.28	70.10	78.82	73.70	87.21	91.18	77.95	83.43	72.79	71.12	77.92	88.73	84.40	83.13	72.90	74.38	79.51
Decoder p:0.2*	91.00	77.66	74.43	74.94	84.48	69.73	78.46	74.86	87.30	91.31	78.08	82.99	72.90	70.77	77.45	88.72	84.23	82.74	73.23	74.45	79.48
Decoder p:0.3	90.61	78.24	74.14	75.01	84.24	69.12	78.49	75.24	87.15	90.71	78.07	82.60	72.12	70.17	77.11	88.74	84.06	82.38	73.36	74.10	79.28
Decoder p:0.4	90.46	78.27	73.82	74.57	84.28	69.08	78.32	76.18	86.80	90.52	78.20	81.35	71.41	69.70	76.77	88.49	83.53	81.89	73.48	73.74	79.04

Table 3. Sequence-wise 3DPCK scores comparison for different models using **only matched ground-truth** sequences

Table 4. AUSE scores between model and its oracle, Spearman's rank correlation coefficient, on **test dataset MuPoTS**

	with Ground-Truth				**with Predictions**				**AUSE L1**		**AUSE L2**	
Models	X-axis	Y-axis	Z-axis	Average	X-axis	Y-axis	Z-axis	Average	Sum Area	Mean Area	Sum Area	Mean Area
Encoder p:0.1	-0.00345	0.00623	-0.00311	-0.00053	0.75391	-0.51479	0.99970	0.99973	**0.060799**	**0.156114**	**0.080499**	**0.206118**
Encoder p:0.2	-0.00345	0.00623	-0.00311	-0.00053	0.75391	-0.51479	0.99970	0.99973	0.060799	0.156115	0.080500	0.206122
Encoder p:0.3	-0.00345	0.00623	-0.00311	-0.00053	0.75391	-0.51479	0.99970	0.99973	0.060799	0.156115	0.080502	0.206125
Encoder p:0.4	-0.00345	0.00623	-0.00311	-0.00053	0.75391	-0.51479	0.99970	0.99973	0.060802	0.156113	0.080501	0.206122
Decoder p:0.1	-0.00344	0.00623	-0.00304	-0.00053	0.75012	-0.51340	0.99970	0.99972	0.060799	0.156114	0.080823	0.206127
Decoder p:0.2	-0.00345	0.00623	-0.00311	-0.00053	0.75391	-0.51479	0.99970	0.99973	0.060796	0.156209	0.080486	0206185
Decoder p:0.3	-0.00344	0.00623	-0.00306	-0.00050	0.74957	-0.51267	0.99970	0.99972	0.050799	0.156213	0.080502	0.206226
Decoder p:0.4	-0.00345	0.00620	-0.00306	-0.00051	0.74927	-0.51294	0.99970	0.99973	0.060799	0.156213	0.080501	0.206224

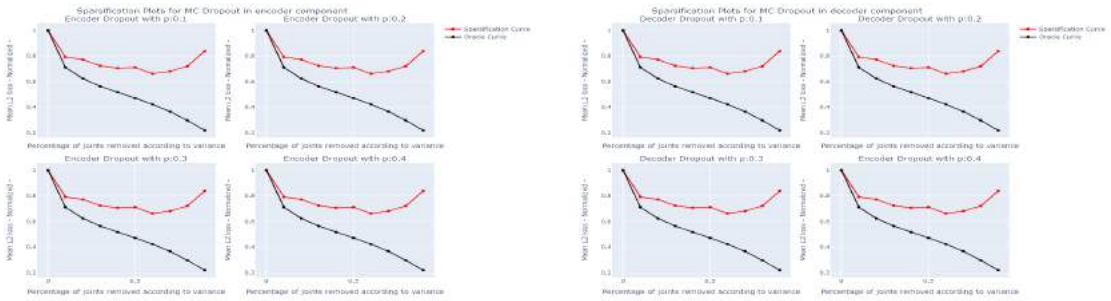


Figure 9. Sparsification graphs for MuPoTS test dataset. Left and right figures represent the sparsification plots for using dropout in encoder and decoder components, respectively.

9.1.3 Log-likelihood maximization (Log) :

Training *PoseNet* with log-likelihood maximization makes the network able to encode mean and variance of the distribution. The additional logarithmic term discourages infinite predictions for any joint coordinate. Regarding numerical stability, the network is trained to estimate the log-variance in order to avoid zero values of the variance.

Incorporating directly, the loss into the network lead to exploding gradient issues, thus a change of the architecture is needed. After trials, we found the illustrated architecture in figure 10 provides stable training. By training two parallel networks in the decoder component one for predicting means (Joints coordinates for each axis) and 2nd network for estimating variances. This latter, is initialized by *Xavier initiliazation* followed by *SoftPlus* to make variance's values positive.

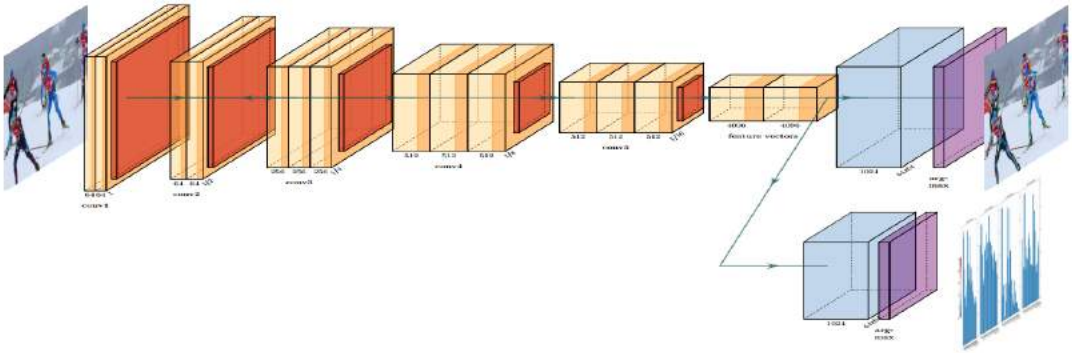


Figure 10. Proposed PoseNet architecture to estimate joint's coordinates and variances. Top network -*PoseNet-Means*- performs deconvolution after ResNet18 extracted features followed by Arg-max to get means (3D joints coordinates). Bottom network has same deconvolution layers as *PoseNet-Means* to provide variances values for each joint.

Note :
- Training with log-likelihood maximization increased the training time by 40% from regular training where 1.68 hour per epoch.
- Both types of log-likelihood Gaussian/Laplacian were adapted as a loss for training *PoseNet*.

We also applied Deep Ensembles technique for 3 trained models with log-likelihood maximization for Gaussian distribution - named KL Loss - in order to benefit from better

variance estimation and good accuracy as indicated in tables 9, and 10.

9.1.4 Regularized heatmaps :

By analysing the regular trained *PoseNet* volumetric heatmaps generated for each pose before *arg-max* operation, we noticed that are noisy and do not focus on the keypoint corresponding to the joint as illustrated in Figure 24.

Thus, we would like the volumetric heatmaps value's to mimic a specific probability distribution -Spherical Gaussian distribution-. By making sure the the mean value of the distribution matches the ground-truth joint coordinates. Nibal et al [44] propose to add a regularization term to the $L2$ loss, know as *Jensen-Shannon divergence (JSD)* [29] making the training loss as follows :

$$l_{train} = ||\mu - \tilde{\mu}|| + \lambda(JSD(H_{xy} - \mathcal{N}(\tilde{\mu}_{xy}, \sigma^2)) + JSD(H_{zy} - \mathcal{N}(\tilde{\mu}_{zy}, \sigma^2)) + JSD(H_{xz} - \mathcal{N}(\tilde{\mu}_{xz}, \sigma^2))) \quad (14)$$

where μ, and $\tilde{\mu}$ are predicted joints coordinates, and GT coordinates respectively. $\mathcal{N}$ represents a spherical Gaussian distribution with mean μ and $\sigma = 1$ pixel for 64×64 heatmaps. H_{ij} is a marginal heatmap by converting volumetric heatmaps -3D- into 2D marginal heatmaps -more details in Appendix 12.2-.

Table 5. Sequence-wise 3DPCK scores comparison for different models using **all ground-truths**

Seq	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	Avg
Published Model	93.18	75.60	80.31	81.48	84.62	75.35	84.47	69.31	90.13	91.97	80.95	80.96	73.38	73.48	81.79	89.63	88.35	84.28	74.53	70.56	81.2
Model 1	93.00	76.11	77.94	79.28	84.73	76.80	82.40	68.05	87.98	91.86	78.08	79.73	72.52	73.13	81.09	89.79	90.60	83.37	75.83	70.97	80.6
Model 2	93.2	77.2	76.6	76.3	85.9	72.6	85.4	71.7	87.0	90.7	80.9	84.2	72.5	68.8	81.5	90.0	87.4	82.6	72.6	68.7	80.07
Model 3	93.1	76.3	76.2	78.3	85.0	73.7	83.4	72.2	88.8	92.4	82.2	85.1	71.2	64.2	79.9	89.4	85.2	83.3	71.3	69.0	80.12
Regularized Model II	92.61	76.72	81.46	78.70	86.95	68.75	85.47	70.72	90.03	92.13	80.92	81.72	71.34	67.61	81.40	88.58	89.01	82.58	74.56	69.10	80.55
Deep Ensemble	93.1	76.3	76.4	77.96	77.96	73.7	83.4	72.2	88.8	92.4	82.2	84.65	72.07	68.71	81.5	89.7	86.3	84.95	71.95	68.85	**80.81**
MC Dropout -All-	92.24	73.80	75.00	75.11	85.67	69.07	83.27	68.72	88.26	91.66	82.01	80.14	71.46	63.86	76.54	88.81	86.65	81.71	71.43	68.34	78.72
Kendel Loss (Gaussian)	92.04	74.94	74.61	74.50	83.05	71.77	85.72	67.68	88.32	90.08	79.79	84.09	73.34	70.13	77.10	90.60	88.44	82.90	72.18	68.57	79.49
Kendel Loss (Laplacian)	91.81	74.49	69.04	73.30	81.83	73.47	84.89	68.34	85.41	88.79	79.77	80.00	70.90	70.83	75.70	90.38	88.84	84.30	74.68	71.56	78.9
Emprical Ensemble KL Loss	92.8	75.84	77.12	75.68	81.02	71.09	84.65	72.88	88.77	92.94	80.14	83.19	73.53	73.76	82.36	89.12	85.44	82.41	72.07	71.32	80.37

Table 6. Sequence-wise 3DPCK scores comparison for different models using **only matched ground-truth** sequences

Seq	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	Avg
Published Model	93.88	83.04	80.31	81.48	85.44	75.35	84.47	77.15	90.13	91.97	80.95	80.96	74.25	75.95	81.79	89.63	88.35	84.28	75.46	76.25	82.55
Model 1	93.00	83.24	77.94	79.28	85.71	76.80	82.40	77.87	87.98	91.86	78.08	79.73	73.46	74.64	81.10	89.79	90.60	83.37	77.08	77.41	82.07
Model 2	94.1	83.0	76.6	76.3	86.7	72.6	85.4	77.6	87.0	90.7	80.9	84.2	73.7	70.9	81.5	90.0	87.4	82.6	74.0	74.2	81.5
Model 3	94.0	81.0	76.2	78.3	85.8	73.7	83.4	77.9	88.8	92.4	82.2	85.1	72.3	68.6	79.9	89.4	85.2	83.6	72.7	74.7	81.3
Regularized Model II	94.25	83.36	81.46	78.70	87.63	68.76	85.47	77.70	90.03	92.16	80.92	81.72	72.48	72.60	81.40	88.64	89.01	82.58	76.21	74.77	81.99
Deep Ensemble	94.7	82.41	76.91	77.96	86.07	74.37	83.73	77.79	87.93	91.65	80.39	83.01	73.15	74.38	80.83	89.73	87.73	83.19	74.59	75.44	**83.91**
MC Dropout -All-	93.63	81.61	75.00	75.11	86.50	69.07	83.28	76.11	88.24	91.66	82.01	80.14	72.59	68.93	76.54	88.81	86.65	81.71	72.61	74.22	80.22
Kendel Loss (Gaussian)	92.96	85.31	74.61	74.50	83.85	71.77	85.72	77.37	88.32	90.08	79.79	84.09	74.58	71.85	77.10	90.66	88.44	82.90	73.66	74.75	81.12
Kendel Loss (Laplacian)	92.04	80.41	69.04	73.30	82.78	73.47	84.89	79.70	85.41	88.79	79.77	80.00	71.96	72.69	75.70	90.38	88.84	84.30	75.38	77.71	80.32
Empirical Ensemble KL Loss	92.9	83.05	78.65	77.98	86.11	74.24	84.65	75.92	86.12	91.24	77.59	84.64	71.15	76.88	83.11	89.65	88.64	83.44	75.77	78.89	82.03

Accuracy : Tables 5, and 6 report the 3DPCK scores of different uncertainty estimation models. We can notice how, in general, predictive models fail at improving 3D HPE. On the other hand, deep ensemble outperforms all the other models benefiting from aggregating 3 variant PoseNet models performance, while dropout has the lowest 3DPCK scores on both evaluation protocols.

Additionally, we notice that the added term of regularization -*Regularized Model II*- has improved the 3DPCK score on MuPoTS dataset, since it tries to minimize the variance across marginal heatmaps. Therefore, it leads to small expectation values when we apply *arg-max operation* [50] to have the 3D joint's coordinates.

Table 7. AUSE scores between model and its oracle, Spearman's rank correlation coefficient, on **test dataset MuPoTS**

	with Ground-Truth				with Predictions				AUSE L1		AUSE L2	
Models	X-axis	Y-axis	Z-axis	Average	X-axis	Y-axis	Z-axis	Average	Sum Area	Mean Area	Sum Area	Mean Area
Model I	0.19579	0.61217	0.82784	0.45240	0.06773	0.11258	0.00277	0.00176	0.06911	0.16802	0.16694	0.37594
Empirical Deep Ensemble	-0.00343	0.00573	-0.00327	-0.00084	0.73724	-0.51567	0.99967	0.99967	0.05080	0.13610	0.08053	0.20620
Heatmaps Deep Ensemble	0.19315	0.61328	0.83498	0.45234	0.06718	0.11260	0.00243	0.00154	0.06934	0.16891	0.16736	0.37712
Regularized Model II	0.19060	0.61225	0.81711	0.45071	0.06807	0.11276	0.00223	0.00150	0.0593187	0.14914	0.147185	0.33779
MC Dropout -All-	-0.00342	0.00610	-0.00298	-0.00052	0.73904	-0.51353	0.99970	0.99973	0.05080	0.13611	0.08054	0.20622
Kendel Loss -Gaussian-	0.02238	0.18449	0.06603	0.16435	0.00484	0.02086	0.00055	0.003769	0.07274	0.17584	0.14067	0.22781
Kendel Loss -Laplacian-	0.05798	0.59613	0.01037	0.04829	0.78561	-0.53142	-9.77749	0.00012	0.07914	0.18720	0.18195	0.40776
Empirical Ensemble KL Loss	-0.00346	0.00748	-0.00351	0.21661	0.87868	-0.97879	0.99973	0.99996	**0.05045**	**0.13519**	**0.07937**	**0.20323**

Note : Sparsifications graphs are generated using `L2 mean error` due to :
- **Mean** : in order to validate whether the models are well calibrated at estimating uncertainty, by using sum of losses of the removed joints, sparsification plots are always linear because the L2 error decreases as we increase the number of removed joints.
- **L2** : in order to magnify the loss error between the predicted coordinates and the ground-truth such that we avoid having superimposed plots of models.

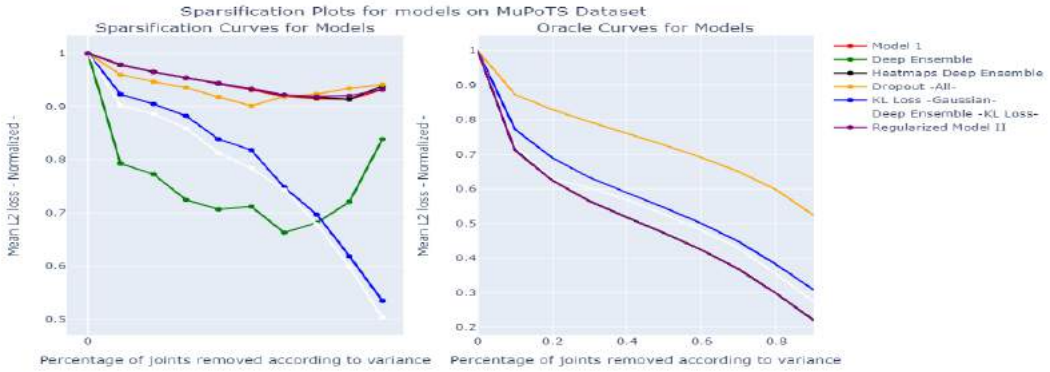


Figure 11. Sparsifcation graphs for MuPoTS test dataset. **Left figure** represents the sparsification plots for models -Mean L2 loss- for joints removed according to variance. **Right figure** represents Oracle plots -Mean L2 Loss- for joints removed according to L2 loss with respect to ground-truth coordinates.

Uncertainty : Table 7 summarises performance of modelled uncertainties at reducing errors on the estimated 3D human pose coordinates. Predictive methods perform better. Among them, empirical ensemble of 3 models using KL Loss -Gaussian- outperforms at estimating uncertainty other models, often aggregating different models outputs achieves better accuracy, experiment proved that it is also the case for uncertainty estimation.

Surprisingly, *MC Dropout -All-* model performs good given AUSE score, but by analysing the plots in figure 11, we notice that *drop-out*, *model1*, and *Deep Ensembles* under-estimate the last 25 quantile of the removed joints which resulted from having over-confident estimates although the L2 error of the predicted coordinates with respect to ground-truth coordinates is very high. *Heatmaps deep ensemble* performs poorly due to high variance in the volumetric heatmaps produced by PoseNet for each axis coordinate as explained in Appendix 12.2. For *regularized heatmaps* models, we notice it didn't improve AUSE score, we believe this is due reconstruction of marginal heatmaps -Z-axis coordinates by equation 19 which is not not accurate- and due to heads of deconvolution with BatchNorm in *PoseNet* which have same effect of the regularization loss [54], [9].

Regarding different MC Dropout models, tables 2, 3 and 4 show a significant variation in their performance in terms of uncertainty and accuracy of the coordinate's predictions. We found that placing dropout in the encoder - MC Dropout Encoder- leads to lower errors than introducing it in the decoder -MC Dropout Decoder-. We believe that applying MC dropout in the decoder produces deterministic image representations because the encoder is deterministic making the reconstruction hard. Nevertheless, MC dropout in the encoder allows us to have probabilistic representations, which seems a more appropriate model to tolerate noise and changes in the input. Looking at the different dropout parameters, the accuracy improves if we lower the probability of dropout from 0,4 to 0,1.

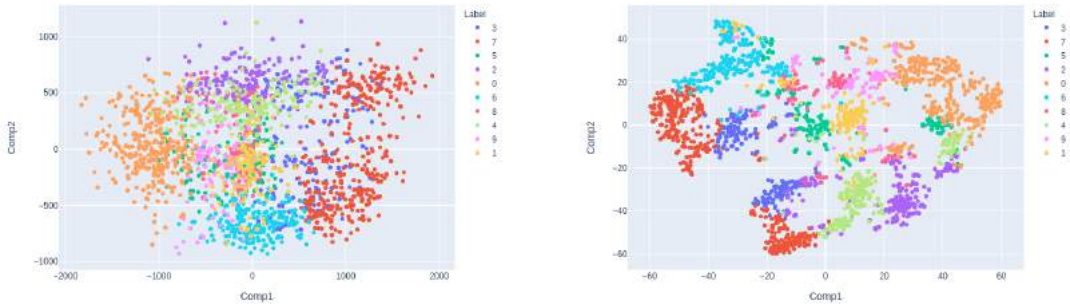
On the other hand, predictive models *KL Loss -Gaussian-* and *Deep Ensemble KL Loss* have quasi-linear sparsification plots proving that their variance estimation is very reliable and highly correlated with the error with respect to the ground-truth coordinates.

Furthermore, we regard Spearman's rank correlation coefficient as not an appropriate uncertainty quantification metric for 3D HPE, due to the type of the problem we are studying. This coefficient in our case study is calculated either by correlation between each coordinate axis and its variances for both ground-truth coordinates and model predictions or by averaging the coordinates of all 3 axes, leads to having non-interpretable values in table 7. Resulting from the fact that an error between the model prediction and GT coordinates in one axis makes a change in the average value, which drives the signal to be crooked unsimilar to variance signal implies having a poor Spearman's coefficient.

Summary : In general, predictive methods performs better for uncertainty modelling but their performance on 3D HPE accuracy is less than empirical models *-Deep Ensemble-*. But, we firmly believe that they are more reliable (Predictions are well calibrated - Sparsification plot is quasi-linear-) along with the difference in performance in terms of accuracy is small, which could be further improved by changing the proposed architecture -Figure 10- and/or the initialization of the network parameters.

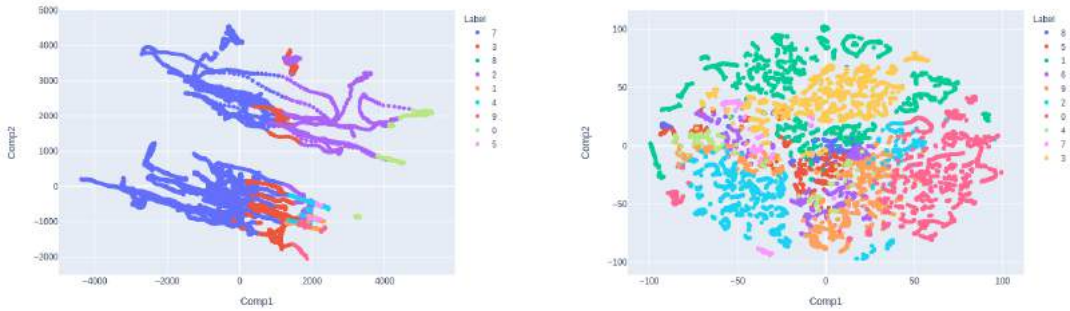
9.2 3D HPE with K-Medoids

MuCo dataset is quite large dataset to be clustered in one-time run. Thus, we thought about creating 10 centers for each sequence within 200 sequences in the dataset -Each sequence has 500 frames-, after normalizing the human poses for each frame. Furthermore, we apply K-medoids clustering for the resulting 10 centers for each video sequence (10 $\times$200) to get 100 centric poses for MuCo dataset.



(a) PCA **(b)** tSNE

Figure 12. Visualisation of K-medoids clustering for 200 human poses sampled from MuCo dataset into 10 clusters, where the center for each cluster represent our centric human pose.



(a) PCA **(b)** tSNE

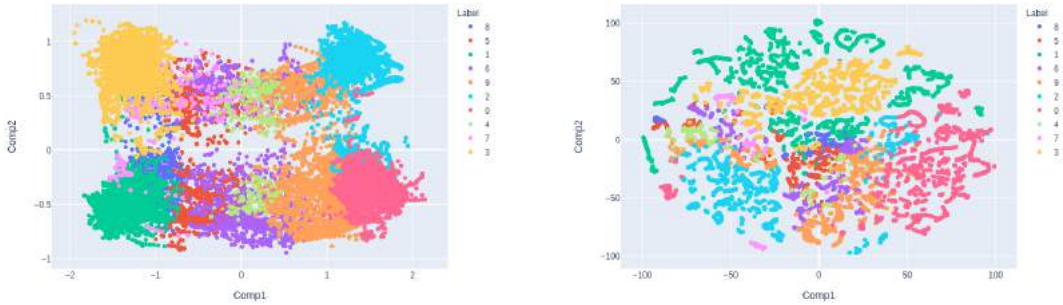
Figure 13. Visualisation of K-medoids clustering for *MuPoTS dataset* by using the 10 center human poses from MuCo dataset.

Dimentionality reduction techniques such as *PCA*[10], *tSNE*[35] are used for visualisation purposes only to check if centric poses of K-medoids are able to generate good clusters of 3D poses. Dimentionality reduction is applied after clustering by projecting our 3D coordinates for every joint in the human pose into 2 components, in order to have 2D visualisation.

Figures 12, 13 illustrates a visualisation of the centric poses which will be used as reference poses to calculate similarity with the predicted 3D human pose from *PoseNet*. We notice that using K-medoids algorithm, we can get good clusters of 3D poses. Although, clustering test dataset MuPoTS using the extracted centers is not good enough to create well separated clusters since no normalization was applied.

Figures 15, 14 illustrates how pose coordinates are distributed by using both *t-SNE* [35]

and *PCA*[10] methods after applying normalization procedure -section 8.1.2- Normalization has an impact on the clustering behavior of K-medoids by creating a well separated clusters in MuPoTS dataset.



(a) PCA **(b)** tSNE

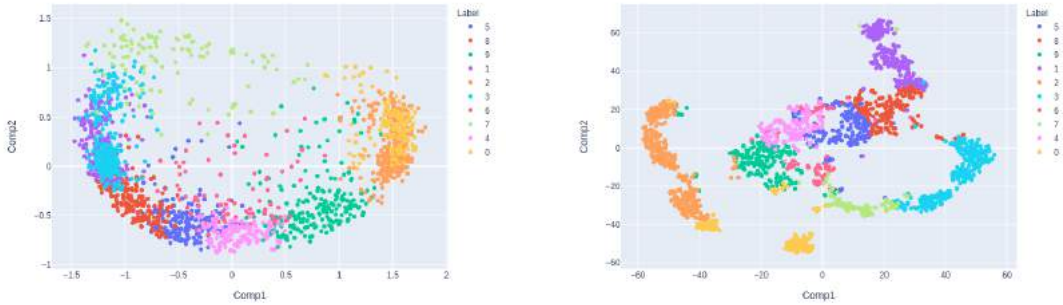
Figure 14. Visualization of MuCo's centers used for MuPoTS clustering *with normalization*

Note : At this stage, we had to try another normalization technique regarding -scale invariance- by qualitatively analysing the centric poses after clustering. We found that using normalization as suggested in Liu et all paper [32] created more diverse set of human poses.

Scale invariance : we normalize all the lengths between joints with respect to "pelvis to neck" length as unit scale.

$$\tilde{P} = \frac{P}{S} \tag{15}$$

where $\tilde{P}$, P and S are the normalized pose coordinates, original pose coordinates and the length of "pelvis to neck", respectively.

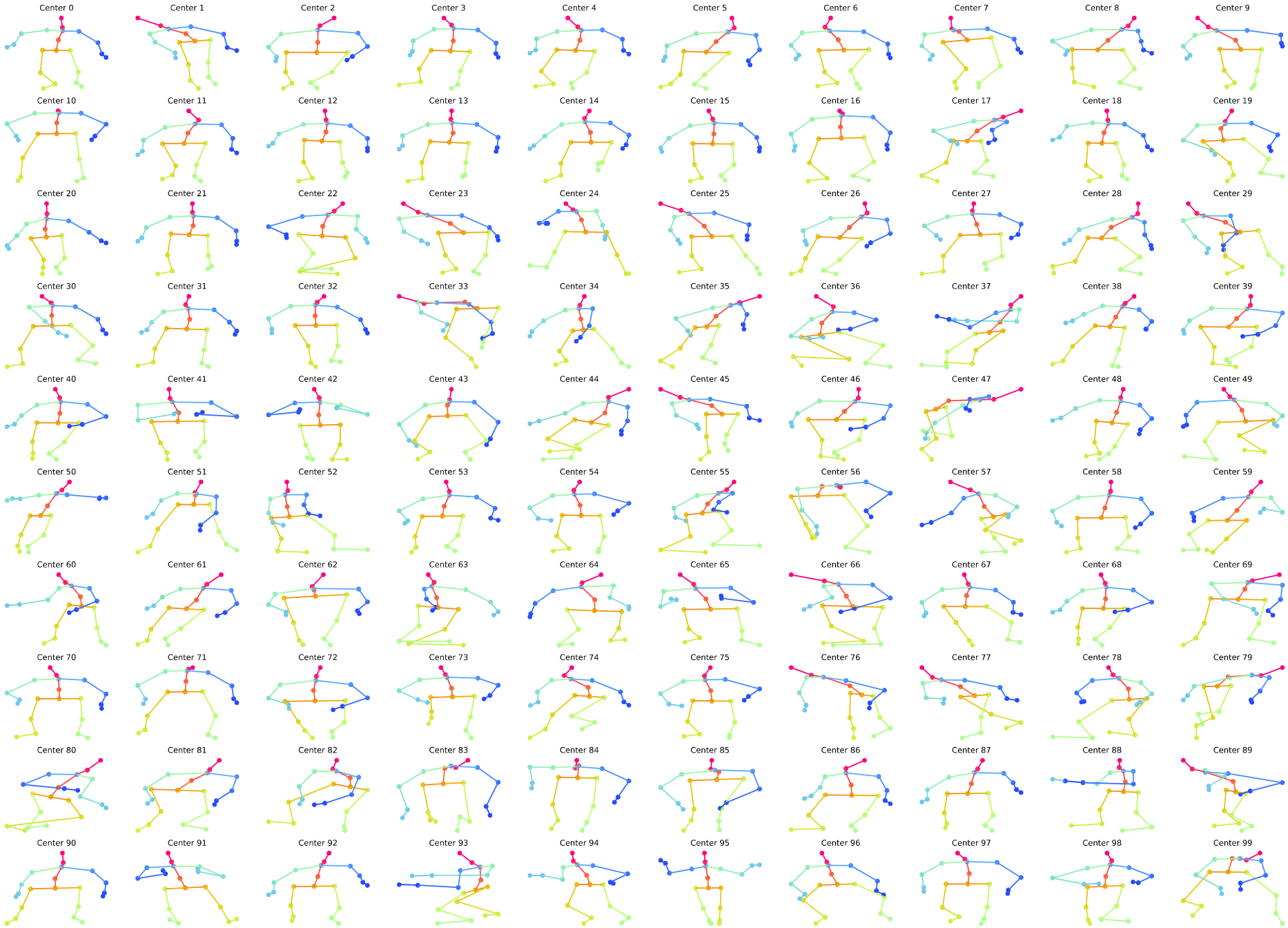


(a) PCA **(b)** tSNE

Figure 15. Visualization of K-medoids clustering for MuCo's 200 sequences centers *with normalization*

In the following two pages, we illustrate the 100 centric poses used as memory bank for our framework.

Center 0
Center 1
Center 2
Center 3
Center 4
Center 5
Center 6
Center 7
Center 8
Center 9
Center 10
Center 11
Center 12
Center 13
Center 14
Center 15
Center 16
Center 17
Center 18
Center 19
Center 20
Center 21
Center 22
Center 23
Center 24
Center 25
Center 26
Center 27
Center 28
Center 29
Center 30
Center 31
Center 32
Center 33
Center 34
Center 35
Center 36
Center 37
Center 38
Center 39
Center 40
Center 41
Center 42
Center 43
Center 44
Center 45
Center 46
Center 47
Center 48
Center 49
Center 50
Center 51
Center 52
Center 53
Center 54
Center 55
Center 56
Center 57
Center 58
Center 59
Center 60
Center 61
Center 62
Center 63
Center 64
Center 65
Center 66
Center 67
Center 68
Center 69
Center 70
Center 71
Center 72
Center 73
Center 74
Center 75
Center 76
Center 77
Center 78
Center 79
Center 80
Center 81
Center 82
Center 83
Center 84
Center 85
Center 86
Center 87
Center 88
Center 89
Center 90
Center 91
Center 92
Center 93
Center 94
Center 95
Center 96
Center 97
Center 98
Center 99


Center 0
Center 1
Center 2
Center 3
Center 4
Center 5
Center 6
Center 7
Center 8
Center 9
Center 10
Center 11
Center 12
Center 13
Center 14
Center 15
Center 16
Center 17
Center 18
Center 19
Center 20
Center 21
Center 22
Center 23
Center 24
Center 25
Center 26
Center 27
Center 28
Center 29
Center 30
Center 31
Center 32
Center 33
Center 34
Center 35
Center 36
Center 37
Center 38
Center 39
Center 40
Center 41
Center 42
Center 43
Center 44
Center 45
Center 46
Center 47
Center 48
Center 49
Center 50
Center 51
Center 52
Center 53
Center 54
Center 55
Center 56
Center 57
Center 58
Center 59
Center 60
Center 61
Center 62
Center 63
Center 64
Center 65
Center 66
Center 67
Center 68
Center 69
Center 70
Center 71
Center 72
Center 73
Center 74
Center 75
Center 76
Center 77
Center 78
Center 79
Center 80
Center 81
Center 82
Center 83
Center 84
Center 85
Center 86
Center 87
Center 88
Center 89
Center 90
Center 91
Center 92
Center 93
Center 94
Center 95
Center 96
Center 97
Center 98
Center 99

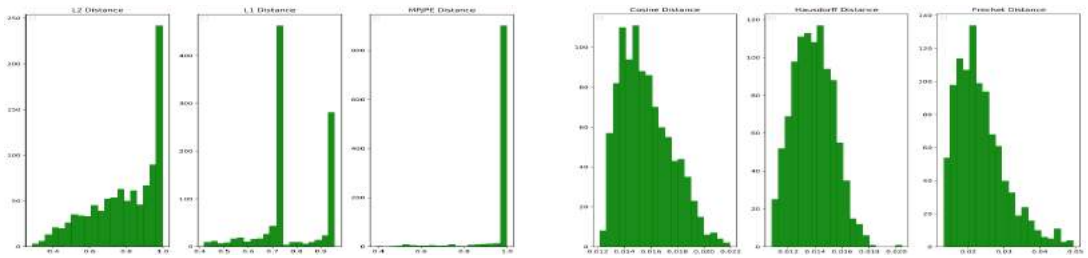
9.3 Uncertainty with K-Medoids

We propose a method to provide a confidence score whether the predicted pose by *PoseNet* is plausible with respect to 3D human pose centers extracted from the training dataset via K-Medoids. Firstly, by calculating a similarity between predicted pose and centric poses using *Contrastive Loss* equation [11] with $\tau = 0.5$.

Figure 16. Confidence score estimation for samples from MuPoTS dataset using *cosine similarity*. Columns 2nd, 3rd and 4th correspond to the 3 first similar centric poses from memory bank to the predicted pose in 1st column

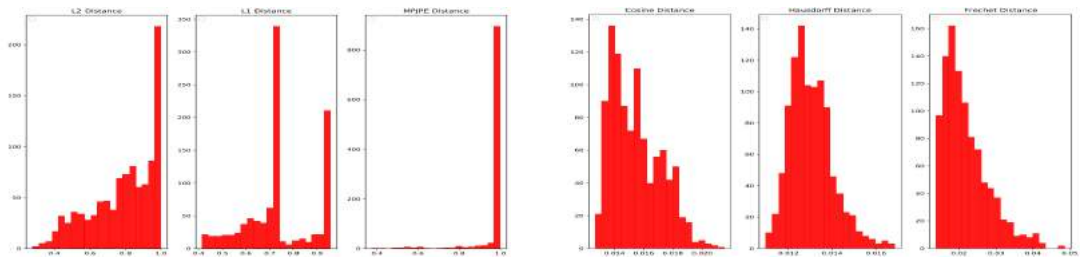
By qualitatively analysing some samples, we found that *Cosine* and *PA-MPJPE* used as similarity metrics give very similar poses from the memory bank to the predicted pose but the probability scores assigned to it are not very coherent, both metrics suffer from underestimating such that they give very low probability scores. On the other hand, metrics -L1/L2/Fréchet/Hausdorff- do overestimate the confidence scores.

Figures 17, and 18 illustrates the distribution of confidence scores of 1000 samples taken from *MuCo-3DHP* and *MuPoTS*. Histograms for both datasets are having same distribution



(a) L1/L2/MPJPE as similarity metric **(b)** Cosine/Hausdorff/Frechet as similarity metric

Figure 17. Distribution of confidence scores for each similarity metric used as distance between centric pose and 1000 predicted pose from *MuCo-3DHP*. **Left** and **Right** figures illustrate metrics that do overestimate, and underestimate, respectively



(a) L1/L2/MPJPE as similarity metric **(b)** Cosine/Hausdorff/Frechet as similarity metric

Figure 18. Distribution of confidence scores for each similarity metric used as distance between centric pose and 1000 predicted pose from *MuPoTS*. Same confidence score's distribution as train dataset *MuCo* although many sample poses from *MuPoTS* are not within the memory bank.

approximately, L1/L2/MPJPE do overestimate as illustrated in figure 18 although same predicted poses are not wihtin the memory bank like (sitting as illustrated in figure 19). On the hand, Cosine/Hausdorff/Frechet produce very low confidence scores proving that the predicted poses although they are not within the memory bank but they are not far in terms of distance with the centric poses.

Furthermore, to evaluate the reliability of the confidence scores, we have picked manually centric poses with following indices [19, 27, 33, 43, 96] as a memory bank in order to calculate confidence scores for similar and non-similar samples from *MuPoTS* dataset.

Figure 20 shows that that metrics - Cosine/Hausdorff/Frechet- provide approximately similar confidence scores for different centric poses. For cosine similarity (1st row), we expect the last 3 pose centers to have better confidence score than center 2 since poses reflect sitting position as in the predicted pose.

Summary : As conclusion, we resume that using Contrastive Loss equation [11] does not provide a good basis to estimate confidence scores for 3D human poses since distance between 3D coordinates does not reflect whether it physically feasible nor similar to poses within the memory bank.

Figure 19. Visualization of confidence score for a sample -column 1st- from *MuPoTS* not in memory bank. Rows correspond to a similarity metric, and columns to similar centric poses from memory bank *100 poses* with their respective confidence scores.

Figure 20. Visualization of confidence score for a sample -1st column- from *MuPoTS* not in memory bank. Rows correspond to a similarity metric, and columns to similar centric poses from memory bank of *5 poses* with their respective confidence scores.

9.4 Out-of-Distribution Detection

Therefore, we changed calculating the confidence score procedure to be a distance between the predicted pose and the center poses as the following :

$$score_{i,j} = \frac{d(\tilde{p}_i, C_j)}{\max_d} \tag{16}$$

where d, $\tilde{p}_i$, C_j are distance, pose coordinates and center pose coordinates, respectively. $\max_d$ correspond to the maximum distance between sampled poses and centric poses.

Intuitively, by detecting Out-Of-Distribution (OOD) poses with respect to our memory bank, we expect our framework estimates OOD with high confidence score, and the scores of OODs are globally positioned too far from IN-Distribution (IND) poses. Therefore, we sampled 100 from both training and test dataset labelled as class 1 and class 0, respectively. Then, we plot histograms for confidence score for each similarity metric d using equation [11].

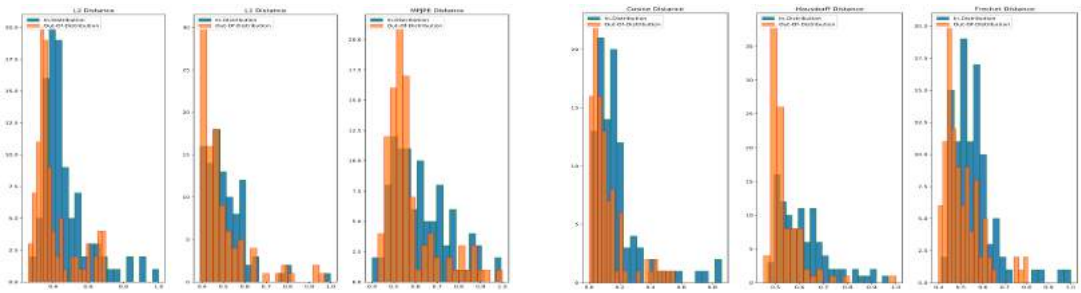


Figure 21. Distribution of confidence scores for each similarity metric used as distance between centric pose and predicted pose.

Figures 21 summarises distribution of confidence score for each metric used as a distance between the centric pose and predicted pose. The expected behavior should be well separated bins of OOD and IND, because the distance between the predicted pose and centric one from memory bank should be small. Thus, most of IND poses should be located on the left corner of the histgrams.

We conclude that similarity distance relying 3D coordinates is sufficient to detect OOD poses, this conclusion is proved by the AUC scores for ROC curves in table 8, where most metrics are much better than random classifier.

Table 8. Area Under Curve ROC score for each distance metric used as similarity between center and predicted pose pairs.

Similarity Metric used	L2	L1	MPJPE	Cosine	Hausdorff	Fréchet
Distance metric [16]	0.717	0. 5996	0.6311	0.6788	**0.7703**	0.6753

A ROC curve called as Receiver Operating Characteristic Curve is constructed by plotting the true positive rate (TPR) against the false positive rate (FPR).

Classifiers that give curves closer to the top-left corner indicate a better performance. Accuracy is measured by the area under the ROC curve. An area of 1 represents a perfect binary classifier. As a baseline, a random classifier is expected to give points lying along the diagonal (FPR = TPR).

A drawback of using equation [16] rather equation [11] is that it provides only whether the predicted pose is either OOD or IND, not the likelihood of how similar the predicted pose to the centric poses in memory bank.

10 Limitations And Challenges

During this internship, we encountered many challenges and obstacles while conducting experiments , analysing results and we come up with a list of most important points which next work should focus on :

- Log-likelihood maximization loss is hard to be optimized, it requires tweaking the network or variable change trick to be incorporated within the network.

- Distance used as a similarity metric for pose matching is crucial part of the prediction to have coherent confidence scores, otherwise leading to under/over estimated confidence scores.
- Similarity metrics could be applied on other features such "Angles between joints" rather than 3D human pose coordinates.
- Quantify uncertainty in models with other metrics "Calibration for Regression", since differences in AUSE scores for our models is very small making it harder to be analysed.
- The benchmarked datasets such as MuPoTS and MuCo contain truncated images.
- A proposed solution was to feed the features vectors of *PoseNet* for input sample to an *autoencoder* then we apply similarity metrics on the latent features provided by autoencoder, but the dimentionality of PoseNet features -volumetric heatmaps- is big to be used for training an autoencoder.

11 Conclusion

To conclude, results obtained within this internship are both satisfying and promising since work done on uncertainty in 3D HPE is limited. We evaluated different methods of uncertainty quantification for HPE, heatmaps variance proposed method and surprisingly the good results of MC Dropout for uncertainty estimation contrary to what's known in literature. Finally, a baseline framework to give confidence scores for the predicted human pose.

As future work, we would like to investigate other uncertainty quantification metrics since differences in AUSE scores for our models is very small. Fine-tune the predicted confidence scores by using calculating similarity on other features such as angle between joints could be to used instead of 3D coordinates or design new similarity metric adaptive to HPE.

Thanks to this project, I could put the knowledge I accumulated and the skills I have developed in computer vision into practice in a real case. Moreover, I discovered a promising and crucial sub-field of Machine learning "Uncertainty in Deep Learning", this later is very important to develop robust and reliable predictive systems for high-stack applications. Finally, I look forward to continue working and delving more into this discipline and making usage of the insights and tips learnt from my internship tutors for future career.

12 Appendices

12.1 Appendix A : K-Medoids Clustering

`K-Medoids` (also called as Partitioning Around Medoid) algorithm was proposed in 1987 by Kaufman and Rousseeuw. A medoid can be defined as the point in the cluster, whose dissimilarities with all the other points in the cluster is minimum.

The dissimilarity of the medoid C_i and object P_i is calculated by using : $E = ||C_i - P_i||$

Algorithme 2. Partitioning Around Medoid Algorithm

```
Initialize : select k random points out of the n data points as the medoids. ;
Associate each data point to the closest medoid by using any common distance
  metric methods. ;
while the cost decreases do
    foreach medoid m do
        foreach data o point which is not a medoid do
            Swap m and o, associate each data point to the closest medoid,
              recompute the cost. ;
            If the total cost is more than that in the previous step, undo the swap. ;
        end foreach
    end foreach
end while
```

Note : K-Medoids may obtain generate different results for different runs on the same dataset because the first k medoids are chosen randomly. Thus, we save the clustering centers from the first run to further analysis.

12.2 Appendix B : Regularised Heatmaps

One way of extending the heatmaps to produce 3D HPE is by designing a model which produces volumetric heatmaps as used in *PoseNet*[40]. Volumetric heatmaps increase the memory requirements of the system considerably. Thus, Nibali et al [44] propose what's called *Marginal heatmaps* as an alternative which are 2D heatmaps to lower the computational footprint. Therefore, in our regularization technique for volumetric heatmaps, we will convert them into marginal heatmaps as follows [8] :

Let X, Y, and Z be random variables corresponding to the x-, y-, and z-coordinates of the predicted location for a particular human pose joint in 3D space. Under the argmax prediction strategy for pose estimation [50], the estimated location of the joint μ is taken to be the expectation of the random variables. That is, $\mu_x = E[X]$, $\mu_y = E[Y]$, and $\mu_z = E[Z]$. We can represent the trivariate probability mass function $P(X = x, Y = y, Z = z)$ with a volumetric heatmap. The volumetric heatmap $\hat{H}$, is a depth $\times$height $\times$width tensor where each element represents the probability that the joint is located at the corresponding spatial location. All elements of the heatmap must be non-negative and sum to one in order to define a valid probability mass function.

Equation 17 describes how the estimated location may be computed from $\hat{H}$ under the argmax scheme. The coordinate indicator tensors X, Y, and Z consist of elements referencing their own x, y, and z coordinates respectively (that is, $X_{i,j,k} = k, Y_{i,j,k} = j$, and $Z_{i,j,k} = i$). We denote the scalar product of vectorised tensors by $< ., . >$.

$$\mu_x =< \hat{H}, X > \ \mid \ \mu_y =< \hat{H}, Y > \ \mid \ \mu_z =< \hat{H}, Z > \tag{17}$$

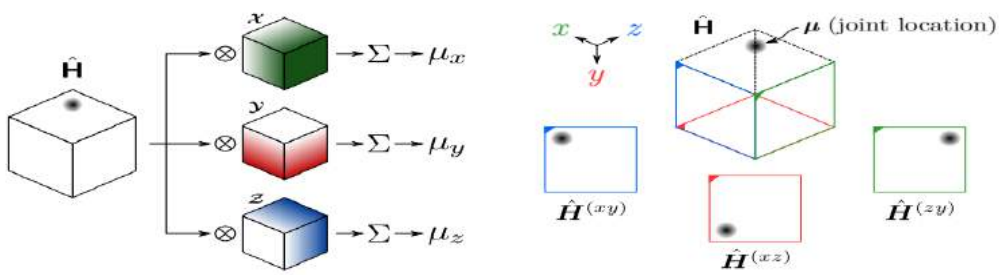


Figure 22. **Left figure** represent graphical description of the argmax operation for a volumetric heatmap. Darker colours indicate higher tensor element values. **Right figure** illustrates multiple two-dimensional "views" of a spherical Gaussian from outside the volumetric heatmap can fully describe the location of a joint.

Consider the bivariate marginal probability mass functions P(X = x, Y = y), P(Y = y, Z = z), and P(X = x, Z = z), and their corresponding two-dimensional heatmap representations $\hat{H}^{xy}$, $\hat{H}^{zy}$, $\hat{H}^{xz}$. Like $\hat{H}$, these marginal heatmaps are sufficient for calculating the expected values of joint coordinates. Conceptually, the volumetric and marginal heatmaps are related as follows :

$$\hat{H}^{xy} = \sum_i \hat{H}_{i,:,:} \ \mid \ \hat{H}^{zy} = (\sum_k \hat{H}_{:,:,k})^T \ \mid \ \hat{H}^{xz} = \sum_j \hat{H}_{:,j,:} \tag{18}$$

There are two ways to calculate each of $E[X]$, $E[Y]$, and $E[Z]$, depending on which marginal heatmap is used. They address this by making coordinate predictions according to

8. Figure 22, and the mathematical definition of *Marginal heatmaps* are firmly adapted from work entitled "3D Human Pose Estimation with 2D Marginal Heatmaps"[44].

equation 19. Due to the xy heatmap having the same orientation as the input image, the x- and y-coordinates are predicted using $\hat{H}^{xy}$ alone. However, the z-coordinate is taken to be the average expectation obtained from the other two heatmaps to better estimate depth.

$$\mu_x =< \hat{H}^{xy}, X_{1,:,:} > \ \mid \ \mu_y =< \hat{H}^{xy}, Y_{1,:,:} > \ \mid \ \mu_z = \frac{1}{2} \times < \hat{H}^{(zy)^T}, Z_{:,:,1} > + \frac{1}{2} \times < \hat{H}^{xz}, Z_{:,1,:} > \tag{19}$$

After converting volumetric heatmaps into marginal heatmaps in order to apply regularization such that heatmap will mimic a specific probability distribution -Spherical Gaussian Distribution-. By making sure the the mean value of the distribution matches the ground-truth joint coordinates. Nibal et al [44] propose to add a regularization term to the $L2$ loss, know as *Jensen-Shannon divergence (JSD)* [29] making the training loss as follows :

$$l_{train} = ||\mu - \tilde{\mu}|| + \lambda(JSD(H_{xy} - \mathcal{N}(\tilde{\mu}_{xy}, \sigma^2)) + JSD(H_{zy} - \mathcal{N}(\tilde{\mu}_{zy}, \sigma^2)) + JSD(H_{xz} - \mathcal{N}(\tilde{\mu}_{xz}, \sigma^2))) \tag{20}$$

where μ, and $\tilde{\mu}$ are predicted joints coordinates, and GT coordiantes respectively. $\mathcal{N}$ represents a spherical Gaussian distribution with mean μ and $\sigma = 1$ pixel for 64×64 heatmaps.

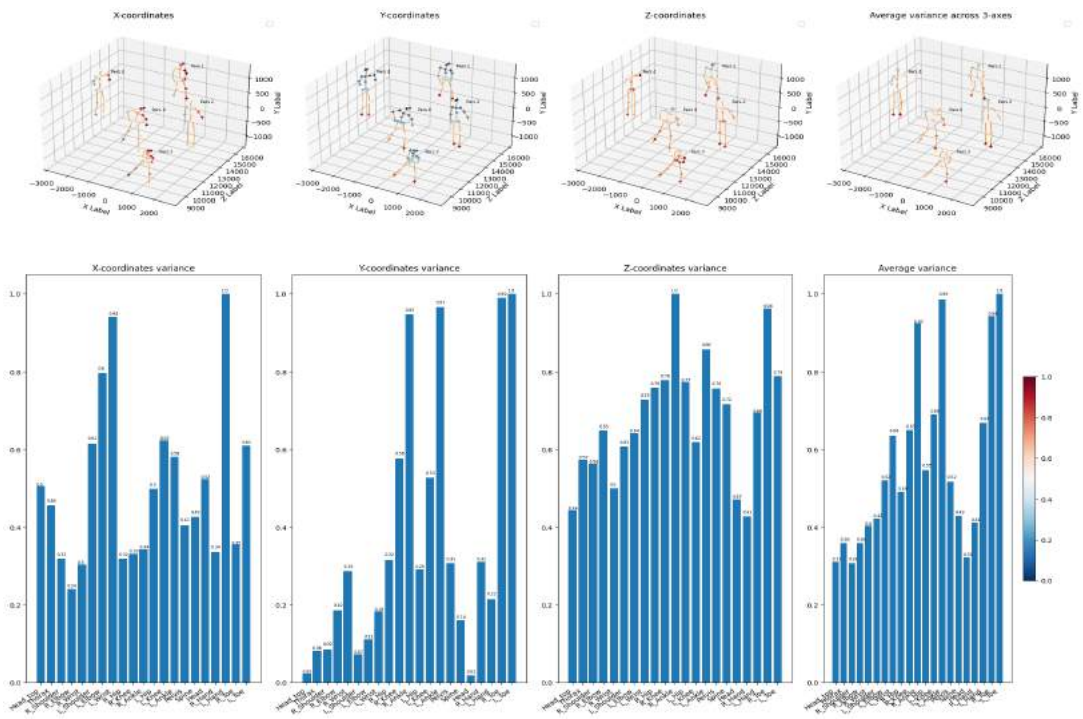


Figure 23. Visualization of the heatmaps variance of a sample image for each joint by original *PoseNet*.

Theoritically, the additional divergence-based loss term helps to make the heatmaps more coherent and interpretable. Figure 24 depicts all three marginal heatmaps generated for a different joints prediction by *PoseNet* after training with JSD regularisation.

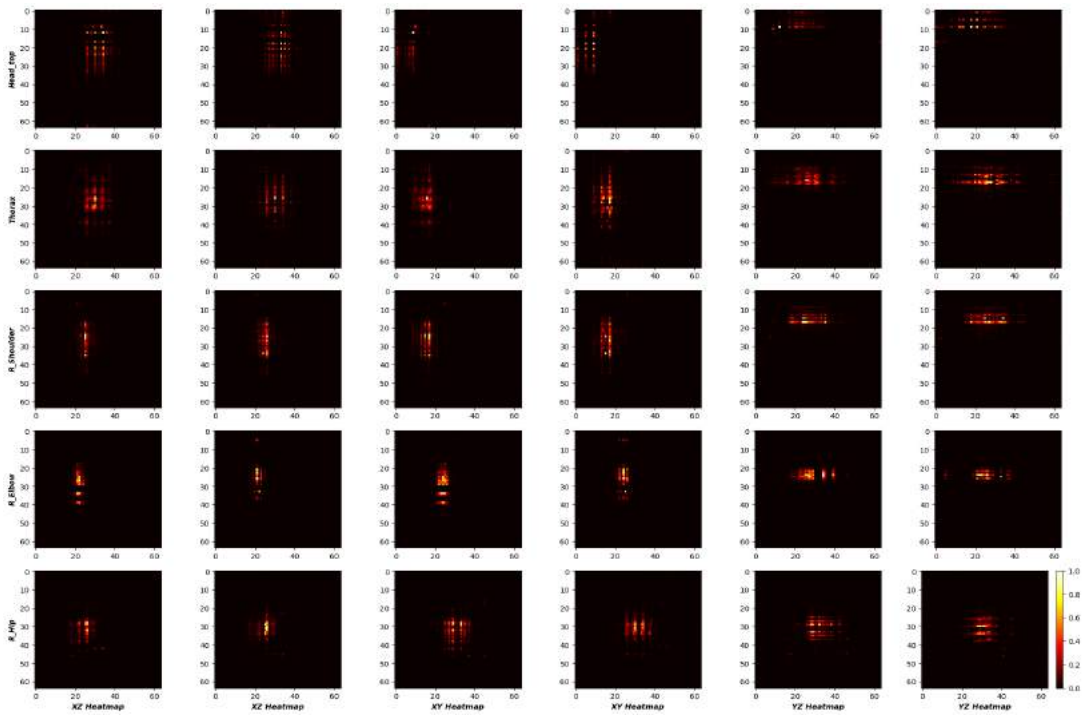


Figure 24. Odd columns represent marginal heatmaps predicted by *PoseNet*. Even columns are marginal heatmaps for regularized *PoseNet* -Regularized Model II- with $\lambda = 0.5$ for each joints.

Table 9. Sequence-wise 3DPCK scores comparison for different models using **all ground-truths**

Seq	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	Avg
Published Model	93.18	75.60	80.31	81.48	84.62	75.35	84.47	69.31	90.13	91.97	80.95	80.96	73.38	73.48	81.79	89.63	88.35	84.28	74.53	70.56	81.2
Regularized Model I	92.64	76.28	76.91	78.61	84.98	69.98	87.21	73.62	88.56	91.06	81.82	83.16	72.45	66.37	76.81	88.27	88.18	80.44	72.91	68.41	79.93
Regularized Model II	92.61	76.72	81.46	78.70	86.95	68.75	85.47	70.72	90.03	92.13	80.92	81.72	71.34	67.61	81.40	88.58	89.01	82.58	74.56	69.10	**80.55**
Regularized Model II	93.18	77.90	76.43	81.25	87.16	69.20	84.88	69.54	89.46	91.65	84.12	82.53	72.47	66.93	81.93	89.02	89.42	81.44	73.40	69.22	80.52

Table 10. Sequence-wise 3DPCK scores comparison for 3 variants of regularized PoseNet model using **only matched ground-truth** sequences

Seq	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	Avg
Published Model	93.88	83.04	80.31	81.48	85.44	75.35	84.47	77.15	90.13	91.97	80.95	80.96	74.25	75.95	81.79	89.63	88.35	84.28	75.46	76.25	82.55
Regularized Model I	93.81	83.43	76.91	78.61	85.47	69.98	87.21	78.23	88.56	91.06	81.82	83.16	73.46	70.46	76.81	88.27	88.18	80.66	74.75	74.19	81.25
Regularized Model II	94.25	83.36	81.46	78.70	87.63	68.76	85.47	77.70	90.03	92.16	80.92	81.72	72.48	72.60	81.40	88.64	89.01	82.58	76.21	74.77	**81.99**
Regularized Model III	94.35	84.28	76.43	81.25	87.5	69.20	84.88	77.17	89.46	91.65	84.12	82.52	73.47	71.10	81.93	89.02	89.43	81.44	74.43	75.23	81.94

The effect of the regularisation was not visually noticable, although it has improved the 3DPCK score for prediction and AUSE score with respect to original *PoseNet* model. We changed the weight for the regularization loss to $\lambda = 0.1, 0.5, 1$. The behavior was consistent.

Table 11. AUSE scores between model and its oracle, Spearman's rank correlation coefficient, on **test dataset MuPoTS**

	with Ground-Truth				**with Predictions**				**AUSE L1**		**AUSE L2**	
Models	X-axis	Y-axis	Z-axis	Average	X-axis	Y-axis	Z-axis	Average	Sum Area	Mean Area	Sum Area	Mean Area
Regularized Model I	0.19057	0.61365	0.83745	0.45026	0.06761	0.11252	0.00265	0.00173	0.0694204	0.16979	0.1672575	0.37819
Regularized Model II	0.19060	0.61225	0.81711	0.45071	0.06807	0.11276	0.00223	0.00150	**0.0593187**	**0.14914**	**0.147185**	**0.33779**
Regularized Model II	0.19057	0.61365	0.83745	0.45026	0.06761	0.11252	0.00265	0.00173	0.0714117	0.17412	0.1771231	0.37920

Références

[1] Giuseppe Airò Farulla, Daniele Pianu, Marco Cempini, Mario Cortese, Ludovico O. Russo, Marco Indaco, Roberto Nerino, Antonio Chimienti, Calogero M. Oddo et Nicola Vitiello. « Vision-Based Pose Estimation for Robot-Mediated Hand Telerehabilitation ». In : *Sensors* 16.2 (2016). issn : 1424-8220. doi : 10.3390/s16020208. url : https://www.mdpi.com/1424-8220/16/2/208.

[2] Lorenzo Bertoni, Sven Kreiss et Alexandre Alahi. *MonoLoco : Monocular 3D Pedestrian Localization and Uncertainty Estimation*. 2019. arXiv : 1906.06059 [cs.CV].

[3] Charles Blundell, Julien Cornebise, Koray Kavukcuoglu et Daan Wierstra. « Weight Uncertainty in Neural Network ». In : *Proceedings of the 32nd International Conference on Machine Learning*. Sous la dir. de Francis Bach et David Blei. T. 37. Proceedings of Machine Learning Research. Lille, France : PMLR, juil. 2015, p. 1613-1622. url : http://proceedings.mlr.press/v37/blundell15.html.

[4] Zhe Cao, Gines Hidalgo, Tomas Simon, Shih-En Wei et Yaser Sheikh. *OpenPose : Real-time Multi-Person 2D Pose Estimation using Part Affinity Fields*. 2019. arXiv : 1812.08008 [cs.CV].

[5] O. Chapelle et M. Wu. *Gradient descent optimization of smoothed information retrieval metrics*. 2010. Information retrieval : 1406.2984.

[6] Ting Chen, Simon Kornblith, Mohammad Norouzi et Geoffrey Hinton. *A Simple Framework for Contrastive Learning of Visual Representations*. 2020. arXiv : 2002.05709 [cs.LG].

[7] Yilun Chen, Zhicheng Wang, Yuxiang Peng, Zhiqiang Zhang, Gang Yu et Jian Sun. *Cascaded Pyramid Network for Multi-Person Pose Estimation*. 2018. arXiv : 1711.07319 [cs.CV].

[8] Yu Chen, Chunhua Shen, Xiu-Shen Wei, Lingqiao Liu et Jian Yang. *Adversarial PoseNet : A Structure-aware Convolutional Network for Human Pose Estimation*. 2017. arXiv : 1705.00389 [cs.CV].

[9] Bowen Cheng, Bin Xiao, Jingdong Wang, Honghui Shi, Thomas S. Huang et Lei Zhang. *HigherHRNet : Scale-Aware Representation Learning for Bottom-Up Human Pose Estimation*. 2020. arXiv : 1908.10357 [cs.CV].

[10] Karl Pearson F.R.S. « LIII. On lines and planes of closest fit to systems of points in space ». In : *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 2.11 (1901), p. 559-572. doi : 10.1080/14786440109462720.

[11] Di Feng, Lars Rosenbaum et Klaus Dietmayer. *Towards Safe Autonomous Driving : Capture Uncertainty in the Deep Neural Network For Lidar 3D Vehicle Detection*. 2018. arXiv : 1804.05132 [cs.RO].

[12] Yarin Gal et Zoubin Ghahramani. *Dropout as a Bayesian Approximation : Representing Model Uncertainty in Deep Learning*. 2016. arXiv : 1506.02142 [stat.ML].

[13] Yarin Gal, Jiri Hron et Alex Kendall. *Concrete Dropout*. 2017. arXiv : 1705.07832 [stat.ML].

[14] John C Gower. *Generalized procrustes analysis*. 1975.

[15] Alex Graves. « Practical Variational Inference for Neural Networks ». In : *Proceedings of the 24th International Conference on Neural Information Processing Systems*. NIPS'11. Granada, Spain : Curran Associates Inc., 2011, p. 2348-2356. isbn : 9781618395993.

[16] Nitesh B Gundavarapu, Divyansh Srivastava, Rahul Mitra, Abhishek Sharma et Arjun Jain. « Structured Aleatoric Uncertainty in Human Pose Estimation. » In : *CVPR Workshops* (2019).

[17] Fredrik K. Gustafsson, Martin Danelljan et Thomas B. Schön. *Evaluating Scalable Bayesian Deep Learning Methods for Robust Computer Vision*. 2020. arXiv : 1906.01620 [cs.LG].

[18] Kaiming He, Georgia Gkioxari, Piotr Dollár et Ross Girshick. *Mask R-CNN*. 2018. arXiv : 1703.06870 [cs.CV].

[19] Kaiming He, Xiangyu Zhang, Shaoqing Ren et Jian Sun. *Deep Residual Learning for Image Recognition*. 2015. arXiv : 1512.03385 [cs.CV].

[20] Eddy Ilg, Özgün Çiçek, Silvio Galesso, Aaron Klein, Osama Makansi, Frank Hutter et Thomas Brox. *Uncertainty Estimates and Multi-Hypotheses Networks for Optical Flow*. 2018. arXiv : 1802.07095 [cs.CV].

[21] Sergey Ioffe et Christian Szegedy. *Batch Normalization : Accelerating Deep Network Training by Reducing Internal Covariate Shift*. 2015. arXiv : 1502.03167 [cs.LG].

[22] Catalin Ionescu, Dragos Papava, Vlad Olaru et Cristian Sminchisescu. « Human3.6M : Large Scale Datasets and Predictive Methods for 3D Human Sensing in Natural Environments ». In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36.7 (juil. 2014), p. 1325-1339.

[23] Alex Kendall et Yarin Gal. *What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision ?* 2017. arXiv : 1703.04977 [cs.CV].

[24] Diederik P. Kingma et Jimmy Ba. *Adam : A Method for Stochastic Optimization*. 2017. arXiv : 1412.6980 [cs.LG].

[25] A. Der Kiureghian et O. Ditlevsen. *Aleatory or epistemic ? does it matter ?* 2017.

[26] Claudia Kondermann, Rudolf Mester et Christoph Garbe. « A Statistical Confidence Measure for Optical Flows ». In : *Computer Vision – ECCV 2008*. Sous la dir. de David Forsyth, Philip Torr et Andrew Zisserman. Berlin, Heidelberg : Springer Berlin Heidelberg, 2008, p. 290-301.

[27] Balaji Lakshminarayanan, Alexander Pritzel et Charles Blundell. *Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles*. 2017. arXiv : 1612.01474 [stat.ML].

[28] Yining Li, Chen Huang et Chen Change Loy. *Dense Intrinsic Appearance Flow for Human Pose Transfer*. 2019. arXiv : 1903.11326 [cs.CV].

[29] J. Lin. « Divergence measures based on the Shannon entropy ». In : *IEEE Transactions on Information Theory* 37.1 (1991), p. 145-151. doi : 10.1109/18.61115.

[30] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan et Serge Belongie. *Feature Pyramid Networks for Object Detection*. 2017. arXiv : 1612.03144 [cs.CV].

[31] Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick et Piotr Dollár. *Microsoft COCO : Common Objects in Context*. 2015. arXiv : 1405.0312 [cs.CV].

[32] Ting Liu, Jennifer J. Sun, Long Zhao, Jiaping Zhao, Liangzhe Yuan, Yuxiao Wang, Liang-Chieh Chen, Florian Schroff et Hartwig Adam. *View-Invariant, Occlusion-Robust Probabilistic Embedding for Human Pose*. 2020. arXiv : 2010.13321 [cs.CV].

[33] Diogo C. Luvizon, David Picard et Hedi Tabia. *2D/3D Pose Estimation and Action Recognition using Multitask Deep Learning*. 2018. arXiv : 1802.09232 [cs.CV].

[34] Diogo C. Luvizon, Hedi Tabia et David Picard. *Human Pose Regression by Combining Indirect Part Detection and Contextual Information*. 2017. arXiv : 1710.02322 [cs.CV].

[35] Laurens van der Maaten et Geoffrey Hinton. *Visualizing data using t-SNE*. 2008.

[36] Julieta Martinez, Rayat Hossain, Javier Romero et James J. Little. *A simple yet effective baseline for 3d human pose estimation*. 2017. arXiv : 1705.03098 [cs.CV].

[37] Dushyant Mehta, Helge Rhodin, Dan Casas, Pascal Fua, Oleksandr Sotnychenko, Weipeng Xu et Christian Theobalt. *Monocular 3D Human Pose Estimation In The Wild Using Improved CNN Supervision*. 2017. arXiv : 1611.09813 [cs.CV].

[38] Dushyant Mehta, Oleksandr Sotnychenko, Franziska Mueller, Weipeng Xu, Srinath Sridhar, Gerard Pons-Moll et Christian Theobalt. *Single-Shot Multi-Person 3D Pose Estimation From Monocular RGB*. 2018. arXiv : 1712.03453 [cs.CV].

[39] Lu Mi, Hao Wang, Yonglong Tian et Nir Shavit. *Training-Free Uncertainty Estimation for Dense Regression : Sensitivity as a Surrogate*. 2021. arXiv : 1910.04858 [cs.CV].

[40] Gyeongsik Moon, Ju Yong Chang et Kyoung Mu Lee. *Camera Distance-aware Top-down Approach for 3D Multi-person Pose Estimation from a Single RGB Image*. 2019. arXiv : 1907.11346 [cs.CV].

[41] Jishnu Mukhoti et Yarin Gal. *Evaluating Bayesian Deep Learning Methods for Semantic Segmentation*. 2019. arXiv : 1811.12709 [cs.CV].

[42] Farhood Negin, Saeid Agahian et Cemal Köse. « Improving bag-of-poses with semi-temporal pose descriptors for skeleton-based action recognition ». In : *The Visual Computer* (fév. 2018). doi : 10.1007/s00371-018-1489-7. url : https://hal.inria.fr/hal-01849283.

[43] Alejandro Newell, Kaiyu Yang et Jia Deng. *Stacked Hourglass Networks for Human Pose Estimation*. 2016. arXiv : 1603.06937 [cs.CV].

[44] Aiden Nibali, Zhen He, Stuart Morgan et Luke Prendergast. *3D Human Pose Estimation with 2D Marginal Heatmaps*. 2018. arXiv : 1806.01484 [cs.CV].

[45] Matteo Poggi, Filippo Aleotti, Fabio Tosi et Stefano Mattoccia. *On the uncertainty of self-supervised monocular depth estimation*. 2020. arXiv : 2005.06209 [cs.CV].

[46] Tim Salimans, Diederik P. Kingma et Max Welling. *Markov Chain Monte Carlo and Variational Inference : Bridging the Gap*. 2015. arXiv : 1410.6460 [stat.CO].

[47] Philip Sedgwick. « Spearman's rank correlation coefficient ». In : *BMJ (online)* 349 (nov. 2014), g7327. doi : 10.1136/bmj.g7327.

[48] Guanya Shi, Yifeng Zhu, Jonathan Tremblay, Stan Birchfield, Fabio Ramos, Animashree Anandkumar et Yuke Zhu. *Fast Uncertainty Quantification for Deep Object Pose Estimation*. 2021. arXiv : 2011.07748 [cs.RO].

[49] Aliaksandr Siarohin, Enver Sangineto, Stephane Lathuiliere et Nicu Sebe. *Deformable GANs for Pose-based Human Image Generation*. 2018. arXiv : 1801.00055 [cs.CV].

[50] Xiao Sun, Bin Xiao, Fangyin Wei, Shuang Liang et Yichen Wei. *Integral Human Pose Regression*. 2018. arXiv : 1711.08229 [cs.CV].

[51] Jonathan Tompson, Arjun Jain, Yann LeCun et Christoph Bregler. *Joint Training of a Convolutional Network and a Graphical Model for Human Pose Estimation*. 2014. arXiv : 1406.2984 [cs.CV].

[52] Alexander Toshev et Christian Szegedy. « DeepPose : Human Pose Estimation via Deep Neural Networks ». In : *2014 IEEE Conference on Computer Vision and Pattern Recognition* (juin 2014). doi : 10.1109/cvpr.2014.214. url : http://dx.doi.org/10.1109/CVPR.2014.214.

[53] Shih-En Wei, Varun Ramakrishna, Takeo Kanade et Yaser Sheikh. *Convolutional Pose Machines*. 2016. arXiv : 1602.00134 [cs.CV].

[54] Bin Xiao, Haiping Wu et Yichen Wei. *Simple Baselines for Human Pose Estimation and Tracking*. 2018. arXiv : 1804.06208 [cs.CV].

[55] Wei Yang, Shuang Li, Wanli Ouyang, Hongsheng Li et Xiaogang Wang. *Learning Feature Pyramids for Human Pose Estimation*. 2017. arXiv : 1708.01101 [cs.CV].

[56] Long Zhao, Xi Peng, Yu Tian, Mubbasir Kapadia et Dimitris N. Metaxas. « Semantic Graph Convolutional Networks for 3D Human Pose Regression ». In : *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (juin 2019). doi : 10.1109/cvpr.2019.00354. url : http://dx.doi.org/10.1109/CVPR.2019.00354.

2 Ethics4EU Poster: Is GPT-3 The Type of AI We Are Looking For From Ethical Perspective ?

My poster for "Digital Ethics: Can we do better?" workshop at Ethics4EU 2021

Digital Ethics – Can We Do Better?

Is GPT-3 The Type Of AI We Are Looking For From Ethical Perspective ?

Imad Eddine MAROUF, *imad.marouf@ip-paris.fr*

Introduction

GPT-3 is autoregressive language model that learns from existing text and produce human-like text. As stated by OpenAI, the purpose of developing this GPT-3 was to "greatly lower the barrier to producing beneficial AI-powered products, resulting in tools and services that are hard to imagine today." Making AI more accessible and easy to use is a well-intentioned advancement. While the intended use of this technology is hopeful about benefitting society, it is only in beta form because its potential power and actual use raised many worries and questions.

In this poster, I describe 4 ethical issues raised by using GPT-3 motivated by real-world scenarios. I conclude by discussing the open questions spurred by these issues, and suggesting solutions to them.

Ethical Issues

In this section, we will pipoint 4 ethical issues related to GPT-3 usage.

1- Racial & Religous Biases:

OpenAI investigated racial & religous bias by asking questions about races and prompted GPT-3 to describe the practioners of the belief system. They found that the model tended to describe religions in a similar way that they are presented today, stereotypes and all. For examples, words like "Racists" co-occured with Judaism, and "terrorism" with Islam. Although, heavy curation was done on the data collected (Wikipedia, Common Crawl, Books), because accepting large amounts of web text as 'representative' of 'all' of humanity we risk perpetuating dominant viewpoints, increasing power imbalances, and further reifying inequality.

Is GPT-3 ready to be deployed for business applications?

2- Environmental and Energy Considerations

Compared to well-know models nowadays, GPT-3 is of magnitudes larger in scale (175 Billion parameters), requiring an estimated 28000 GPU-days to train excluding R&D. Bigger may be better when it comes to performance, yet at what cost does this come to the planet? Carbontracker [7] reckons training GPT-3 just once requires the same amount of power used by *126 homes in Denmark per year, or driving to the Moon and back.*

Is is really necessary to create GPT-3 with this amount of energy ?

3- Intellectual Property and Plagiarism

Language models like GPT-3 that are capable of generating large, realistic text corpora pose the risk of providing malicious actors the opportunity to commit identify fraud, falsify academic essays. Plathera of sites are using GPT-3 API to produce essays for students freely or with small fees like *Shortlyai.com, philosopherai.com.* Which raises concerns about the plagiarism and authorship credits.

How can we value collaborative writing between humans and machines ?

4- Black-Box Model with Unpredicted Output

GPT-3 prediction process is based on stochastic process know as temperature sensitivity - The temperature controls how much randomness is in the output.- This property makes predicting GPT-3 output impossible and could lead to unintended consequences, especially in critical usage cases like health-care, finance, and education.

Should we use obaque AI models ?

Potential Misuses

In this section, we would like to analyse some examples of GPT-3 misuses.

1- Spamming

A bot powered by GPT-3 was found to be interacting with people in a Reddit thread. This bot masked itself as a human Redditor and created several comments before spotted by another human Redditor P. Winston. Apparently, as per Winston, the text generator by the GPT-3 bot matches the output of Philosopher AI, a tool powered by GPT-3, which answers questions on philosophy. This case, proves that such tools like GPT-3 are capable of widespread misinformation, create spam, racist opinions and intensifying discrimination againt minorities.

Fig. 1: A GPT-3 Bot interacting with Reddit users (5)

2- Intellectual Property and Plagiarism

Language Models (LMs) like GPT-3 that are capable of generating large, realistic text corpora pose the risk of providing malicious actors the opportunity to commit identify fraud, falsify academic essays. Several cases have been witnessed of using GPT-3 to write articles, unlikely to be distinguished from human-writing style. A journalist at The Guadian published an article written by GPT-3, went viral shared more than 71,624 time [4]. Alarming, on how can we evaluate human skills (writing, numerical painting, web design) if they are aided by machines. Using machines as aid for creative skills has economical consequences, since it shortens the production by magnitudes for authors when drafting essays or designs for paintings.

Fig. 2: Collaborative writing with GPT3, The Guardian (4)

3- Medical Q&A

Start-ups started fine-tunning GPT-3 in order to deploy it as medical assitant without further consideration of the dangers of that since no transparency is provided on how GPT-3 provides its inferences and answers.

Fig. 3: Medical QA using GPT-3, Test done by Nabla Start-up (3)

Discussions and Opinions

GPT-3 spews hate speech and generate sexist stereotypes, if prompted —faithfully reflecting the associations in its training data. It outputs overtly abusive language [8], then a set of risks arises such as: propagating or proliferating overtly abusive views, amplifying abusive language - abusive language that may be used in the next iteration of data collection-. We believe those drawbacks should prevent large models from being deployed to public usage or companies, since the risks are very crucial. Although, OpenAI started selling GPT-3 API which will widespread miuses [9].

Fig. 4: Gary Marcus (Emiritus Professor at NYC University) feedback on OpenAI selling GPT-3 API

GPT-3 sometimes give nonsensical answers ("A pencil is heavier than a toaster") or outright dangerous replies, since it lacks causal reasing and inference-interpretability which are essential to have transparent ethical AI. We cannot rely on models where the output is unpredictable, especially in high-stack applications. Hype created by media comparing GPT-3 to *Artificial General Intelligence*, this kind of over-expectation could lead to what's know *AI winter* [6], reducing funding, and jobs when research output doesn't go as expected. Public awareness, and humble enthisiam about AI progress are important to keep momentum in developing technologies.

Recommendations

LMs trained on large, uncurated, static datasets from the Web encode hegemonic views that are harmful to marginalized populations. We need to invest significant resources into curating and documenting training data, work on projects that build towards a technological ecosystem whose benefits are at least evenly distributed or better accrue to marginalized populations. Considering the financial and environmental costs of model development up front, and how it will shape the field afterwards. Researchers must report training time and sensitivity to hyperparameters when the released model is meant to be re-trained to reduce carbon footprint.

On the other hand, research should focus on instilling models with causal reasoning, and common sense in order to mitigate biases. Explainability and interpretability on such models are important features, to have transparent ethical AI.

Finally, filling legal voids about usage of LMs in critical domains such as justice, and health-care by incorporating effors of AI ethicists and law firms, to strengthen regulations, and put clear context about the allocation of rights/credits to human-machine collaborations. Investigation by goverments/public about whether companies are allowed to deploy such systems to be used by companies altough their biases and risks are apparent.

References

1 - "Language models are few-shot learners." arXiv:2005.14165 (2020).
2 - A student used GPT-3 to write fake blog posts, The Verge, (16/08/2020)
3 - Doctor GPT-3: hype or reality ? Nabla Blog, (27/10/2020)
4 - A robot wrote this entire article. Are you scared yet, human? by The Guardian
5 - A GPT-3 Bot Posted On Reddit, AnalyticsIndiaMag, (09/10/2020)
6 - Why AI is harder Than We think, Melanie Mitchell (2021).
7 - Carbontracker: Tracking and Predicting Carbon Footprint, W. Anthony
8 - RealToxicityPrompts: Evaluating Neural Toxic in Models, Gehman et al, (2020).
9 - The GPT-3 Economy, BDTechTalks.com, (21/09/2020).

3 Joint Unsupervised IR-RGB Video Registration and Fusion

Joint Unsupervised Infrared-RGB Video Registration and Fusion

Imad Eddine Marouf [1,2] *Luca Barras[2]* *Hakki Can Karaimer[2,3]* *Sabine Süsstrunk[2]*
[1] *Institut Polytechnique de Paris*
[2] *School of Computer and Communication Sciences (IC), Ecole Polytechnique Fédérale de Lausanne (EPFL)*
[3] *Advanced Micro Devices, Inc. (AMD)*
imad.marouf@ip-paris.fr luca.barras@epfl.ch hakki.karaimer@epfl.ch sabine.susstrunk@epfl.ch

Abstract

We present a system to perform joint registration and fusion for RGB and Infrared (IR) video pairs. While RGB is related to human perception, IR is associated with heat. However, IR images often lack contour and texture information. The goal with the fusion of the visible and IR images is to obtain more information from them. This requires two completely matched images. However, classical methods assuming ideal imaging conditions fail to achieve satisfactory performance in actual cases. From the data-dependent modeling point of view, labeling the dataset is costly and impractical.

In this context, we present a framework that tackles two challenging tasks. First, a video registration procedure that aims to align IR and RGB videos. Second, a fusion method brings all the essential information from the two video modalities to a single video. We evaluate our approach on a challenging dataset of RGB and IR video pairs collected for firefighters to handle their tasks effectively in challenging visibility conditions such as heavy smoke after a fire, see our project page.

Introduction and Motivation

The mission of firefighters is to rescue people and animals from hazardous fire. Challenging working conditions such as smokes, toxic fumes, and superheated glasses make it very difficult to save lives. In 2011, 70,090 firefighters in the U.S. alone were injured in the line of duty with, 61 deaths [5, 12, 16, 22]. Over 60% of the firefighter deaths and over 20% of the firefighting injuries are caused by exposure to fire conditions such as smoke inhalation, burns, overexertion/stress, or being trapped [7, 13, 16]. Thus, they cannot perform effectively in smoke-filled environments where low visibility and high temperature are present. Poor visibility might lead to influences of human behavior such as redirection of movement and their initial response speed [2, 3, 11]. As a solution, firefighters use IR cameras to aid in seeing through smokes and detect the variation of temperature in the surroundings. While IR cameras' response is related to the temperature within the captured frame, RGB cameras' response is related to human perception.

In order to overcome these limitations, we provide an effective framework to align and fuse the scenes captured by an IR/RGB camera pair. This helps to have a full view of the environment and facilitates firefighters' missions in extreme conditions. We present our results on a dataset containing IR/RGB video pairs of scenes that mimic working environments for firefighters. The dataset comprises IR-RGB video pairs captured by two cameras attached to a head-mounted wearable device. However, it suffers from misalignment and the sensing differences between the two cameras.

Contribution: This paper presents a deep-learning-based framework for unsupervised joint registration and fusion for RGB-IR video pairs to produce well-aligned and fused videos. Thus fusion mutually complements the drawbacks of each sensing device and maximizes the vision capability within the environment. Our proposed method is evaluated on a very challenging dataset consisting of videos mimicking the extreme working conditions of firefighters.

Related Work

Image Registration: Demand for fast and accurate registration methods motivated the development of deep learning methods based on transformation estimation techniques. Challenges associated with generating ground truth data have recently led many researchers to develop unsupervised frameworks. Two recent works [4, 17], presents an unsupervised learning-based image registration methods. Both propose a neural network consisting of a CNN and a spatial transformation function [10] that warps images to one another. However, these two initial methods are only demonstrated on limited subsets, such as 3D sub-regions [17] or 2D slices [4], and support only small transformations [4]. All above methods were demonstrated on medical images. Rare works has been done on non-medical images such as [9], where the method relies on detecting corners between input pairs and evaluating using a similarity metric. In their seminal work, Firmenichy et al. [6] applies registration between Near-Infrared and RGB pairs based on detection of the feature points.

Image Fusion: The earliest fusion work involving neural networks poses multi-focus fusion as a classification task [19]. Three focus measures define the input features to a shallow network which outputs the weight maps corresponding to the source images. Due to architectural constraints, the method can only run on image patches, and generates boundary artifacts. More recently, convolutional neural networks have been trained to generate decision maps for multi-focus [21], multiexposure [23], medical [20], and thermal fusion [18]. Although these approaches often achieve better performance than their classical counterparts, they still have major drawbacks. First, they require large datasets for training. Second, deep networks often overfit the datasets they are trained on, e.g., a network trained for multi-focus image fusion will only be suitable for that task. The method proposed in [15] does not require training, which alleviates the necessity of collecting data by using a pre-trained network as a feature extractor.

Background and Preliminaries

Image Registration

Image registration is the process of transforming different images into one coordinate system with matched scene contents. Applications include medical imaging, remote sensing, and 3-D computer vision. Registration may be necessary when analyzing a pair of images acquired from different viewpoints, at different times, or using various sensors/modalities like IR and visible images. Recently, the need for fast and accurate registration

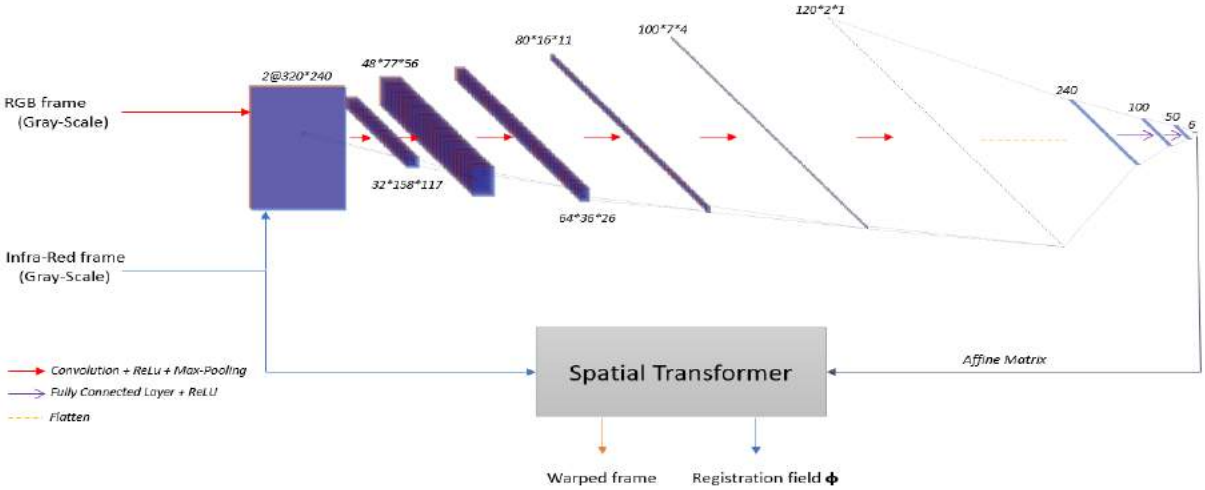


Figure 1. *Affine network architecture: The affine network is a spatial transformer network. It takes a 2-channel image obtained by concatenating the IR frame and gray-scale of the RGB frame as input. After a number of convolution and non-linear layers, it generates a warped frame and a registration field.*

methods encouraged researchers to propose deep-learning-based transformation estimation techniques.

VoxelMorph Network Firstly, we evaluated an unsupervised registration model [1] since it is state-of-the-art in medical image registration. Dalca et al. [1] propose a CNN function with parameters shared across population. This makes it possible for registration to be achieved through a function evaluation, which can be optimized for various cost functions. The model is fed by a pair of fixed and moving images, which are RGB and IR volumes in our case.

The VoxelMorph model is similar the UNet architecture [25] consisting of an encoder-decoder with skip connections. The network takes fixed f, moving m volumes and apply convolutions. This is followed by Leaky ReLU activations in both the encoder and decoder stage. The convolutional layers capture hierarchical features of the input image pair necessary to estimate the correspondence registration field Φ defined as $g_\theta = (m; f) = \Phi$, where the goal is to optimize the learnt parameters θ to estimate a deformation field Φ. The authors propose an unsupervised loss consisting of two terms defined as:

$$L_{us}(f, m, \Phi) = L_{sim}(f, m \cdot \Phi) + \lambda L_{smooth}(\Phi), \tag{1}$$

$$L_{sim}(f, m \cdot \Phi) = \frac{1}{\Sigma} \sum_{p \in \Sigma} [f(p) - [m \cdot \Phi](p)]^2, \tag{2}$$

$$L_{smooth}(\Phi) = \sum_{p \in \Sigma} || \bigtriangledown u(p)||^2. \tag{3}$$

For each pixel p, we compute a (subpixel) pixel location $\tilde{p} = p + u(p)$ in m. Because image values are only defined at integer locations, we linearly interpolate the values at the eight neighboring voxels:

$$m \cdot \Phi(p) = \sum_{q \in Z(\tilde{p})} m(q) \prod_{d \in x,y,z} (1 - |\tilde{p}_d - q_d|). \tag{4}$$

Minimizing L_{sim} will help $m \cdot \Phi$ approximate f but may generate a non-smooth Φ that could be physically impractical. Here $Z(\tilde{p})$ are the pixel neighbors of p, and d iterates over dimensions of Φ. Because it is possible to compute gradients, the model can backpropagate errors during optimization.

The fact that this method is unsupervised, we need a way to know if the deformation field Φ is doing good by making sure that $m \cdot \Phi$ (m warped by Φ) is close to f. Regularizing θ makes the deformation field smooth. A spatial transformation function, $T(\Phi)$, is used to interpolate neighboring voxels' heights to overcome the shortcoming of the model being spatially invariant to the input data. The use of spatial transformer helps the model remain invariant to translation, scale, rotation. This also makes the overall system capable of modeling more generic warping.

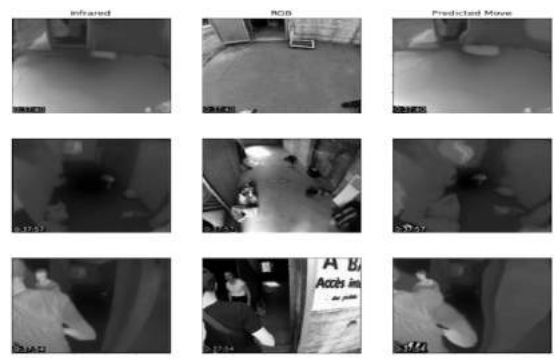


Figure 2. *Applying the state-of-the-art VoxelMorph model directly on our IR-RGB pairs performs poorly on the challenging scenes in our dataset. We employed a segmentation procedure to tackle this problem leveraging the IR camera's response based on temperature.*

Spatial Transformation The spatial transformer network (STN) proposed by Jaderberg et al. [10] was one of the first methods that exploited deep learning for image alignment. The STN is designed as part of a neural network. The goal of the STN is to spatially transform input images such that the image registration is simplified. Transformations might be performed using a global transformation model or a thin plate spline model. In the application of an STN, image registration is an implicit result.

Image Fusion

Image fusion is the process of combining multiple input images into a single output image that contains a better description of the scene than the one provided by any. The applications include night-time surveillance, military reconnaissance missions, and firefighting by fusing visible and IR images. In this paper, we are targeting fusion of IR and RGB images using state-of-the-art method [15] which uses a pretrained VGG19 model [26] considered as feature extractor. Therefore, this method alleviates

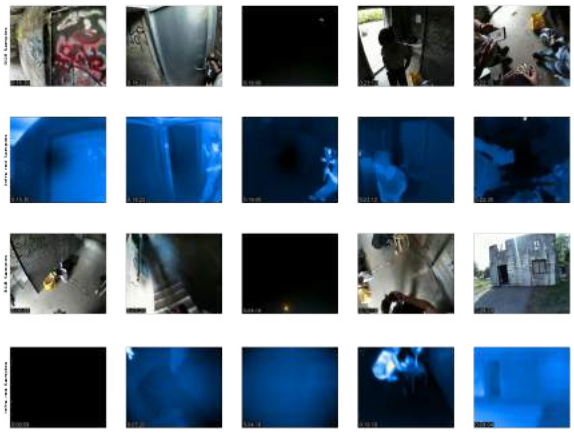


Figure 3. *Image samples from our dataset. The data is captured with the two cameras attached to a head-mounted wearable device in various indoor and outdoor conditions to mimic the working situations for firefighters ranging from extremely dark scenes to very bright ones.*

the need to do training and collection of data, and can generalize well to any type of fusion. The method works as follows: First, it decomposes both input images (IR, RGB) into base layer representing large scale intensity variation, and a detail layer containing small scale changes to avoid mixing low and high frequency information and reduces halo artifacts. Base layer is obtained by applying smoothing filter on the image, and the detail layer is the difference between the original image and the base layer. Then, the base layers are fused based on visual saliency maps reflecting the image's essential features. The detail layers are combined using deep feature maps extracted from detail image sources using a pre-trained network according to their activation levels at each layer in the pre-trained network. Each feature map of the detail layer indicates the contribution of the image at a specific pixel where high pixel values correspond to high activity. Finally, after obtaining both the base and detail layers, the final fused image combines both layers pixel-by-pixel and removes out-of-range pixel values of the fused image.

Proposed Method

Our framework consists of two stages: First, we created a spatial transformer network named *affine network*. The affine network takes IR-RGB pairs as input and produces a flow field. Flow field output of the affine network will apply the transformation on the IR frames. Second, a fusion network takes a registered image and produces an image that will complement all the essential information of the well-matched IR-RGB pairs. This is followed by alpha-blending to color the resulting image.

Affine network The affine network is a spatial transformer network. It takes a 2-channel image obtained by concatenating the IR frame and gray-scale of the RGB frame as input. The input size is $2 \times 320 \times 240$ in our case. The network begin with a succession of 2D convolutions followed by ReLU activation with kernel size of 7×7, 5×5, 5×5, 5×5, 3×3, 3×3 respectively. All the convolutions are done with a stride of size 1. Successively, we flatten the output of convolution features, followed by fully connected layers of size $240, 100, 50, 6$. Output of the network can be seen as affine matrix of size 2×3. In the spatial transformer we apply the affine matrix on the IR image and get the first output (warped image) of the network. The second output is the flow field generated by the affine matrix. While the first output has a size of 320×240, the second output is $2 \times 320 \times 240$. The two channels are because we have a channel

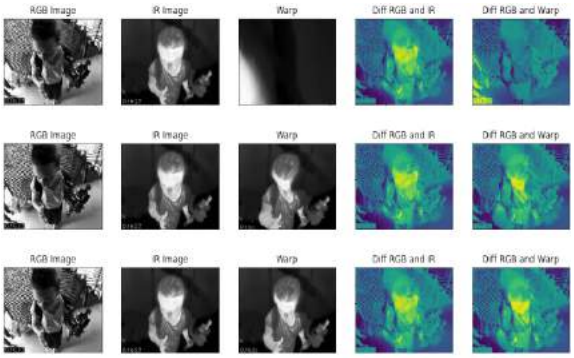


Figure 4. *IR-RGB registration with the VoxelMorph architecture and the Affine network. For small λ values, such as $\lambda = 0.2$, VoxelMorph (1st, 2nd row) poorly warps the output image and causes misaligned results. For larger λ values, such as $\lambda = 0.6$, the output is less warped but still poorly aligned. With the affine network (3rd row), the output looks only scaled and misaligned with the RGB frame.*

for each direction (x, y). Figure 1 shows the visualization of the architecture.

The loss function computes the similarity between the RGB image (fixed input) and the warped image. It can be expressed as:

$$L(F, M, \theta) = MSE(f, m \cdot \Phi) = \frac{1}{\Sigma} \sum_{p \in \Sigma} [f(p) - [m \cdot \Phi](p)]^2, \quad (5)$$

where F is the RGB image, M is the IR image and $M(\Phi)$ is the IR image after processed by the spatial transformer. L_{sim} is the mean square error in our experiment. We use an architecture very similar to VoxelMorph but we start with a CNN to regress to a 2×3 affine matrix, and apply this matrix on the image. So the shape in the image cannot be deformed because we apply an affine matrix on the image. VoxelMorph applies a flow field in the spatial transform component. So the pixels can move anywhere. This is the reason why $L_{smooth}(\Phi)$ loss term exists in the loss function. This is unnecessary for the affine network because the linearity of the transformation is guaranteed with the affine matrix.

Segmentation Applying VoxelMorph or Affine model directly on our IR-RGB pairs performs poorly as in Figure 2. This is mostly because the input pairs are not well aligned, and the objects in the images varied in size. This makes the problem more challenging than medical imaging benchmarks, as RGB/IR pairs are not segmented. The difference between IR and RGB images is mostly at the borders of the captured scene, which is different from the well centered and segmented medical imaging benchmark datasets where VoxelMorph is mostly evaluated. In order to resolve this, we apply semantic segmentation on the RGB pairs using the pretrained Mask-RCNN model [8]. Similarly, the IR images are segmented using the adaptive thresholding method depending on the frame sequences. Then the image registration is applied to the segmentation result. This way, the model learns to align the result of the segmentation.

Mask-RCNN [8] is a deep neural network proposed to tackle the semantic segmentation problem in computer vision. It takes an image as input and produces the bounding boxes for objects, classes, and masks. Mask-RCNN is a two-stage framework: The first 10 stage scans the image and generates proposals (i.e., areas likely to contain an object). The second stage classifies the proposals and generates bounding boxes and masks. Both stages are connected to the backbone structure. Mask-RCNN is an extension of Faster R-CNN [24] with an extra mask head. The

Figure 5. *IR-RGB registration with the VoxelMorph architecture and the Affine network. When VoxelMorph (2nd and 3rd row) is trained with $\lambda = 0.6$ and $\lambda = 1$, it poorly deforms the shape of the objects in the image. When compared visually, the affine network (1st row) achieves better results.*

extra mask head allows us to pixel-wise segment each object and extracts each object separately from the background.

Experimental Setup and Analysis

Dataset We demonstrate our work on IR and visible video pairs with varying duration ranging from 6 minutes to 12 minutes. The data is captured with the two cameras attached to a head-mounted wearable device to mimic the working situations for firefighters ranging from extremely dark scenes to very bright ones. Therefore, before tackling the registration, we had to apply pre-processing by converting our videos into frames and resizing them to 320 $\times$ 240 size. We generated the masks using Mask-RCNN [8] corresponding to the frames. The masks were obtained only for persons. So in this part, we use only frames that contain humans. For the RGB video frames, we use a pre-trained Mask-RCNN to segment the images. For the IR video frames, we applied thresholding based on the pixels belong to humans.

Training Details For the Voxelmorph network, we adapted the official Pytorch implementation with adjustments to fit our 2D image dataset, VoxelMorph is initialised with the number of features for encoder and decoder as [256, 256, 256, 256], [256, 256, 256, 256, 256, 128]. For the two networks, we use Adam optimizer [14] with a learning rate of 10^{-4}. To speed up the learning, we use a batch of 20 pairs of images for the two networks. To segment the RGB frames, we use the official pre-trained Mask-RCNN. To better see if an IR frame is aligned with its corresponding RGB frame, we implemented a visualization method that shows IR image, RGB image, warped image, the difference between the RGB and IR image, and the difference between the RGB and the warped image (see Figure 5). Then we fuse the resulting image using [15]. This is followed by alpha blending as an interpolation between the two images. The formula is given by: αY + (1 - α)F, where F is the original RGB frame and Y is the fused IR-RGB frame.

Results

In this section, we present our results. First, without employing any segmentation, then using a semantic segmentation procedure for humans.

Without segmentation After training both networks for 20 epochs, the results without segmentation are not satisfactory (see Figure 4). When we increase the number of epochs, the results become worse. In another approach, we vary λ between 0.1 to 0.6 for the first network. For small λ values, the output image is overly warped and not aligned. Larger values for λ further warp the output but make it poorly aligned.

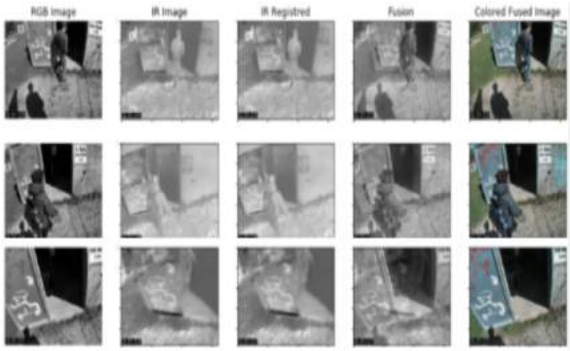


Figure 6. *Fusion of IR/RGB wrapped after segmentation, using Zero-Fusion [15]. The right-most column corresponds to applying alpha-blending to fused IR-RGB images.*

Figure 7. *The white t-shirt and the black t-shirt on the RGB frame appear the same on the IR frame. Although the IR-RGB pair are quite different, this results in a very low loss score.*

Using semantic segmentation for humans: When we employ semantic segmentation for humans, we train the network 100 epochs and vary λ between 0.6 and 1. This way, VoxelMorph achieves better alignment performance. However, it deforms the image's shape. So visually, the affine network achieves better results. Results are shown in Figure 5.

IR-RGB Fusion The fusion process is applied after IR-RGB registration. Figure 6 illustrates the results from the fusion process using the registered IR frame with original gray-scale image to complement two imaging modalities. It brings essential information about the scene. The fusion process is followed by alpha blending to highlight the original colors of the image.

Discussion The results turn much better with the segmentation (see Figure 5). Applying VoxelMorph registration directly leads to poor results because we cannot compare the RGB frame with the IR frame. After all, pixel values are not correlated. We explain this with an example in Figure 7. As in Figure 7, the IR camera's response is the same for the two persons regardless of the two different colors and textures of the t-shirts. So if we compute the mean square error between the IR and RGB images, this will measure the similarity between the two images which is not desired. The only correlation between these two is the borders of the objects.

Concluding Remarks

We presented a system for tackling the joint IR-RGB image registration and fusion problem to assist firefighters in performing their missions in extreme visibility conditions. For this challenging task, we made significant progress in our attempts at targetting cases very similar to the working conditions of firefighters. Our results provide a backbone for future improvements to overcome our limitations on the segmentation of the video pairs.

Acknowledgments This study was funded by the Hasler Foundation (grant no. 16076, S.A.V.E.). We thankfully acknowledge their support.

References

[1] G. Balakrishnan, A. Zhao, M. R. Sabuncu, J. V. Guttag, and A. V. Dalca. VoxelMorph: A Learning Framework for Deformable Medical Image Registration. *arXiv*, 2018.

[2] J. Bryan. Behavioral response to fire and smoke. *Society of Fire Protection Engineers*, 2, 2002.

[3] G. Cook and M. Wright. The effects of smoke on people's walking speeds using overhead lighting and wayguidance provision. *International Symposium on Human Behaviour in Fire*, 2001.

[4] B. D. de Vos, F. F. Berendsen, M. A. Viergever, M. Staring, and I. Isgum. End-to-end unsupervised deformable image registration with a convolutional neural network. *Lecture Notes in Computer Science*, page 204–212, 2017.

[5] Fire Incident Data Organization. Deadliest Fires in the U.S. with 5 or more Firefighter Deaths at the Fire Grounds, 1977–2012. *Technical report*, 2013.

[6] D. Firmenichy, M. Brown, and S. Süsstrunk. Multispectral interest points for RGB-NIR image registration. In *IEEE International Conference on Image Processing*, 2011.

[7] S. Gwynne and E. Ronchi. Fire loss in the United States during 2010. *Technical Report*, 2010.

[8] K. He, G. Gkioxari, P. Dollár, and R. B. Girshick. Mask R-CNN. *IEEE International Conference on Computer Vision*, 2017.

[9] T. Hrkac, Z. Kalafatic, and J. Krapac. Infrared-visual image registration based on corners and hausdorff distance. In *Image Analysis, 15th Scandinavian Conference*, 2007.

[10] M. Jaderberg, K. Simonyan, A. Zisserman, and K. Kavukcuoglu. Spatial transformer networks. *Advances in Neural Information Processing Systems*, 2016.

[11] T. Jin. Visibility through fire smoke. *Report of Fire Research Institute of Japan*, 42, 2001.

[12] M. Karter. Fire loss in the United States during 2010. *Technical Report*, 2011.

[13] M. Karter. Selected special analyses of firefighter fatalities. *Technical Report*, 2011.

[14] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations*, 2015.

[15] F. Lahoud and S. Süsstrunk. Fast and efficient zero-learning image fusion. *arXiv*, 2019.

[16] P. LeBlanc and R. Fahy. Firefighter fatalities in the united states-2011. *Technical Report*, 2012.

[17] H. Li and Y. Fan. Non-rigid image registration using fully convolutional networks with deep self-supervision. *arXiv*, 2017.

[18] H. Li, X.-J. Wu, and J. Kittler. Infrared and visible image fusion using a deep learning framework. *IEEE International Conference on Pattern Recognition*, 2018.

[19] S. Li, J. T. Kwok, and Y. Wang. Multifocus image fusion using artificial neural networks. *Pattern Recognition Letters*, 23, 2002.

[20] Y. Liu, X. Chen, J. Cheng, and H. Peng. A medical image fusion method based on convolutional neural networks. *IEEE Fusion*, 2017.

[21] Y. Liu, X. Chen, H. Peng, and Z. Wang. Multi-focus image fusion with a deep convolutional neural network. *Information Fusion*, 36, 2017.

[22] J. Molis and M. Karter. US Firefighter Injuries-2011. *Technical Report*, 2012.

[23] K. R. Prabhakar, V. S. Srikar, and R. V. Babu. A deep unsupervised approach for exposure fusion with extreme exposure image pairs. *IEEE International Conference on Computer Vision*, 2017.

[24] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *arXiv*, 2016.

[25] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation. *arXiv*, 2015.

[26] K. Simonyan and A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. *International Conference on Learning Representations*, 2015.

Supplemental Material

Data Cleaning Here we summarize our dataset cleaning procedure: Initially, we selected only IR frames containing humans. Using the pixels belonging to human objects, we threshold the IR frames by sequences of frames. After that, we segment the associated RGB frame with Mask-RCNN. This helps us to identify matching frames between the RGB and IR pairs. Furthermore, we checked again and removed all the poorly segmented RGB frames. Finally, we have 10431 4-tuples *(RGB, IR, mask RGB, mask IR frames)*. We split it into 8000 frames for the training set, 1000 frames for the validation set, and 1400 frames for the test set.

Visualization Figure 8 shows how we visualize to see if the frames are aligned to reconstruct the video after merging the warped frame (i.e., IR frame after the transformation) and RGB frame.

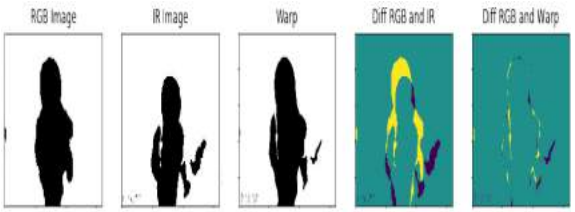


Figure 8. *Our visualization method shows the IR image, RGB image, warped image, the difference between the RGB and IR image, and the difference between the RGB and the warped image. This is useful when we work with the masked version of the images. Here we show an example of the visualization method on the masks.*

Ablation Study Before applying human semantic segmentation, we investigated the registration procedure based on contours. To extract the contours of the shapes, first, we applied a Sobel filter in each direction (x, y) and computed its magnitude. The Canny filter, adaptive thresholding, and Laplacian filter followed this. When we apply the registration to this final image, the resulting image did not move significantly. This could be due to the contrast on RGB frames (see Figure 9). We observed similar results when we tried this approach on very clean images (i.e., images of contours of circles). So we understood that the problem was coming from the loss function. When the loss L_{sim} function is very flat, the loss function will have almost the same value for the cases of a slight motion. Thus the optimizer will not update the parameters of the network because the gradient will be very close to 0. We solved this by the segmentation approach. The segmentation procedure significantly boosted the registration performance. We present a demonstration video on the project page.

Figure 9. *Extracting contours from RGB-IR pairs: It is clear that there is much more contrast on the RGB image (on the left). Such samples can be found in our dataset resulting in poor registration based on contours.*

4 Covid Tweets Prediction: Course project X-INF554

For this challenge, our mission was to accurately predict the number of retweets a tweet will get. The provided dataset contains tweet related information, such as the text and the number of hashtags, mentions and URLs contained in the tweet, and user related information, such as the number of followers and tweets that the user has published. Your solution can be based on supervised or unsupervised techniques or on a combination of both. You should aim for a minimum Mean Absolute Error (MAE), i.e., the MAE will be the loss function we use to evaluate your models.

This data challenge is hosted on Kaggle as an in-class competition.

Machine/Deep Learning INF-554

Twittes Prediction Project

Imad Eddine MAROUF
Anton EMELCHENKOV
Ali KESHAVARZI
Institut Polytechnique de Paris, France

***Abstract*—In this report, we present the details and steps used to build machine learning model to predict the number of retweets each tweet gets during the Coronavirus pandemic based on a dataset. The main objective is to have a minimum mean absolute error (MAE) in the in-class competition hosted on kaggle platform. This report is in the context of "Machine/Deep Leaning INF-554" course.**

Team Members (*ID: Hawks*):

- ***Imad Eddine MAROUF, KaggleID: @maroufimad***
- ***Anton EMELCHENKOV, KaggleID: @antonemelchenkov***
- ***Ali KESHAVARZI, KaggleID: @ali2066***

Code provided on *GitHub(ClickMe)*

INTRODUCTION

Each day millions of tweets are generated by Twitter users. The retweet prediction task can greatly help us to study the process of trend detection, social behavior prediction, and information propagation on social medias. In late 2019, the outbreak of COVID-19 was reported which has changed our lifestyles significantly. Therefore, like any other hot topic, COVID-19 has been one of the major topics for discussion on Twitter ever since. In this data challenge, our main goal was to predict the potential number of retweets for the given data. Firstly, by using useful features and neglecting useless ones and also by generating new features from the giving data, we tried to enhance our model's capacity which can be observable according to results. Secondly, we used a variety of machine and deep learning techniques to alleviate the retweet prediction task.

I. FEATURE SELECTION AND EXTRACTION

Feature selection is an important step within machine learning pipeline. It is intended to reduce the number of input variables to those that are believed to be most useful to a model in order to predict the target variable. Additionally, it reduces the complexity of the model and prevents ourfitting, and removing the redundant information. During this project we have tested many features and models. Thus, we noticed that each model can rely on specific type of features (categorical, numerical, text, ..etc). Therefore, using feature selection was an iterative process to fine-tune the model each time.

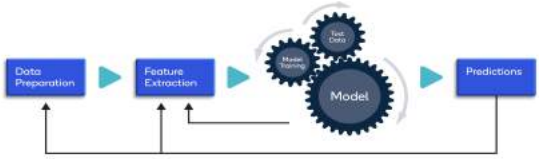


Figure 1: Machine Learning Pipeline

In this section, we will describe the process used to select features for each model and the new generated features. But before, delving into that, an exploratory data analysis (EDA) is required to build intuitions, see the patterns with in our dataset, and pre-process the missing the values.

A. Exploratory Data Analysis

Our dataset contains 665777 samples with 11 features each labeled as *'id', 'timestamp', 'retweet_count', 'user_verified', 'user_statuses_count', 'user_followers_count', 'user_friends_count', 'user_mentions', 'urls', 'hashtags', 'text'*. With an overall

view of the dataset, we have noticed that it is has many *missing values* as illustrated in 2.

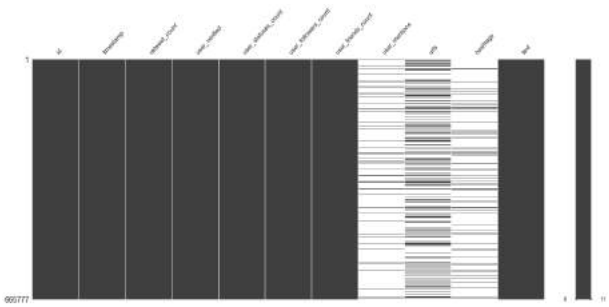


Figure 2: Illustration of missing value within dataset

Therefore, pre-processing was required as discussed in Appendix.

Additionally, it is unbalanced with respect to retweet_count (having zeros retweets) and number of verified/non-verified users, thus, for some of the models used we had to apply upsampling before training the models in order to have same distribution in training/test datasets.

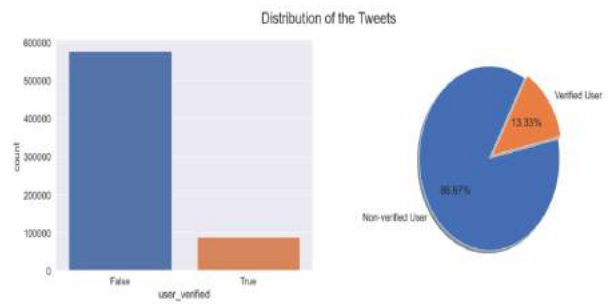


Figure 3: Distribution of Tweets (Verified users Versus Non-Verified

After, we did sanity check for the most used words in our dataset, to check the most used words in out dataset 4.

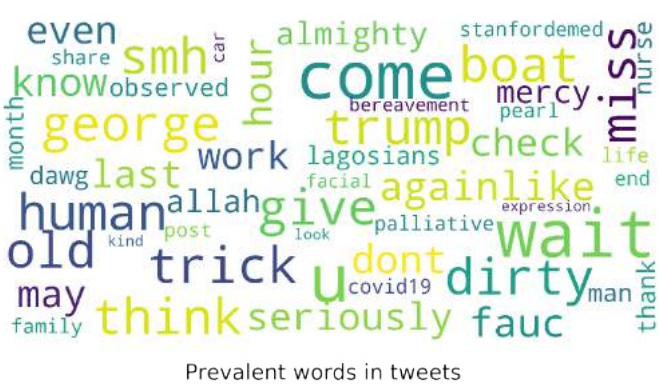


Figure 4: Word Cloud

More details about the dataset are provided in notebook ”DataVisualization”.

B. Features Selection

Early in model development, we focused mainly on using already provided features. Thus, we created a correlation matrix 5 in order to see which features are highly correlated to *retweet_count*. As shown, it has no strong correlation with features. For some models we tried reduced the number of features by taking the most correlated with target variable. The threshold usually was 0.02.

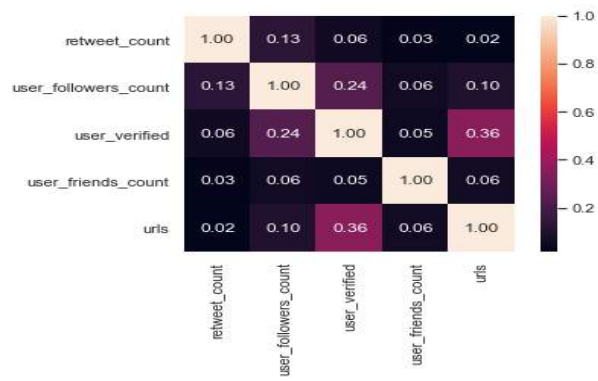


Figure 5: Correlation between Features

This step, is considered an iterative one, as illustrated in Fig 1. Where we had go through it in each model, in order to improve the accuracy. More details about that are described in the notebooks provided.

C. Features Generation

Within first tries with different models, we have concluded that the provided features do not achieve good baseline to predict the number of retweets. Thus, we had to generate new features which could be highly correlated with $retweet_count$ most of them are generated from ”Text” colums since it has insightful information about the situation where the dataset was saved. As the majority of models use only numerical data, we used different models with different input types such as embeddings and text features. More detailed explanantion for each model used ican be found in the upcoming section, and generated features method are given in Appendix.

Note that for different models we use different set of features in order to achieve higher accuracy, including only features which have correlation higher than 0.02 with the target variable and are almost uncorelated between each other.

D. Unbalanced Dataset

As described above, the dataset was unbalanced. Which is a burden to train a model, therefore, we have to apply preprocessing in order to have same distribution between the training/test datasets. Thus, could be achieved by:

- *Up-sampling*: all the samples from the minority class are kept and sampling without replacement is performed in the majority class where the number of samples sampled is same as size of the minority class.
- *Down-sampling*: means training on a disproportionately low subset of the majority class examples(i.e which are tweets getting zero-retweets)

Both methods were investigated for each tested model.

E. Evaluation

To evaluate quality of predictions Mean Absolute Error metric (MAE) is used. It can be interpreted as a total distance between model predictions and ground-truth data. Note that MAE is sensitive to the predicted number of retweets in particular to the tweets with high number of retweets (popular tweet with few retweets prediction will amplify the MAE loss).

II. Models Experimentation

A. Logistic Regression

Logistic regression is a powerful model for regression, using linear combination of features (numerical), we achieved MAE 159.69647. We were not able to train on all dataset. Thus, we used only tenth 121486 samples (equal distribution of zero/non-zeros tweets) with all standard features except text feature.

B. K-Nearest Neighbors

K-Nearest Neigbours is a classic ML algorithm which uses k most relevant (in terms of used distance) tweets. Due to the fact that the data is heterogenious, KNN could be a good method for the task. However, from our experiments we found out that it takes a lot of time to train KNN even on the part of the dataset. We were able to train it on 10000 tweets only. Best MAE score was higher than 230.

C. Random Forest Regressor

Random Forest seems to be another good method to deal with heterogenious data. As KNN it also works only with numerical features so texts features should be transformed. We tried different combination of features (using only prediction of bert for classification as a numerical feature for text to and quantitative features described above) as well as depth of the trees (from 5 to 50) and number of trees (from 50 to 500). Addtionally, we had to use fractions of the dataset, and manage the splitting Data/Test. The best MAE score was: 198.21.

D. Gradient Boosting Regressor

Boosting is an iterative technique in which the predictors are not made independently, but sequentially. Therefore, the probability of observations in subsequent models is unequal, and those with the highest error are most appear.

We used XGBoost library due to the fact that it provides parallelization and usage of GPU while creating decision trees, enabling it to be created much faster. In order to fine-tune the parameters we used *Grid-Search algorithm*, the best parameters are *max-depth: 10, n-estimators: 5000, colsample-bylevel: 0.8, max-leaves: 25*.

The features we used were all numerical, we had to delete as well the categorical features *user_verified* due to that XGBoost regressor is not good with categorical features.

E. CatBoostRegressor

Another boostig algorithm that we used is CatBoostRegressor. It is known as one of current SOTA boosting methods. First we ran CatBoost with all the features we created. We tried different methods, changing the number of rows from 50000 to 350000 and the number of iterations from 1000 to 10000. We also tried to use different depth of the model. We ended up with the foloowing parameters: CatBoostRegressor(iterations=10000, depth=7, loss_function ='MAE', eval_metric = 'MAE', random_seed=57, learning_rate=0.1,l2_leaf_reg=6). Best MAE score for it was: 154.48. From what we observe CatboostRegressor starts to overfit for a big number of iterations (5000, 10000). So we ended up using predefined 1000 iterations for the models. To enhance the quality of predictions we tried to make a prediction on 4 batches of 100000 rows each and then take mean prediction. Best MAE score was: 155.53. Another thing that we tried is to use the most relevant features from the most efficient CatBoost models we had. The best of such models achieved almost same accuracy but MAE metric slightly increased. The MAE score was: 154.59.

F. Long Short-Term Memory

After preprocessing and cleaning our data, we used a simple architecture with fewer parameters due to the computation power that LSTMs need for their training process. As it is shown in Picture [Number of Pictures], at first we used an embedding layer. Subsequently, a Drop-out layer was used to avoid overfitting. Afterward, we used an LSTM layer with 16 units. This layer is responsible for looking at the sequence of words in the tweet, along with their word embeddings, and uses both of these to determine the sentiment of a given tweet. After using another drop-out layer we used a dense layer. A linear activation function determines the output from this node which gives us the estimated number of retweets that a particular tweet will get. Best MAE: 161.

G. Neural Networks

We had as well designed many neural networks architectures to tackle the regression problem, each architecture will be detailed in the following sub-sections.

Before feading the dataset into the networks we had to apply normalization and standarlization suing *StandardScaler()*

Figure 6: LSTM Architecture

function from scikit-library, batch-size : 64, learning-rate: 0.01, with Adam optimizer.

1) Regression Architecture: We investigated diffenrent architectures of ordinary neural network of varying between 3, and 5 hidden layers while only feeding them with numerical features *'user_verified', 'user_followers_count', 'user_friends_count', 'urls'*. The best [64, 64, 32, output] with *Relu* as an activation function. Best MAE score was 191 for 50 epochs.

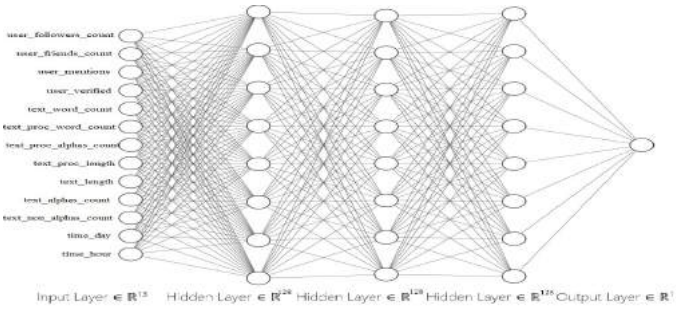


Figure 7: Neural Nets Architecture for regression

2) Regression Architecture II: Our Best score was given by a neural network fed by 13 numerical features extracted from text, due to that we realized that the numerical features we used for architecture I. This network consisting 3 layers [128, 128, 128, 1], it was trained for 20 epochs, and changing the loss function from L2 loss t L1 norm, it helped the the model to prevent getting stuck at local mimina like the architecute I.

More details about the features are descriped in "Car-Regressor_XGBoost_NeuralNets" notebook.

3) Autoencoder Architecture: Insipired from the autoencoder architecture, to get a good representation of the dataset. We used 5 layers [128, 64, 32, 64 128, output] with latent-layer contains 32 neurons for same numerical featuers. Best MAE score was: 231 for 50 epochs.

4) NNs with Tfidf vectorizer: For this model, we did use only text feature, after processing the text and vectorizing to 100 features. We feed into NN consisting of [input, 128, 64, 1]. Best MAE score was: 251.

H. DistilledBERT for Regression

Transformers are well for their accuracy, and effenciency for NLP tasks. Thus, we worked on taking this privilege and applying transformers for regression task. Our method consisted of using DistilledBERT[6] (a light-weight version for official BERT), and changing its last layer into 1 neuro, and changing the loss function from soft-max into mean-square-error. After preprocessing the text, and tokenizing it. Best MAE score was: 225.

I. Bert

We also decided to try to use BERT to classify texts. We decided to solve classification task with BERT and not regression because the data is highly unballanced. In our first experiment with Bert we tried to solve binary classification problem. The model was supposed to predict if a tweet was retweeted or not, i.e. its retweet-count ≥ 0. We cleaned raw tweets as detailed in IV. We also balanced the classes, as the 0-class was much bigger that the non-0 class. After that we finetuned pretrained BertForSequenceClassification model. We used a single epoch and only part of the training set, because otherwise overfitting occurred. We got nice validation flat accuracy score of 0.92. Afterwards we used the prediction of BERT as a feature.

We also tried milticlass classification on intervals: [0], [1, 10], [11, 50], [51, 100], [101, 300], [301, 1000]. The idea was to assign median of all the targets on the training set in the given interval (as median minimizes MAE). We also wanted to use class probabilities predicted by BERT as training features. This experiment didn't go well: flat accuracy was low. We then replaced BERT loss with BCE-WithLogitsLoss, and the results got better, but still overall accuracy wasn't good. We also used soft labels: instead of setting ground truth to be vector of zeros with a single 1 in the place of the right class, we used 0.8 in place of the right class and 0.1 on the left and on the right of it. Features from multiclass BERT we didn't use afterwards.

We also wanted to try other thresholds for binary classification (instead of ≥ 1), for example we tried ≥ 1000 and ≥ 100 with balanced classes, but accuracy was bad, so we decided not to use these features.

J. Support Vector Machines

Nowadays, SVMs are popular because of their potentials for generalization. Two different SVM examined for this dataset, SVR with RBF kernel and LinearSVR. According to Figure 8, although the best performance among different type of SVMs belongs to SVR with MAE equal to 161, the overall performance of these models are not close to other achievements.

Regarding SVR, the fit time complexity is more than quadratic with the number of samples which makes it hard to scale to datasets with more than a couple of 10000 samples. LinearSVR is Similar to SVR with parameter kernel='linear', but implemented in terms of liblinear rather than libsvm, so it has more flexibility in the choice of penalties and loss functions and should scale better to large numbers of samples[10].

Additionally, we tried different types of normalization (StandardScalar, MinMaxScalar, etc) as a part of the preprocessing phase for these models. Because of the nature of our data, the MinMaxScalar gave us slightly better results;but at the end, we were not able to decrease the models' Mean Absolute Error significantly.

K. *Ensemble Methods*

1) Logistic Regression with LSTMs: Although we achieved a good result with LSTMs, it is not an effective model to predict high retweet values. Therefore, we got the result of both Logistic Regression and LSTM and combine them with a naive method (The rounded mean of predicted values). In this way, we achieved a slightly better MAE of 159.

2) Hybrid- Architecture: BERT with XGBoost: This architecture, is an ensemble of two models. First, consisting of BERT for binary classification (whether the the tweet will get zero/non-zero retweet) using only the text feature. We got 89% accuracy regarding binary classification task.

Second, will apply XGBoost regression for regression when the feed sample was classified as non-zero retweet using numerical features *user_statuses_count, user_followers_count, user_friends_count*. It was not very effective, we got MAE score: 191.

III. Results and Discussions

To make a prediction we tried both classical ML algorithms, recent SOTA methods as well as different NN architectures. Between the classical methods we would like to emphasize on *LogisticRegression* as it slightly enhanced the score obtained from all-zeros prediction. However other classical methods were not as efficient and could not tackle the unbalanceness of the large dataset. So we moved to more robust methods. We tried models of different nature: those which are working with text and those working with numerical features. For the text we mainly focused on Bert models as they are known as one of the mos robust methods to work with text. However, from what we observed Bert was not able predict the number of retweets using the text only. So we decided to move to numerical models, using result of the BertClassifier as one of the features. For the numerical features we used CatBooostRegressor (CBR) which is a robust boosting method that does not need much computation power. Best MAE score achieved with CBR was 154.48. Finally we used various NN architectures. We were able to reproduce CBR's result with fully-connected NN. MAE score for the NN was 154.44.

Furthermore, the crucial part, we found is the feature extraction when it comes to NLP tasks. The recent best scores we got are due to more feature generation we used from the text due to the fact that numerical features are not very sufficient to predict the re-tweets number.

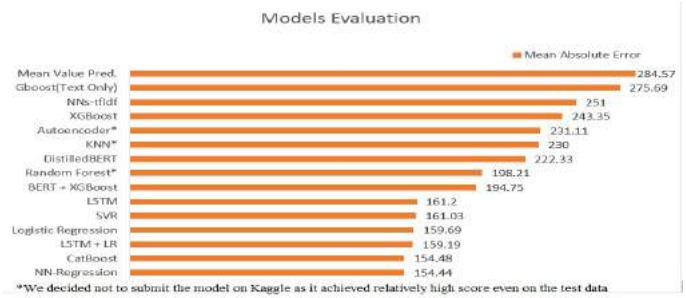


Figure 8: Model's score

IV. Insights Taken

Although, we did not manage to rock the leaderboard, throughout, this project we explored many models, pros and cons of each. As well, the challenges with respect to NLP. It was our first time dealing with NLP tasks. Unpredictability of datasamples, which we encountred in this dataset outliers, non-important tweets getting retweeted many times - all this needs to be taken into account, as well the importance of feature selection and using common-knowledge about the dataset topic could lead to much better results.

Conclusion

Machine learning is a rapidly evolving field. Every year, there arise dozens of improvements. Despite the test of time, classical methods no longer achieve the most efficient results tackling modern problems. This competition showed that to obtain an effective result it is better to use modern state-of-the-art methods which are best suited for specific tasks. For this competition we found out that CatBoostRegressor and FCNN best served our needs.

References

Project Handout :
[1] Project Description: COVID19 Retweet Prediction

Tutorials:
[2] XGBoost Documentation
[3] XGBoost: An end-to-end Guide
[4] Regression using NNs: Medium Article
[5] Cat Boost Regressor Library using Grid search
[6] DistilledBERT by Hugging-Face
[7] General Approach for Text Processing
[8] Visual Notebook for BERT
[8] Logistic Regression: Detailed Overview
[9] Neural Networks with Tfidf vectors using Keras
[10] Sklearn.svm.LinearSVC Documentation

APPENDIX

A. Further EDA

In order to have coherence between our numerical values, we have to fill the NaN values in *'user_mentions', 'urls', 'hashtags'* by analyzing the corresing tweet text using regular expressions.

As illustrated below 9, 10. There exist high variance between the values of the features. We created a new features by applying log to original features. In other models (like neural nets) we had to apply normalization, and standaralization before training.

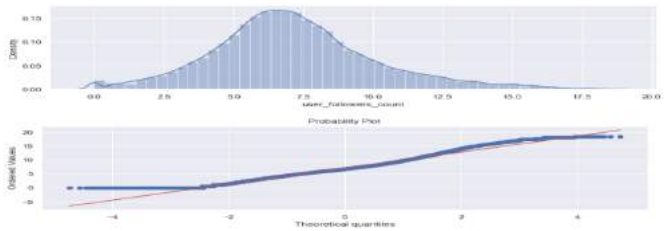


Figure 9: User-Followers Probability Plot

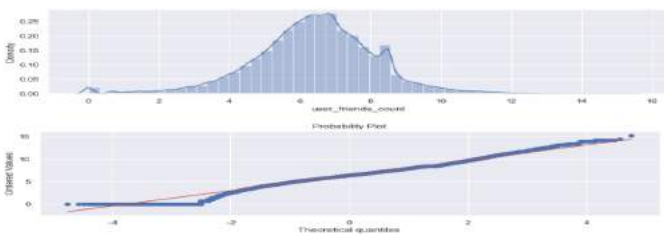


Figure 10: Users Friends Probability Plot

Regarding the other features like *Hashtags, URLs, user_mensions* columns from *text* column.

B. Text Cleaning

Before feeding the data into our models. Data cleaning process was required especially when we would to make use of the *text* feature. The steps we applied to text are:

- Remove URLs, Emojis, HTML tags, and punctuation.
- Tokenized the tweet base texts: is about splitting the strings into words.
- Lower cased clean text
- Remove stop words
- Applied part of speech tags.
- Applying word lemmatize
- Converted tokenized text to string again.

Furthermore, we analyzed the most used retweeted words, mostly about Covid-19, since it was trendy topic at the time of capturing data. Additionally, we noticed the difficulty of the dataset where sometimes non-useful/non-important tweets get re-tweeted many times.

For more insights about the dataset, please refer to notebook provided DataVisualization.

C. Feature Generation

In order to generate more relevant features, with respect text input. We generated : categorical (binary mostly, related to coronavirus) and numerical features (frequency, mean, count) using 'Text' column. One group of features includes binary features for the most commonly used words which show whether the word is in the text or not.

We also tried to used most common bigrams and trigrams. Besides, we counted number of emojis, different punctuation symbols as well of the length of the text. We did it for both cleaned and uncleaned text. Moreover, we extracted number of urls, hashtags and mentions. As regarding numerical features we mainly considered timestamp to extract more features. We used categorical features to divide the timestamp to groups by part of the day (as in some part of the day tweets get higher retweets). Additionally we extracted time, date and hour (minutes and seconds are not relevant). In some models we also tried to use result of Bert for binary classification (Further details in the Models section).

- Categorical (Binary) features: Does the tweet has most popular words (e.g. 'covid'), has-emojis, ... etc.
- Numerical features: text-length, text-processed-length(after clearning), ponctuation-count, urls-count, emojis-count, ...etc

All those features are based on n-grams we generated from the tweets, we think they are correlated to retweet prediction as illustrated in 11.

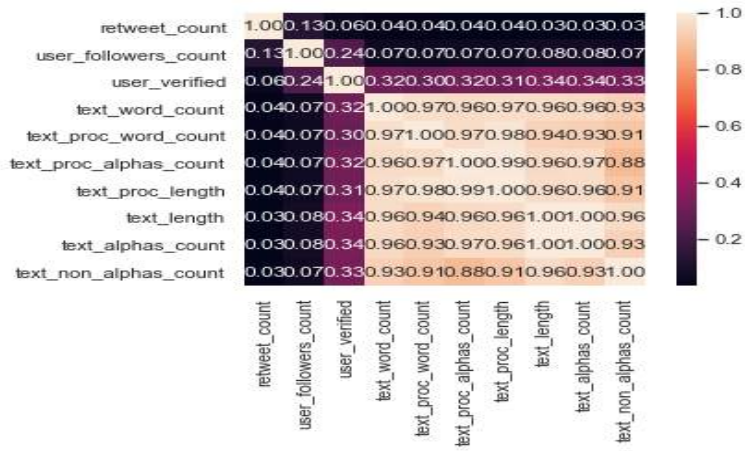


Figure 11: More features

D. Code

We attached our report, with all the experiments conducted for each model. Each seperately developed in jupyter-notebook, before re-running the cells please, make sure to install the required packages from YML file, and the dataset. Note that some as the models use result of Bert as a feature. The updeated dataset is available *here (ClickMe)*

5 Multi Echelon Supply Chain Optimization with Reinforcement Learning: Project Course X-INF581

Multi-Echelon Supply Chain Optimization with Reinforcement Learning

Imad Eddine MAROUF
Anton EMELCHENKOV
Valerii MARKIN
Zouhair KHATOURI

***Abstract*—This work studies the application of the various Reinforcement Learning methods for multi-echelon supply chain management problem. The environment is represented by a simple multiple levels supply chain with stochastic demand, predefined costs, profits, losses and products manufacturing at each level.**

The RL agents are trained to maximize the profit from the supply chain. The performance of models is tested with different demand distributions, uncertain change in parameters. The models are also tested for stability to changing the inner parameters of the environment.

Proximal Policy Optimization, and A3C agents achieve the best performance, in different scenarios. Overall, our study emphasis on the strong potential of RL algorithms to learn dynamic policies for multi-echelon supply-chain management with strong rewards, and they are non-sensitive to uncertainties in demand distribution compared to static policy by Derivative Free Optimization and Linear Programming.

I. Introduction

Multi-echelon supply chain optimization problem is a type of operational research problem which aims to maximize the profit of the multi-stage supply chain (e.g. a chain of factories which produce material needed on the next stage). From the practical point of view the multi-echelon approach is said to be the best solution for maintaining cost-effective inventory management as it optimizes inventory across all the of supply chain levels. Operational research problems have not been studied much from the reinforcement learning perspective.

In general, problem of multi-echelon supply chain optimization can be solved with the linear programming methods. However such methods involve estimation of the supply-chain policy each time its parameters updated. Despite obtained solutions could be unstable with the change of time which might be an issue in real-world applications. With the reinforcement learning approach we may expect more stable solutions without significant loss of profit in comparison to exact solutions. To test these hypotheses we compare various reinforcement learning methods, such as PPO, ARS, DFO, Reinforce and A3C, to the problem. We use OR-Gym environment [18] to test the methods.

Our contributions consists mainly:

1) Implement well-known algorithms in Reinforcement learning to multi-echelon supply chain environment OR-Gym [18].
2) Analyse the behavior of the agents in different contexts (demand distribution, prices of products, and losses).
3) Analyse the performance of differnet RL algorithms with respect to complexity of the supply chain levels.
4) Robustness of RL algorithms to uncertainity of parameters changes in environments (Demand distribution, products price).

The code is available on the GitHub [1].

II. Background and Related Work

Although it is an area of interest to industries, there are very few articles dealing with the application of RL to multi-echelon systems optimization. Among these few articles stand out [2] by Jiang et al being the most cited. More specifically, this article deals with two level echelon system optimization problem, in which the retailer has to optimize (r, Q) or (T, S) policy parameters. The model's only decision is made by the retailers, so it's not a true multi-echelon problem. In order to to lower the complexity degree of their model, Jiang et al turned the states and the action spaces into finite one using discretization and truncation procedures. This rather turned their model into too simplistic one.

RL is used by Gijsbrechts et al. (2019) to solve a one-warehouse, multi-retailer (OWMR) problem (along with two complicated single-echelon problems). They address the problem using the asynchronous advantage actor-critic (A3C) algorithm (Mnih et al. 2016), which produces positive results as opposed to traditional approaches. Their approach takes into account a different supply chain structure (WMR vs. serial) and employs a distinct RL approach than ours.

The inventory management problem (IMP) that is presented in this work is built upon the problem structure presented in Glasserman and Tayur [4], which presents a single-product, multi-period, serial capacitated supply chain with production and inventory holding locations at each echelon. In their work, Glasserman and Tayur [4] use infinitesimal perturbation analysis (IPA) in order to determine optimal base stock levels in an order-up-to policy by optimizing over a sample path of the system. Although the base stock strategy is accepted as not being ideal for multi-stage capacitated systems, its simplicity makes it appealing for realistic implementations.

[1] Code found here: https://github.com/IemProg/Multi-Echelon-Supply-Chain-with-RL

Other methods to the IMP that have been studied in the literature are those of Bertsimas and Thiele [3], Chu et al. [4], Mortazavi et al. [5], and Simchi-Levi and Zhao [6]. For both capacitated and uncapacitated systems, a general optimization approach based on robust optimization techniques is proposed by [3]. Capacity limits on the orders and inventory are part of their work, but not on production. Their model is tractable in 12 terms and can be solved as a linear program (LP) or a mixed-integer program (MIP) depending on whether fixed costs are included. Another approach [4] consists in refining a multi-echelon supply chain with a (r, Q) reorder framework using agent-based modeling and a cutting plane algorithm in a virtual setting. When approving the enhancements, Monte Carlo models are used to assess goals, followed by hypothesis checking to cope with the consequences of noise. In [5], a four-echelon supply chain model with a retailer, distributor and manufacturer is developed. With non-stationary demand drawn from a Poisson distribution, they use Q-learning to learn a dynamic policy to re-order stock over a 12-week cycle. There are also several papers in the area of inventory optimization available. Finally in [6], they compare and contrast these approaches: queuing-inventory, lead-time demand, and flow-unit method, addressing their strengths and disadvantages, variations and relations, and explaining how to use them routinely to characterize and analyze various supply networks of various supply systems, inventory strategies, and demand processes.

To our knowledge, there is no paper, investigated RL methods on multi-echelon supply chain on such environment like OR-Gym. Our focus is not on finding optimal parameters for static inventory policies, but rather to determine and compare different dynamic policy approaches to the IMP, except for paper [], where they have designed DQN algorithm as an agent for Beer production management system.

III. The Environment

The environment is a single supply chain consisted of M stages. Each stage consist of the production facility (factory) and the inventory.

We can understand it on the example of screws. It all begin from the iron ore (raw material), which is produced at the ore mine (Stage 4). Next, the ore is melted into steel at the steel plant (stage 3). Then the steel transforms into steel rods and then into screws on stages 2 and 1. And finally, screws are delivered to a retailer which is represented by stage 0.

The customer comes to the retailer and tries to buy some number of screws. If the retailer doesn't have enough, it loses profit. Facilities at each stage k has a production power p_k and the inventory capacity I_k. They can order materials for production from the previous stage. At each stage there is also a lead time which is required to make a delivery to a next factory. One unit of inventory produces one unit of product at each factory. There is also a cost of storing the inventory. The example of the parameters of the environment is given in the Table 4.

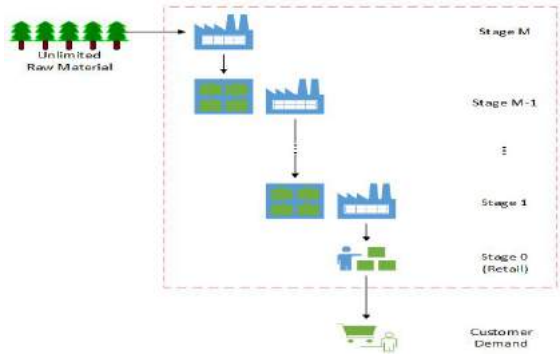


Fig. 1. Mutli-Echelon Supply Chain Illustration [18]

Parameter	Symbol	Stage 0	Stage 1	Stage 2	Stage 3
Initial Inventory	I_0	100	100	200	-
Unit Sales Price	p	\$2.00	\$1.50	\$1.00	\$0.75
Unit Replenishment Cost	r	\$1.50	\$1.00	\$0.75	\$0.50
Unit Backlog Cost	k	\$0.10	\$0.075	\$0.05	\$0.025
Unit Holding Cost	h	\$0.15	\$0.10	\$0.05	-
Production Capacity	c	-	100	90	80
Lead Times	L	3	5	10	-

Fig. 2. Default parameter values for the environment.

The following systems of equations describes the processes in the environment

$$
\begin{aligned}
&I_{t+1}^m = I_t^m + R_{t-L_m}^m - S_t^m \quad \forall m \in \mathcal{M}, t \in \mathcal{T} \\
&T_{t+1}^m = T_t^m - R_{t-L_m}^m + R_t^m \\
&R_t^m = \min\left(c^{m+1}, I_t^{m+1}, \hat{R}_t^m\right) \quad \forall m \in \mathcal{M}\backslash\{|\mathcal{M}|\}, t \in \mathcal{T} \\
&S_t^m = \begin{cases} R_t^{m-1} & \text{if } m > 0 \\ \min\left(I_t^0 + R_{t-L_m}^0, D_t + B_{t-1}^0\right) & \text{if } m = 0 \end{cases} \\
&U_t^m = \hat{R}_t^{m-1} - S_t^m \\
&P_t^m = (\alpha)^n \cdot \left(p^m S_t^m - r^m R_t^m - k^m U_t^m - h^m I_{t+1}^m\right)
\end{aligned}
$$

Here we define as:

- I_t^m - on-hand material balance at the stage m at the time t
- R_t^m - materials arrived to stage m at time t
- S_t^m - sales at stage m at time t
- T_t^m - the materials in order at stage m at time t
- c^m - production capacity on factory m
- $\hat{R}_m^t$ - requested amount of material at stage m time t (**This is the action of the agent**)
- D_t - demand at time t
- U_t^m is an unfulfilled demand
- P_m^t - discounted profit. r, k, h are the costs of reorder, unfulfilled demand and storage.

The environment can be modeled as Markov Decision Process MDP, where the agent have to decide how many products to ask for from higher levels at each time step. Next, we can define the state-action space in more formal way:

- **State space**: it is a vector, containing the amount of goods at each stage and the history of $\max(L)$ previous actions. Here $\max(L)$ defines the maximum lead time in

the supply chain. The state vector is shown on the Figure 3

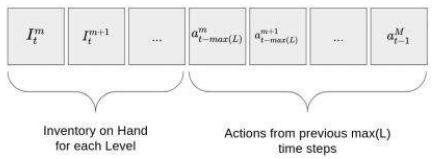


Fig. 3. Illustration of the state space vector

- **Action space**: Action space is a vector of integers of dimension $M-1$. Each element in this vector represent the amount of products to order from each supplier.
- **Reward**: The reward is a total profit made on this timestamp.
- Stochasticity in the environment is introduced with a distribution D. This distribution generates the demand for the final product at each step.

In this paper, we are considering two different environments *"InvManagement-v1"* - without backlog cost -, where *"InvManagement-v0"* meaning that a loss (per unit) is applied for unfulfilled orders (demand or replenishment orders).

IV. The Agent

In this paper, we evaluate the performance of various algorithms such as: *Proximal Policy Optimization (PPO)*, *Augmented Random Search (ARS)*, *Reinforce*, *Derivative Free Optimization (DFO)* and *Asynchronous Advantage Actor-Critic (A3C)* on different demand distribution for our environments. We consider DFO (Optimization Algorithm) as baseline to our evaluation. DFO agent use static base policies, whereas other agents use dynamic reorder policies trying to find the optimal values to maximize the profit.

A. Derivative Free Optimization (DFO)

DFO is an optimization which does not rely on the states. The objective is to find a balance between the inputs and outputs for our environments, in other words, it tries to balance the cost for each level, and the benefit gained. The implementation is coded in a way that if a factory is below certain threshold products, it have to re-order the difference to reach this threshold. For example, if the policy at stage 0 is to re-order up to 20 products, and the currently, we have 16 widgets, then the policy states we're going to re-order 4. In the RL case, it would take into account the current pipeline and all of the other information that we provide into the state. Our implementation it relies an optimizer from Scipy library [17] and article [16].

DFO is based on Powerll's method, because it does not rely on gradients, therefore, we can apply it to our environment which is not differentiable - discrete action space -. Thus, Powell's method wil round the base stock levels in each level to the nearest integer or performing local search to find the nearest integer.

B. Proximal Policy Optimization (PPO)

PPO is a novel Policy Gradient algorithm proposed in [14]. It is driven by the idea that each update of the parameters should not change a policy distribution $\pi_\theta(a|s)$ too much. This can be achieved by using the surrogate objective function and optimize it to find the new parameters vector

$$\theta_{new} = \arg\min_\theta \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t \right) \right]$$

where

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

and $\hat{A}_t$ is the estimation of advantage function. In this method $\hat{A}_t$ is usually obtained in the same way as in Actor-Critic methods, using another neural network to estimate $V(s)$.
The original paper shows, that this methods has a very stable learning process due to precise updates of the parameters.

C. Augmented Random Search (ARS)

Augmented Random Search is a very fast policy iteration algorithm, it is a variant version of the Basic Random Search (BRS) algorithm [11]. Where the agent picks a paramaterized policy π_θ, then it perturbs the parameters θ by applying a constant noise $v\sigma$, where v is constant < 1 and σ a random number generate from a normal distribution. Then, we run a simulation test, if the reward improves we keep the given weights otherwise, discard them [12]. Unlike other recent RL algorithms, ARS does not rely on rely neural networks, but it uses perceptrons which makes converge rapidly and get good reward scores.

Algorithm 2 Augmented Random Search (ARS): four versions **V1**, **V1-t**, **V2** and **V2-t**

1: **Hyperparameters:** step-size α, number of directions sampled per iteration N, standard deviation of the exploration noise ν, number of top-performing directions to use b ($b < N$ is allowed only for **V1-t** and **V2-t**)
2: **Initialize:** $M_0 = \mathbf{0} \in \mathbb{R}^{p \times n}$, $\mu_0 = \mathbf{0} \in \mathbb{R}^n$, and $\Sigma_0 = \mathbf{I}_n \in \mathbb{R}^{n \times n}$, $j = 0$.
3: while ending condition not satisfied **do**
4: Sample $\delta_1, \delta_2, \ldots, \delta_N$ in $\mathbb{R}^{p \times n}$ with i.i.d. standard normal entries.
5: Collect $2N$ rollouts of horizon H and their corresponding rewards using the $2N$ policies

$$\mathbf{V1:} \begin{cases} \pi_{j,k,+}(x) = (M_j + \nu\delta_k)x \\ \pi_{j,k,-}(x) = (M_j - \nu\delta_k)x \end{cases}$$

$$\mathbf{V2:} \begin{cases} \pi_{j,k,+}(x) = (M_j + \nu\delta_k)\,\text{diag}\,(\Sigma_j)^{-1/2}(x - \mu_j) \\ \pi_{j,k,-}(x) = (M_j - \nu\delta_k)\,\text{diag}(\Sigma_j)^{-1/2}(x - \mu_j) \end{cases}$$

for $k \in \{1, 2, \ldots, N\}$.
6: Sort the directions δ_k by $\max\{r(\pi_{j,k,+}), r(\pi_{j,k,-})\}$, denote by $\delta_{(k)}$ the k-th largest direction, and by $\pi_{j,(k),+}$ and $\pi_{j,(k),-}$ the corresponding policies.
7: Make the update step:

$$M_{j+1} = M_j + \frac{\alpha}{b\sigma_R} \sum_{k=1}^{b} \left[r(\pi_{j,(k),+}) - r(\pi_{j,(k),-}) \right] \delta_{(k)},$$

where σ_R is the standard deviation of the $2b$ rewards used in the update step.
8: **V2 :** Set μ_{j+1}, Σ_{j+1} to be the mean and covariance of the $2NH(j+1)$ states encountered from the start of training.[2]
9: $j \leftarrow j + 1$
10: **end while**

Fig. 4. Augmented Random Search (ARS) [11]

The main improvements for ARS with respect to BRS are: Firstly, we the division by the standard deviation of the collected rewards, in order to prevent brutal variation update for $r(\theta + v\sigma) - r(\theta - v\sigma)$. Secondly, nomalization of the states, to impose that the policies given equal weight on the different states.

D. REINFORCE

Reinforce is a policy gradient method which aims to model and directly optimize policy $\pi_\theta(a|s)$ by maximizing the reward function $J(\theta)$ w.r.t parameter θ. At each step the algorithm estimates the gradient reward function using Monte Carlo approach: it generates trajectory τ and estimates the gradient $\nabla_\theta J(\theta)$ as $\sum_{t=0}^{N+1} \nabla_\theta \log \pi_\theta(a_t|s_t) G_t$, where G_t is the expected return (gain) $G_t = \sum_{k=t+1}^{N+1} \gamma^{k-t-1} R_k$. We estimate a policy with a fully-connected neural network having one hidden layer which outputs vector of probabilities to take each of the actions. REINFORCE Pseudo-code is given in Fig. 5

```
function REINFORCE
    Initialise θ arbitrarily
    for each episode {s_1, a_1, r_2, ..., s_{T-1}, a_{T-1}, r_T} ~ π_θ do
        for t = 1 to T − 1 do
            θ ← θ + α∇_θ log π_θ(s_t, a_t) v_t
        end for
    end for
    return θ
end function
```

Fig. 5. Reinforce algorithm

E. Asynchronous Advantage Actor-Critic (A3C)

A3C Algorithms belongs to Action-Critics policies, A3C aim to combine value-based method- approximate the optimal value function- and policy-based method - approximate the optimal policy for the environment with calculating Q-value function [8], [7]. A3C is composed of two model : one for computing an action based on a state and another one to produce the Q values of the action. The actor takes as input the state and outputs the best action. It essentially controls how the agent behaves by learning the optimal policy (policy-based). The critic, on the other hand, evaluates the action by computing the value function (value based). Those two models participate in a game where they both get better in their own role as the time passes. The result is that the overall architecture will learn more efficiently than the two methods separately.

The actor can be a function approximator like a neural network and its task is to produce the best action for a given state. The critic is another function approximator, which receives as input the environment and the action by the actor, concatenates them and output the action value (Q-value) for the given pair.

There are many variants of action-critic policies. The main advantage of A3C is fast training by paralysing the learning process, where multiple independent networks each interact with the a separate environment, while periodically update a global network in asynchronous manner, to prevent overwriting the weights between the different networks. In this way, we can speed the training process by magnitudes.

Action/Critic policies are represented by a two fully connected layers *[256, 256]* with *tanh* as an activation function, and *learning rate: 0.0001*.

V. Results and Discussion

A. *Evaluating Performance with different Demand Distribution*

To evaluate the performance of our agents, we will set up different demand distribution for our environment Poisson, Binomial, random integers. As first, experiences we have fixed the number of levels to 3 levels. Each simulations runs for 1000 episodes, with same parameters of the environment, by fixing the seed of the random generator to prevent different environment context to each agent.

TABLE I
Parameters for demand distributions

Distribution Type	Poisson	Binome	Random Integers
Parameters	'mu': 20	'n': 50, 'p': 0.65	'low' : 0, 'high': 300

where *mu, n* are mean values, *p* probability between 0 and 1 of getting the mean value.

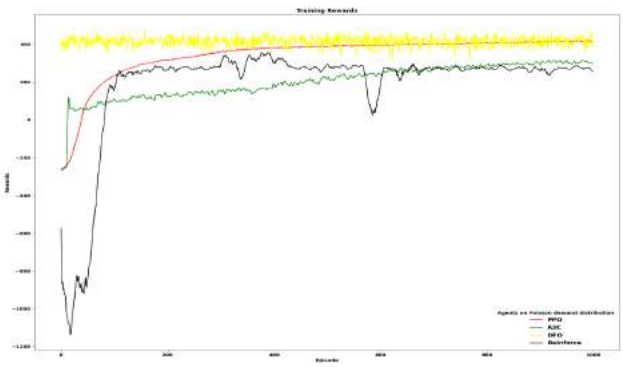


Fig. 6. Performance of agents on Poisson demand distribution

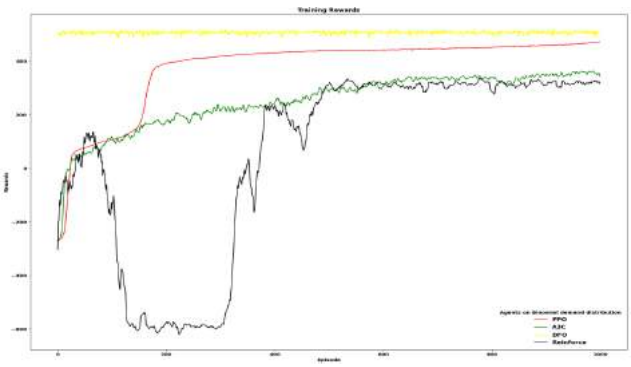


Fig. 7. Performance of agents on Binomial demand distribution

RL perform well irrespective of the demand distribution, they vary from stability, especially for Reinforce is noisy and instable. DFO outperforms all other agents since it uses static base stock policy, it tries to approximate the real values of the demand distribution. DFO is an optimization algorithm, it achieves directly the approximate optimal values by maximizing the reward function, without trial-and-error like RL agents. But, we claim DFO will not perform well

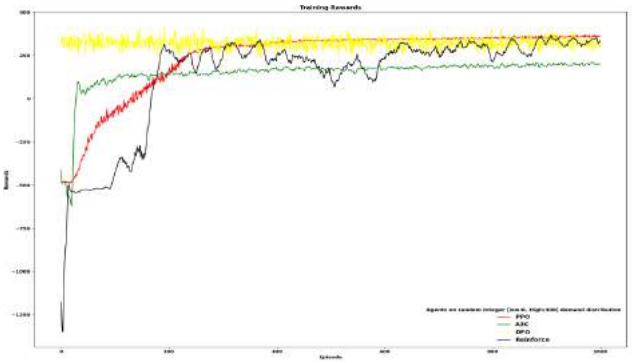


Fig. 8. Performance of agents on random between 0 and 300 requested demand

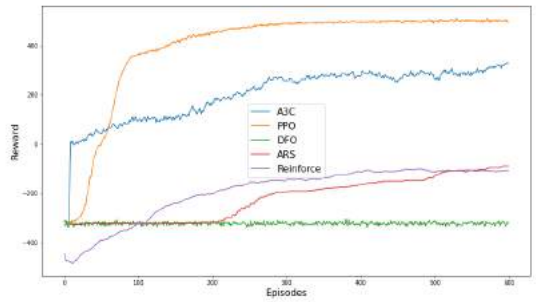


Fig. 9. Performance of agents on Scenario 1 [4 layers]

if we increase the complexity of the environment since the optimization of various parameters will be a burden. On the other hand, PPO has converged very rapidly with good reward scores as DFO.

In order to evaluate the stability of the agents, we did run-the experiments 10 times in order to calculate the average reward after convergence, and the variance with respect to the average reward.

TABLE II
RESULTS FROM 10 ITERATIONS FOR EACH AGENT'S TYPE

Agent	PPO	REINFORCE	ARS	A3C	DFO
Average Reward	380	285	429	340	411
Standard Deviation	7%	18%	2%	10%	5%

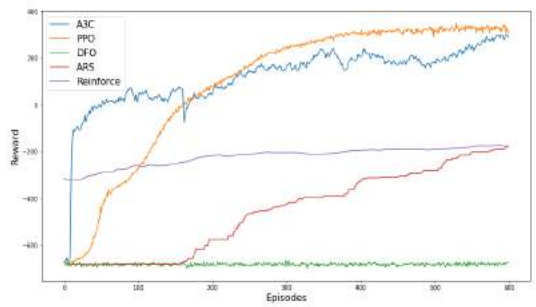


Fig. 10. Performance of agents on Scenario 2 [7 layers]

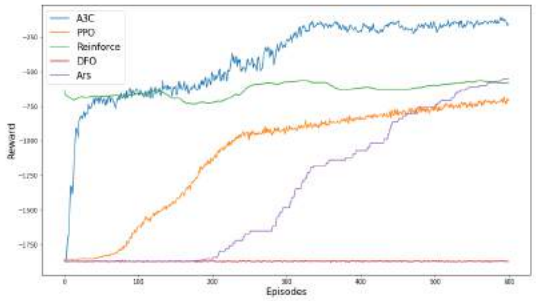


Fig. 11. Performance of agents on Scenario 3 [9 layers]

B. *Effect of Increasing Supply Chain Levels*

In order to evaluate the performance of each agent, for different levels of the supply chain hierarchy. We set three different environment [4 layers, 7 layers, 9 layers] with different values for unit holding cost, production capacities, and waiting time between layers (Parameters given in Appendix).

This experiments shows that the performance of the model depends not only on the size of supply chain, but also on the inner parameters. We also noticed that the convergence rate for all models slows down as the supply chain becomes bigger. This is very natural result, since more complex environments require more iterations to train. PPO is performing well on all the 3 different experiments, although A3C outperforms it in the last experiment. We believe it is due to the lack of hyper-parameters tuning, since it has more of hyper-parameters than A3C and we did not fine-tune them very well like *clipping parameter, beta, and entropy coefficient*. DFO method is turned out to be not very efficient where the reward does not exceed 50 due to its high sensitivity to the inner parameters, and high dimensionality of the reward function makes the optimization process very hard in order to find the optimal values.

C. *Robustness for Abrupt Changes in Demand Distribution*

Manufactures are subject to abrupt changes in demand distribution during the year, it differs from one period, or season to another. Thus, an optimal agent should be robust to sudden changes in demands. We did setup an experiment to measure how each agent defined above is robust to changes in demand distribution and how fast can be to invariant to changes.

We did train each agent for default *Poisson distribution* with $\mu : 20$, then we change the parameters of the environment to be $\mu : 30$ for the agent to be trained for 200 more episodes. We compare how fast he converges and what's the final reward. On the other hand, we trained same agents with the latter

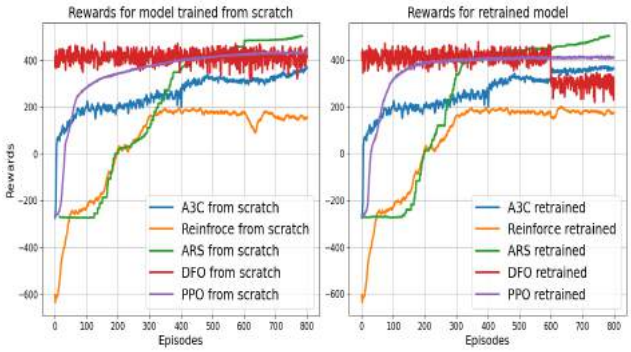


Fig. 12. Comparison of agents subject to abrupt changes, with agents trained from scratch on different distribution

parameters μ : 30 for 800 episodes. In this way, we would like to investigate how trained agents on specific environment are able to transfer their policy to different environment, which is highly required for real-life supply chain, since we would like to avoid training each agents from scratch each time, and more robust agents for noise and randomness in the environment parameters.

As we observe in majority of cases the retrained model can adapt well to the new conditions and even slightly outperforms the initial model in some cases. This property favorably distinguishes the reinforcement learning model from the mathematical programming solvers which are also suitable for the problem but should be ran every time the parameters of the model changes. Furthermore, we observe that majority of the models give stable results in the new conditions and do not suffer at all from the changed demand (especially PPO and A3C). Thus, RL approach may be a good method to get efficient estimates of parameters for the supply chain problem in real life.

D. *Environment with Backlog Cost*

We setup an experiment for an environment with *Poisson distribution*, with μ : 20 for 4 layers of supply chain. In this case, cost of not fulfilling the requested products is applied to the agent.

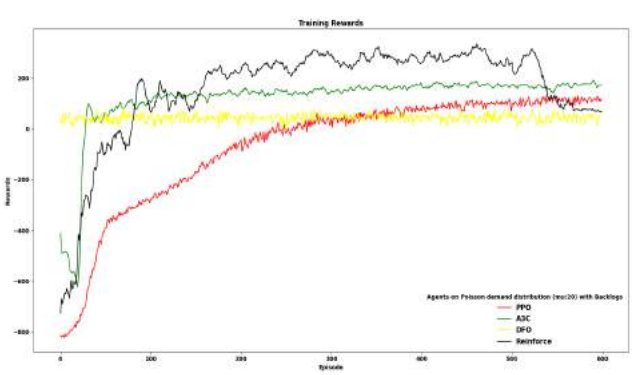


Fig. 13. Performance of Agents on Environment with backlog cost

As illustrated in Fig [12], the RL methods outperformed DFO when taking backlog cost into consideration when orders are unfulfilled, because RL agents learnt to to prevent continuing to request additional inventory towards the end of the 30 days cycle, which lowers the costs for the environment. On the other hand, DFO follows a static policy by requesting until the end of the 30 days period, in order to reach the optimal base stock levels, which increases the costs.

This is backed by checking the actions taken by the agents during the last time steps (last days within 30 days cycle) after converge. DFO still has big value for every level, but for RL agents is really low for every level request which reduces the cost and does not hurt the reward much. The values of actions taken by agents are pasted in Appendix.

We are puzzled by how RL agents learn to follow this policy by decreasing the production in the last days to lower the costs. Nonetheless, we claim that this behavior is subject to the supply chain cycle, in our environment the cycles ends after 30 days, but in real life it is a continuous operations, so a further investigation should be done for different periodic cycles (3 months, 1 years).

E. *Curse of Dimensionality*

The action space is a continuous action range for each action dimension for each level in out supply chain network, While increasing the complexity of the problem, this is expected to enable solving larger problems. SARSA, Q-Learning and DQN are not suited for problems with continuous action spaces. We opted to use the previous algorithms, we have also created the implementation for SARSA, Q-Learning but we did not investigate it since it requires to lower the action-space to unrealistic experiments. Furthermore, it prevents scaling the chain to multi-levels.

VI. Conclusion and Future Work

Our work extends the analysis of the OR-Gym open-source environment, we have shown that RL algorithms are very competitive and represent a solution to multi-echelon supply chain optimization. RL algorithms are dynamic, and robust to uncertainties in supply chain environments that companies might face in demands, outperforming static policy DFO and linear programming.

As a future work, we would like to further fine-tune hyper-parameters of PPO and A3C, and run experiments for different supply cycles. It is also worth to study the performance of the model with a slowly changing inner parameters of supply chain. This allows to model seasonal demand change and inflation and brings the model closer to real life.

We have already tested the algorithms when we change the parameters of demand distributions and find out that the methods such as PPO and A3C can fastly adapt. Analysing whether the models trained on one distribution can also adapt well to the other types of distributions is worth investigating since companies encounter this behavior in real-life scenarios. Furthermore, it would be interesting to test the mixture distributions, e.g. the Gaussian one, which can represent multiple changes in the demand through the year.

REFERENCES

[1] Reinforcement Learning: An Introduction *MIT Press, Cambridge, MA*, 2018.

[2] Sheng, Z.: Case-based Reinforcement Learning for Dynamic Inventory Control in a Multi-agent Supply-chain System. *Expert Systems with Applications 36, 6520-6526*, Vol 36.

[3] Bertsimas et al. A Robust Optimization Approach to Inventory Theory. Operations Research. 54. 150-168. 10.1287/opre.1050.0238, 2006

[4] Chu et al. Simulation-based optimization framework for multi-echelon inventory systems under uncertainty. Computers and Chemical Engineering. ISSN 00981354. doi: 10.1016/j.compchemeng.2014.10.008, 2015

[5] Mortazavi, A. Arshadi Khamseh, and P. Azimi. Designing of an intelligent self-adaptive model for supply chain ordering management system. http://dx.doi.org/10.1016/j.engappai.2014.09.004, 2015

[6] Simchi-Levi and Y. Zhao. Performance Evaluation of Stochastic Multi-Echelon Inventory Systems: A Survey. Advances in Operations Research. https://doi.org/10.1155/2012/126254, 2012

[7] Medium Article *https://medium.com/@henrymao/reinforcement-learning-using-asynchronous-advantage-actor-critic-704147f91686*, 2020.

[8] Volodymyr Mnih et al. Asynchronous Methods for Deep Reinforcement Learning. http://arxiv.org/abs/1602.01783, 2016

[9] Lillicrap et al. Continuous control with Deep Reinforcement Learning. https://arxiv.org/abs/1509.02971, 2015

[10] Schulman et al. Proximal Policy Optimization Algorithms. http://arxiv.org/abs/1707.06347, 2017

[11] Mania et al. Simple random search provides a competitive approach to reinforcement learning http://arxiv.org/abs/1803.07055, 2018

[12] ARS Lecture *https://cse.buffalo.edu/ avereshc/rl_spring20/*, 2020.

[13] O. Vinyals et al. StarCraft II: A New Challenge for Reinforcement Learning. https://arxiv.org/abs/1708.04782, 2017.

[14] Shulman et al. Proximal Policy Optimization Algorithms https://arxiv.org/pdf/1707.06347.pdf, 2017

[15] Larson et al. Derivative-free optimization methods https://arxiv.org/abs/1904.11585, 2017

[16] Supply Chain Optimization with RL *https://towardsdatascience.com/deep-reinforcement-learning-for-supply-chain-optimization-3e4d99ad4b58*, 2020.

[17] Virtanen et al. Scipy library: Fundamental Algorithms for Scientific Computing in Python. https://arxiv.org/abs/1904.11585, 2020

[18] Hubbs C. D. et al. OR-Gym: A Reinforcement Learning Library for Operations Research Problem //arXiv preprint arXiv:2008.06319. – 2020.

[19] A Graphic Guide to Implementing PPO for Atari Games *https://towardsdatascience.com/a-graphic-guide-to-implementing-ppo-for-atari-games-5740ccbe3fbc*, 2020.

VII. APPENDIX

A. *Reproducibility of Experiments*

1) PPO: In order to replicate the experiments above, we do provide the implementation of each agent separately on the link mentioned in the footer. For PPO agent, A3C, and ARS, we used in the last experiments RLLib library of RAY package to speed the process of training. RLlib is an open-source library for reinforcement learning that offers both high scalability and a unified API for a variety of applications.

```
----------------------------------------------------------------------------------------------------
Model: "model"
----------------------------------------------------------------------------------------------------
Layer (type)                    Output Shape         Param #     Connected to
====================================================================================================
observations (InputLayer)       [(None, 33)]         0
----------------------------------------------------------------------------------------------------
fc_1 (Dense)                    (None, 256)          8704        observations[0][0]
----------------------------------------------------------------------------------------------------
fc_value_1 (Dense)              (None, 256)          8704        observations[0][0]
----------------------------------------------------------------------------------------------------
fc_2 (Dense)                    (None, 256)          65792       fc_1[0][0]
----------------------------------------------------------------------------------------------------
fc_value_2 (Dense)              (None, 256)          65792       fc_value_1[0][0]
----------------------------------------------------------------------------------------------------
fc_out (Dense)                  (None, 6)            1542        fc_2[0][0]
----------------------------------------------------------------------------------------------------
value_out (Dense)               (None, 1)            257         fc_value_2[0][0]
====================================================================================================
Total params: 150,791
Trainable params: 150,791
Non-trainable params: 0
```

Fig. 14. PPO Agent : Policy Network Architecutre

with *"Entropy Coeff": 0.0, "clip Param": 0.3, "VF Clip Param": 10.0* Coefficient of the entropy regularizer, PPO clip parameter Clip param for the value function respectively.

2) REINFORCE: As regarding, the Reinforce algorithm we provide implementation with the fine-tuned parameter of the learning rate α. We observe that the learning process and results obtained depend significantly on α. With tried different parameters of α between 10^{-5} and 10^{-3} and found that $\alpha = 2 * 10^{-4}$ allows both fast convergence and stability of the obtained rewards in comparison to other rates (e.g. for $\alpha = 10^{-5}$ we observe that the algorithm does not converge in 1000 episodes while for $\alpha = 10^{-3}$ obtained results are very unstable).

B. *Increasing Levels Parameters*

Here below are the parameters used for Experiment: *Effect of Increasing Supply Chain Levels*

TABLE III
4 LEVELS

Initial inventories	[100, 100, 200, 100]
Unit cost	[1.5, 1.0, 1.0, 0.75, 0.5]
Backlog cost	[0.10, 0.075, 0.05, 0.025, 0.025]
Unit holding cost	[0.15, 0.10, 0.05, 0.025]
Production capacities	[100, 90, 80, 70]
Lead times	[3, 5, 5, 10]

TABLE IV
7 LEVELS

Initial inventories	[100, 100, 200, 200, 100, 200, 100]
Unit cost	[1.5, 1.0, 1.0, 0.75, 0.5, 0.5, 0.5, 0.4]
Backlog cost	[0.10, 0.075, 0.05, 0.025, 0.015, 0.010, 0.008, 0.008]
Unit holding cost	[0.15, 0.10, 0.05, 0.05, 0.035, 0.025, 0.020]
Production capacities	[100, 90, 80, 70, 70, 50, 40]
Lead times	[3, 5, 5, 5, 5, 5, 10]

C. *Actions of Environment with Backlog Cost*

Here we print the actions taken DFO and PPO at episode 600, for environment-v0 in order to backup our conclusion about the experiment D.

TABLE V
9 LEVELS

Initial inventories	[50, 100, 100, 100, 100, 100, 300, 300, 300]
Unit cost	[1.5, 1.0, 1.0, 0.75, 0.5, 0.5, 0.5, 0.4, 0.4, 0.4]
Backlog cost	[0.10, 0.075, 0.05, 0.025, 0.015, 0.010, 0.008, 0.006, 0.004, 0.004]
Unit holding cost	[0.15, 0.10, 0.05, 0.05, 0.035, 0.025, 0.020, 0.20, 0.015]
Production capacities	[100, 90, 80, 70, 70, 50, 50, 40, 40]
Lead times	[3, 5, 5, 5, 5, 5, 5, 5, 10]

1) DFO: [([100., 16., 0.]), ([0., 18., 0.]), ([0., 25., 0.]), ([0., 16., 0.]), ([0., 23., 0.]), ([0., 23., 0.]), ([16., 20., 0.]), ([18., 19., 0.]), ([25., 17., 0.]), ([16., 15., 0.]), ([23., 8., 0.]), ([23., 0., 0.]), ([20., 0., 0.]), ([19., 0., 0.]), ([17., 0., 0.]), ([15., 0., 0.]), ([8., 0., 0.]), ([0., 0., 13.]), ([0., 0., 22.]), ([0., 0., 17.]), ([0., 0., 24.]), ([0., 0., 5.]), ([0., 0., 0.]), ([0., 0., 0.]), ([0., 0., 0.]), ([0., 0., 0.]), ([0., 0., 0.]), ([0., 0., 0.]), ([0., 13., 0.]), ([0., 22., 0.])]

2) PPO: [([0.52286303, 0.68892246, 0.4921896], ([1.3905725, 2.1123915, 0.9492917], ([0.9410382, 1.0093292, 0.6430703], ([0.7951418 , 0.05905469, 1.6123112], ([3.0217667 , 1.9479815 , 0.34838277], ([0.52982444, 0. , 0.], ([0. , 0. , 2.229197], ([0.300132 , 0.39699957, 0.20475984], ([0. , 1.543291, 0.], ([0.8943465, 0.9288419, 0.], ([0.5524455 , 0.09961385, 0.39205688], ([0. , 0. , 0.320917], ([1.3139722 , 0. , 0.90685874], ([0.20656542, 0. , 1.0083854], ([0.43014556, 0. , 0.], ([0. , 0.95464754, 0.], ([0.5597936 , 1.1372159 , 0.91059583], ([0. , 0. , 0.16859925], ([0., 0., 0.], ([0.4626945 , 0.5928247 , 0.86391056], ([0. , 1.455352 , 0.08611251], ([0. , 0.11960514, 0.5693756], ([0.7901925, 0. , 0.637755], ([0.09809983, 1.9997021 , 0.2264214], ([0.27297372, 0.7112419 , 2.6755192], ([1.2634865, 1.0325421, 0.], ([0. , 0. , 0.16839585], ([0.67020756, 0.28818676, 1.6907041], ([0., 0., 0.], ([0.43427277, 0.76449335, 0.]]

D. *Code*

We attached our report, with all results from the experiments conducted for each model as *Pickle files*. Each agent developed in independent Py file. Please, make sure to install the required packages from YML file, before re-running the code.

6 Sustainable Cities and Human Settlements: Project Course Logics & Symbolic AI

Symbolic AI and Logics (IA301)

Sustainable Cities and Human Settlements

Sini SURESH
Imad Eddine MAROUF
Anne-Claire Maréchal
Supervisor : Dr. Natalia Díaz Rodríguez
Telecom-Paris Engineering School, France

***Abstract*—Cities are growing with unseen rate, providing better living conditions at the cost of nature. Which rises the worry, therefore many association are devoted for the aim to provide suitable plans for goverments to create smart, sustainable cities. One of them, is the Agenda2030 published by the UN containing 17 objectives. My team decided to work on GOAL11. Throughtout, this project we built an ontology which is a recommendation model to help urban planning and ensure transportation and energy sustainability in cities.**

Code provided on GitHub: https://github.com/IemProg/IA301_SymbolicAI_Ontologies

Introduction

United Nations General Assembly has developed a plan of action "Agenda 2030" to ensure that all the world's people can live in dignity, freedom and in a healthy environment. This plan is divided into 17 goals in order to ensure sustainable development in many domains such as Education, and Economic Growth.

We would like to focus throughout this project on Goal 11 "make cities and human settlements inclusive, safe, resilient and sustainable." Making cities sustainable means creating business opportunities, safe and affordable housing, resilient societies and environoment-friendly building. Recent surveys [5] has showed that in 2018 about 4.2 billion people are living in cities, expecting to reach 6.5 billion by 2050. Many issues has arised due to that regarding the pollution, resources management and resilience in case of natural disasters. In order to tackle the objetive of creating sustainable cities, the UN has defined 17 objectives and 15 Indicators representing the metrics by which the world aims to track whether these objectives are achieved [5].

I. Gathering Knowledge Requirements

Developing an ontology for this domain rather than using other AI methods is built upon many reasons such the ability to share common understanding of the structure information between people and software agents, ease to embed domain assumptions in explicit manner, and the ability to analyze domain knowledge since it well structured in comprehensible manner.

A. User Study

Metrics to evaluate whether a city is sustainable or not are very vague, and differes from a report to another, but the common metrics were carbon-neutrality, energy management, areas of green spaces, and effeciency of the transportation system. Those properties were shared between the most sustainable cities in the world according to many studies (1. Copenhagen, Denmark, 2. San Francisco, California, 3. Vancouver, Canada)[6]

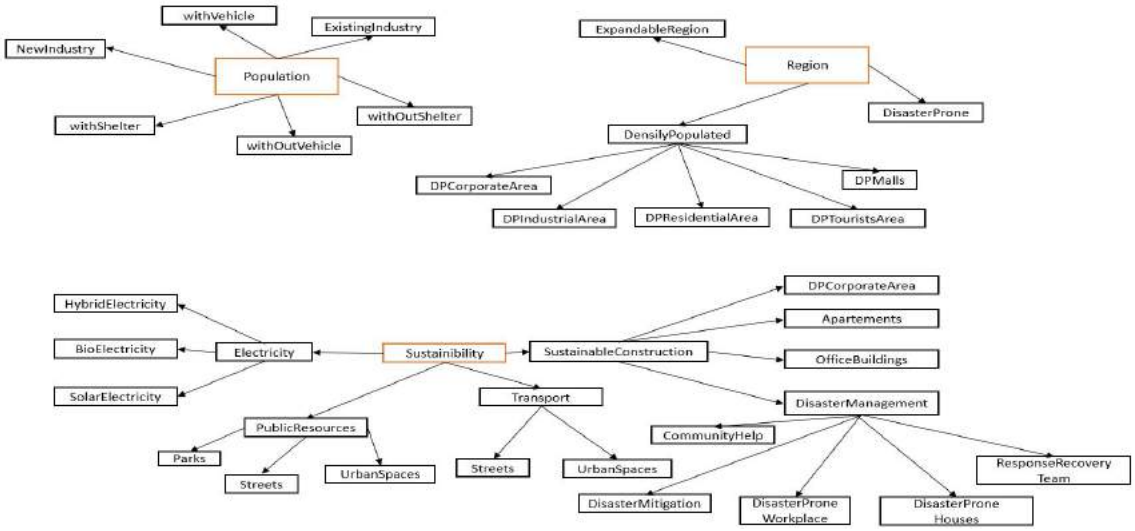


Figure 1: Diagram describing the classes of the ontology

In this project, we will focus mainly on the housing, energy management and the transportation issues. By building ontologies to recommend decisions which reflects the optimal trade-off between the different sections.

In the interest of, representing knowledge about sustainable cities we would like to build an ontology to take decisions for the following inquiries:

1) A recommendation model to help urban planning and ensure sustainability of cities and human settlements.
2) Ensures shelter for everyone in the city and also ensures that any new construction (including residential and industrial) happens only in the expandable regions of the city.
3) Recommends an urban system where disaster prone regions are a part of disaster management where community help/ disaster mitigation/ response and recovery teams are active and assigned.
4) All the existing residential and industrial areas are provided with sustainable resources.

II. Modeling

Building ontologies about sustainable cities is has many sections and variables that we should take into consideration. Sustainable cities means we have an integrated enviromental infrastruture (water cleaning, waste management) as well very dynamic and carbon-neutral transportation system.

Therefore, in this project we would like to encapsulate ontologies in each sector in order to build a broad graph for a sustainable city. As starting point, we develop independant ontologies capable of providing inference towards a specific domain such as transportation, energy management and housing services.

After starting the project, we have noticed that it is quite wide and require further much analysis and time. Thus, we will focus mainly on a broader graph for the sustainable city with its implementation and provide a draft for transportation sector in the last sections.

A. Overall Ontology

We have to define the main concepts of the domain that we would like to model, after discussions we agreed on 3 main classes which are: *Regions*, *Population* and *Sustainibility*

The major concerns of sustainable living among human population is Electricity, Transportation, Construction, Availability of Public resources and Sustainable Recreation Activities - hence the classes branched so under Sustainability.

Sub classes of *Population* are majorly classified as WithShelter and WithoutSheter to ensure Shelter for all and also ensure that the existing Shelter has sustainable features. Similarly, the workplaces are categorized as NewIndustry and ExistingIndustry. Modes of *transportation* is another major concern of Population. Hence the entire Population is classified as WithVehicle and WithoutVehicle, both the modes ensures sustainable transportation.

The Regions are broadly classified as DenselyPopulated, DisasterProne and ExpandbleRegions. DenselyPopulated regions are classified and further the classification is inferred from the given model. The model ensures that DenselyPopulated regions have sufficient sustainability features. Any new construction is based in ExpandableRegion which has subclasses: IndustrialArea and ResidentialArea. Subclasses of DisasterProne area include Houses and Workplace ensures it has sustainable disaster management features.

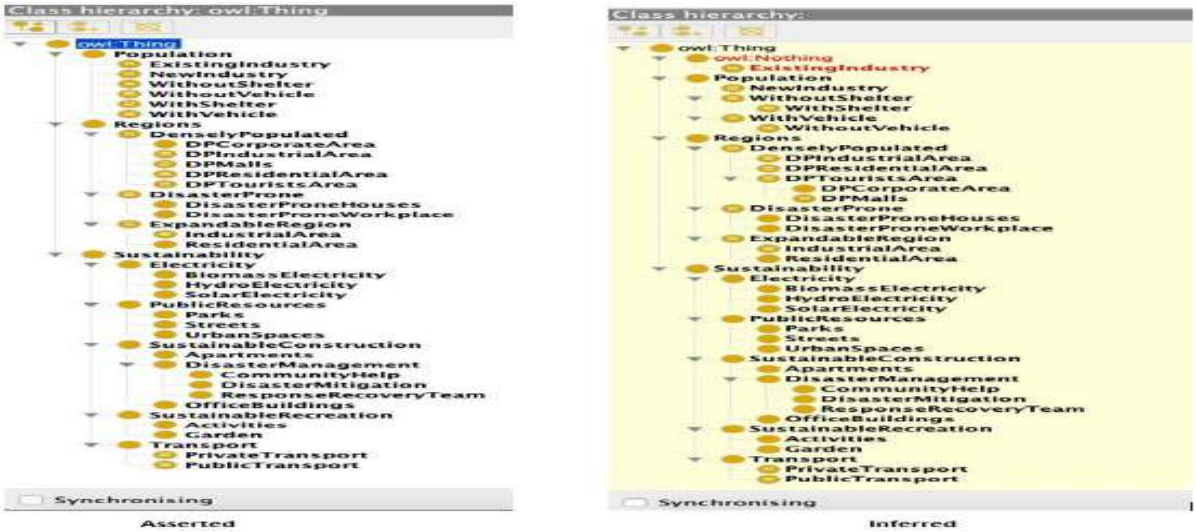


Figure 2: Inferred classes by the reasoner

B. Transportation Sector

In this section, we tried to develop an ontology for the transportation sector, since it is the most import domain, and offers wide of advatanges for city development based on similar work done [7]. We would like to take advantage of information provided by sensors (Internet Of Things) placed over different places in the city in order to monitor the flow of vehicles taking into consideration the physical conditions like weather, roads ..etc.

The ontology for transportation could be divided into smaller modules as follow:

1) Data Module: The data module contains the measures and information provided by different sensors distribued across the city in order to be used afterwarsds for processing and taking decisions. The data provided by sensors could be about: Temperature, Humidity, Wind speed and direction, Cloudiness, Rain and snow volumes.

The goal is to have a dashboard enabling analysis of data with weather measures. The main data source is relative to the traffic and vehicles flow. Each vehicle is periodically detected on the route and its most important features are stored. We can make use of Semantic Sensor Network ontology (SSN) provided by W3C [2] which can model sensor devices, systems, processes, observations and knowledge of the environment.

2) Transport Module: The automatic adjustment of the duration of the traffic lights contributes to optimizing the traffic flow, which allows drivers to gain time along the route. To carry out this process, the system takes into account only the level of congestion and the weather conditions of the corresponding road segment.

Ontologies can describe the current traffic situation, including the location of vehicles, infrastructure elements and sensors; and it allows us to analyze semantically the data collected by the sensors. One of the advantages of our proposal is that agents can know the overall traffic situation at all times by querying the ontology, instead of having only a partial view (and therefore imprecise). This allows greater decision-making capability based on the real circumstances, and that agents can work together using the same knowledge base.

The figure above 3, shows the taxonomy of transportation modes, which can be classified into Private, Public and Priority transport, those concepts will be used to create the ontology for the transportation system in order to maximize the economic benefits of the drivers (choosing the cheaper routes and/or transportation means, the fastest or shortest route), and to minimize the pollution to the environment (in terms of CO2 emissions).

The parameters fed by information provided by the sensors (Data module) such : the price features and the sustainable features related to CO2 emissions per passenger and per product volume. In addition these arguments are combined with the GPS Location, Number of Seats in order to infer an optimized recommendation to the driver.

Furthermore, we have to define relations between the group of vehicles to exchange knowledge, the proposed relations are defined as follows : hasVehicleType(vehicle, vehicleType), hasDriver(vehicle, driver), hasLicence(driver, licence), canDrive(licence, vehicleType), hasRoute(vehicle, route), hasRoutePoint(route, routePoint), hasLocation(vehicle,

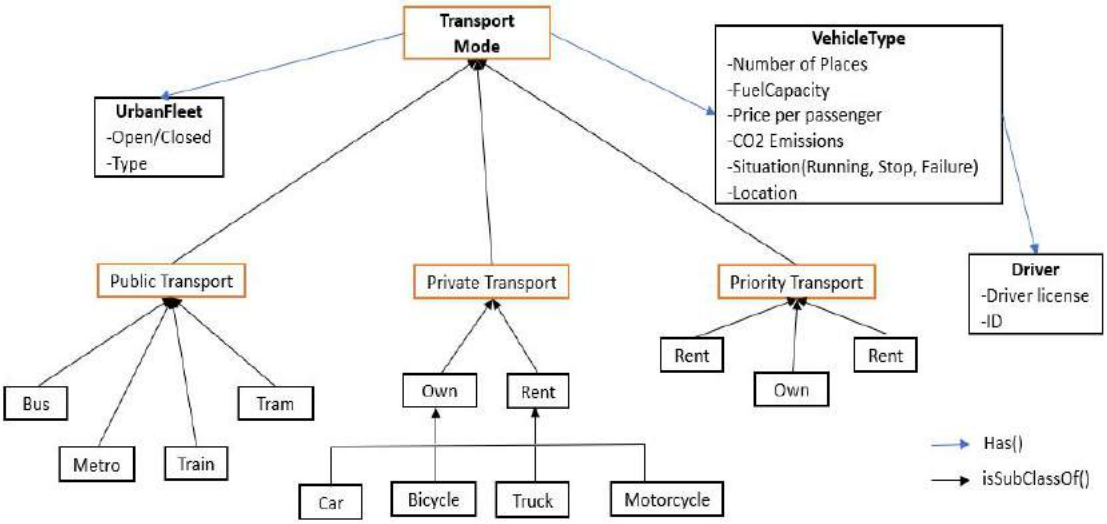


Figure 3: Extended Class for Trasportation System

location), hasLocation(routePoint, location), hasLocation(weatherEvents, location), hasAction(vehicle, action), hasAction(routePoint, action).

Providing decisions based on the sensor's data, we need to have an agent which can perform tasks according to the knowledge representation in the Transport Module and Data Module such as: configuring the red light duration and speed limit, with respect to the weather conditions and the traffic flow. In a similar way to work [8], where the agent could be developed by Java Agent Development Framework (JADE), and an ontology query language SPARQL could be used to process the queries.

We have realized that this task is very complex and may be beyond the scope of the course, thus we could not realize it.

III. Results

The model is successful in classifying regions/places and also an extension of this project will cover all the goals mentioned such as any new construction classified under industrial/residential area in expandable regions, disaster management measures specific to different classes. It has 3 main classes: 3 with 174 axioms: 174. For further details refer to figure 4

At the time being is intuitive, and very limited as but it represents a potential model which satisfies all the criteria of sustainable living in cities.

Conclusion

In this project, we learnt how to develop onlotogy using Protége software as well some domain knowledge regarding sustainable cities. We provide an implementation of it, based on some factors made by UN.

Furthermore, we had concluded to model this domain we have to apply button-up method by providing ontology for each sector (housing, transportation, and energy) due to vast literature knowledge provided in each, then we embed them into a single graph which provided good abstractions based on the different sectors.

References

Project Handout:

[1] Dr. Diaz Rodiriguez, IA301 (2019) Project instructions and Evaluation.pdf.

Articles:

[2] Semantic Sensor Network ontology

[3] Explanation Ontology

[4] An ontology-based monitoring system for multi-source environmental observations, (KES2018), Sep 2018, Belgrade, Serbia. pp.1865-1874. ffhal-02112038

[5] SDG-Tracker for Cities

[6] Most sustainable cities in the world

[7] Ontologies for transportation research: A survey

[8] The Application of Ontologies in Multi-Agent Systems in the Energy Sector: A Scoping Review

[9] Sustainable Development UN Web page

[10] Human Settlements Risks

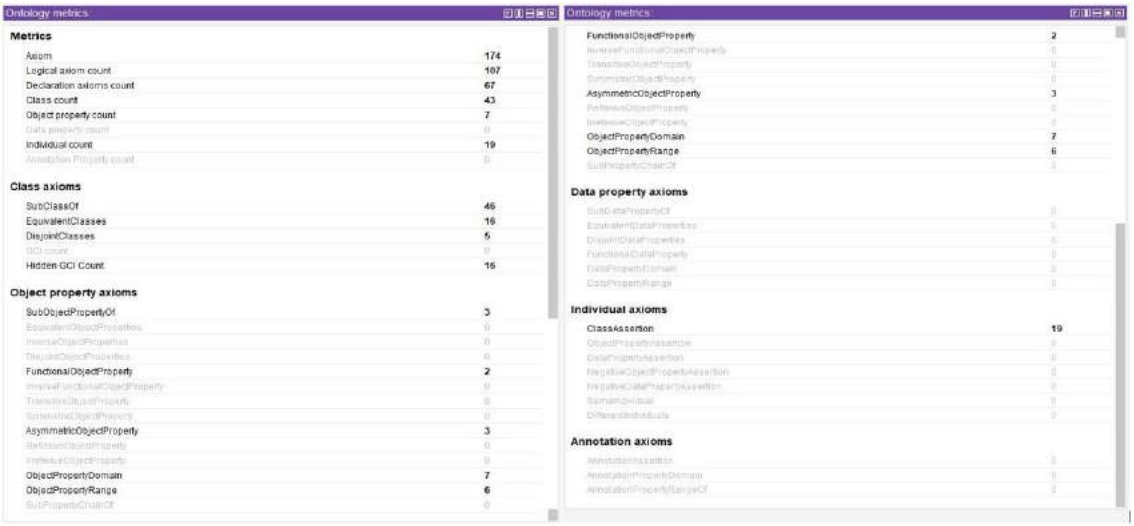


Figure 4: Metrics